Next-Generation Speculative Decoding

EAGLE-3, Parallel Drafting (P-EAGLE), and
Speculative Speculative Decoding (SSD / Saguaro)
for LLM Inference

A mathematical, systems, and empirical review of developments from October 2025 to October 2026

Research report compiled October 27, 2026

How to read the evidence labels. Every quantitative statement in this report carries one of four provenance labels.

- [S] **Source-reported**: a number taken from a paper, documentation page, or table that was retrieved while preparing this report. Numbers are quoted or paraphrased; nothing is re-measured.
- [D] **Derivation**: a mathematical result derived (or re-derived and checked) in this report.
- [M] **Our simulation**: a number produced by the code listed in Appendix B. These are toy or modelled experiments, *not* measurements on real LLM serving stacks.
- [V] **Vendor / blog / anecdotal**: claims from vendor blog posts, press-style pages or forum posts, which were not independently verified.

The author of this report is an AI research assistant (Claude). No experiments on real GPUs were run; see Section 1.6 for the limits of this study.

Abstract

Autoregressive decoding of large language models is bound by memory bandwidth, not by arithmetic: each generated token requires streaming essentially all model weights through the accelerator. Speculative decoding converts spare arithmetic capacity into lower latency by letting a cheap *drafter* propose several tokens that the large *target* model verifies in a single forward pass, with a rejection-sampling rule that leaves the output distribution exactly unchanged. This report reviews the state of the art as of late October 2026, with emphasis on three developments: (i) **EAGLE-3**, which abandons feature regression in favour of direct token prediction trained with a "training-time test" and fuses low-, mid- and high-level target features; (ii) **parallel drafting** (PARD, P-EAGLE, and diffusion-style drafters such as DFlash), which replaces K sequential draft passes by one; and (iii) **speculative speculative decoding (SSD)**, whose Saguaro algorithm removes the remaining serial dependency between drafting and verification by pre-computing continuations for many possible verification outcomes on separate hardware.

The report has four contributions. First, a self-contained mathematical treatment: the speculative-sampling acceptance rule, a full proof that the output law equals the target law, the residual distribution, the expected number of tokens per step $(1-\alpha^{\gamma+1})/(1-\alpha)$, the wall-clock speedup with draft-cost coefficient c, optimal draft length, multi-draft and token-tree verification, a roofline analysis of when batching erases the gain, and an analysis of SSD's overlap model including a Lagrangian derivation of the geometric fan-out law. Second, a set of runnable numerical experiments whose real outputs are reported: Monte-Carlo confirmation of distributional equivalence (and failure of three broken variants), confirmation of every closed form, speedup surfaces, tree-versus-chain trade-offs under a roofline cost model, and simulations of the SSD timing model. Third, a critical review of published empirical results, with provenance flags: EAGLE-3 reports up to 6.5× speedup at batch size one and a 1.38× throughput gain at batch size 64 in SGLang; P-EAGLE reports 1.10–1.36× over autoregressive EAGLE-3 in vLLM and 1.05–1.69× on one B200 in a vendor blog; Saguaro reports about 30% average gain over strong speculative-decoding baselines and up to 5× over autoregressive decoding at batch size one on four H100s plus a fifth for the drafter. Fourth, a discussion of failure modes, production reception, and open problems, including the finding of an MLSys 2026 study that, in a production engine, verification dominates run time and measured speedups fall well short of theoretical bounds as batch size grows.

Key caveats. Headline speedups are overwhelmingly measured at small batch sizes, on a few model families, mostly by the method authors. SSD uses additional hardware. Several quoted figures come from vendor blogs. Our own simulations validate the mathematics but do not substitute for measurements.

Contents

Abstract **1**

1 Introduction **15**

1.1 Why decoding latency is a hard problem . . . 15
1.2 Speculative decoding in one paragraph . . . 15
1.3 The three developments this report focuses on . . . 15
1.4 Scope and timeline of the period under review . . . 16
1.5 Contributions and organization . . . 16
1.6 Methodology and limits of this study . . . 17

2 Background: Autoregressive Decoding and the Memory Wall **18**

2.1 Notation . . . 18
2.2 What a decode step costs . . . 18
2.2.1 Parameters, FLOPs and bytes . . . 18
2.2.2 Arithmetic intensity and the roofline . . . 18
2.2.3 A reality check against published numbers . . . 19
2.3 Why batching helps throughput but hurts speculation . . . 20
2.4 The KV cache and long contexts . . . 20
2.5 Mixture-of-experts models . . . 20
2.6 Sampling, temperature and what “lossless” means . . . 21
2.7 Prefill versus decode . . . 21

3 Speculative Sampling: Algorithm and Correctness **22**

3.1 Single-position speculative sampling . . . 22
3.2 Speculative sampling of a block of γ tokens . . . 23
3.2.1 Deterministic drafts . . . 24
3.3 Optimality of the acceptance rule . . . 24
3.4 Relationship between acceptance and divergences . . . 25
3.5 Monte-Carlo verification . . . 25
3.6 Pitfalls: when “lossless” implementations are not . . . 26

4 Expected Tokens, Speedup, and Optimal Draft Length **28**

4.1 Expected number of tokens per target pass . . . 28
4.1.1 Distribution and variance of N . . . 28
4.2 Wall-clock speedup . . . 29

4.2.1 Reproducing the original table . . . 30
4.2.2 Operation counts and energy . . . 30
4.3 Optimal draft length . . . 30
4.3.1 Numerical optimum . . . 30
4.3.2 A closed form via the Lambert W function . . . 31
4.4 Beyond i.i.d.: position decay . . . 32
4.5 Beyond i.i.d.: heterogeneity across contexts . . . 33
4.6 Worked examples . . . 33
4.7 Throughput, latency and goodput . . . 34

5 Roofline Analysis: Why Speculation Works and When Batching Erases the Gain 36
5.1 A step-time model . . . 36
5.2 A closed form for the weights-only regime . . . 36
5.3 Full simulation with context and drafting cost . . . 37
5.4 Verification time and the step-time curve . . . 39
5.5 Adaptive speculation under load . . . 39
5.6 Long contexts and the MagicDec effect . . . 40
5.7 Mixture-of-experts: verification touches more weights . . . 41
5.8 Summary of the regime picture . . . 41

6 Multiple Candidates and Token-Tree Verification 43
6.1 Tree attention in one paragraph . . . 43
6.2 Exact multi-candidate verification . . . 43
6.2.1 Recursive rejection sampling . . . 43
6.2.2 With-replacement versus without-replacement candidates . . . 44
6.2.3 Trees of depth greater than one . . . 45
6.3 Expected acceptance length of a tree . . . 45
6.3.1 A path-sum formula . . . 45
6.3.2 Optimal trees under a node budget . . . 45
6.4 A simulated comparison of trees and chains . . . 46
6.4.1 Trees meet the batch size . . . 46
6.5 Combining trees with the other ideas . . . 48

7 History and Taxonomy of Speculative Decoding 49
7.1 Prehistory: blockwise parallel decoding and speculative execution . . . 49
7.2 The founding papers (2022–2023) . . . 49
7.3 Trees, heads, and self-speculation (2023–2024) . . . 50
7.4 Overlapping drafting and verification (2024–2025) . . . 51
7.5 Built-in multi-token prediction and production adoption . . . 51

7.6 The review window (Oct 2025 – Oct 2026) in brief . . . 51
7.7 A taxonomy . . . 52

8 The EAGLE Family and EAGLE-3 53
8.1 EAGLE: autoregression at the feature level . . . 53
8.2 Why EAGLE stopped scaling with data . . . 53
8.3 EAGLE-3: direct token prediction and the training-time test . . . 54
8.3.1 Two changes . . . 54
8.3.2 Inference pipeline . . . 54
8.3.3 Training-time test . . . 54
8.4 Where drafting time goes: the output layer . . . 55
8.5 Published results . . . 55
8.5.1 Headline speedups and ablation . . . 55
8.5.2 Scaling with data . . . 56
8.5.3 Throughput at larger batch sizes . . . 56
8.5.4 Compared with a standalone draft model (a counter-example) . . . 57
8.6 Summary and critique . . . 57

9 Parallel Drafting: Medusa, PARD, P-EAGLE and Diffusion Drafters 58
9.1 The cost of sequential drafting . . . 58
9.2 What a parallel drafter can and cannot predict . . . 58
9.3 Medusa heads and ParallelSpec . . . 59
9.4 PARD: target-independent parallel drafts . . . 59
9.5 P-EAGLE . . . 59
9.5.1 Architecture . . . 59
9.5.2 Training recipe and capacity compensation . . . 60
9.5.3 Scalable training for long reasoning traces . . . 60
9.5.4 Acceptance length and throughput results . . . 61
9.5.5 An illustrative cost model . . . 62
9.6 DFlash: block-diffusion drafting . . . 63
9.7 Summary . . . 63

10 Other Drafter Improvements: Vocabulary, Future Signals, MTP, Training-Free and Adaptive Drafters 65
10.1 The output-layer bottleneck and speculative vocabularies . . . 65
10.1.1 Why the LM head matters . . . 65
10.1.2 A coverage bound . . . 65
10.1.3 SpecVocab . . . 66
10.1.4 A toy coverage experiment . . . 66

10.2 Future-aware drafting: ConFu . . . 67
10.3 Multi-token prediction as a built-in drafter . . . 68
10.4 Training-free drafters: prompt lookup, n-gram and suffix decoding . . . 69
10.4.1 A proxy experiment on real text . . . 69
10.5 Adaptive and online drafters . . . 70
10.5.1 Choosing the lookahead: goodput . . . 70
10.5.2 Learning from live traffic . . . 70
10.5.3 Scaling laws for drafter size . . . 70
10.6 Self-speculation and early exit . . . 71
10.7 A comparison of drafter families . . . 71
10.8 Summary . . . 71

11 Speculative Speculative Decoding (SSD) and Saguaro 73
11.1 Setting and definitions . . . 73
11.2 Exactness . . . 73
11.3 Speedup theory . . . 74
11.3.1 Numerical verification of the theory . . . 76
11.4 Challenge 1: predicting verification outcomes (the Saguaro cache) . . . 77
11.4.1 The fan-out problem . . . 77
11.4.2 Optimal geometric fan-out . . . 77
11.5 Challenge 2: balancing hit rate and acceptance (Saguaro sampling) . . . 78
11.5.1 A toy replication of the trade-off . . . 79
11.6 Challenge 3: handling cache misses (Saguaro fallback) . . . 79
11.7 Implementation and cost . . . 81
11.7.1 System design . . . 81
11.7.2 Overheads . . . 81
11.8 Combining with EAGLE-3 and token trees . . . 82
11.9 Published results . . . 82
11.9.1 Main results . . . 82
11.9.2 How strong are the baselines? . . . 82
11.9.3 Independent reproduction attempts . . . 83
11.10 Relation to concurrent and earlier overlap methods . . . 84
11.11 Critical assessment . . . 84
11.12 Summary . . . 85

12 Serving Systems: vLLM, SGLang, TensorRT-LLM and the Training Ecosystem 86
12.1 What an engine must do . . . 86
12.2 vLLM . . . 87

12.3 SGLang and the SpecForge ecosystem . . . 88
12.4 TensorRT-LLM . . . 88
12.5 llama.cpp, Ollama and edge runtimes . . . 89
12.6 Memory accounting . . . 89
12.7 An instrumentation checklist . . . 90
12.8 Summary . . . 90

13 Consolidated Empirical Evidence **92**
13.1 How strong is the evidence? . . . 92
13.2 The independent study in detail . . . 92
13.3 Speedup versus load across sources . . . 94
13.4 A claims audit . . . 95
13.5 Parallel drafting and SSD: complements or substitutes? . . . 95
13.6 Does "lossless" hold in practice? . . . 97
13.7 Gaps in the evidence . . . 97

14 Speculative Decoding versus Other Routes to Fast Inference **99**
14.1 Where each lever acts . . . 99
14.2 Diffusion language models . . . 99
14.2.1 What they are and what is claimed . . . 99
14.2.2 Conceptual comparison . . . 100
14.2.3 Diffusion as a drafter . . . 100
14.3 Asynchronous and disaggregated designs . . . 101
14.4 Per-device accounting . . . 101
14.5 A decision guide . . . 101
14.6 Summary . . . 102

15 Advantages, Disadvantages and Failure Modes **105**
15.1 Advantages . . . 105
15.2 Disadvantages . . . 106
15.3 A catalogue of failure modes . . . 106
15.3.1 Illustration: sensitivity of acceptance to the sampling and verification distributions . . . 108
15.3.2 Grammar-constrained decoding: a short derivation . . . 109
15.4 Reading the risk . . . 109
15.5 Summary . . . 109

16 Community Perception and Reception **110**
16.1 What we looked for and what we found . . . 110

16.2 Academic reception . . . 110
16.3 Public discussion . . . 111
16.3.1 Hacker News: SSD . . . 111
16.3.2 r/LocalLLaMA: EAGLE-3 in llama.cpp . . . 112
16.3.3 Practitioner posts . . . 112
16.4 Vendor and platform posture . . . 113
16.5 Recurring beliefs, checked against the evidence . . . 113
16.6 Summary . . . 113

17 Case Studies and Worked Examples 115
17.1 Reported deployments . . . 115
17.1.1 LinkedIn: n-gram speculation for an agent workload . . . 115
17.1.2 Google Vertex AI with SGLang: EAGLE-3 serving . . . 115
17.1.3 Gemma 4 multi-token-prediction drafters . . . 115
17.1.4 AMD Instinct with vLLM . . . 116
17.1.5 Together AI ATLAS: an adaptive speculator . . . 116
17.1.6 Speculative decoding for Llama at scale . . . 116
17.1.7 MoE-Spec: expert budgeting in verification . . . 116
17.1.8 AgentSpec: batched agents . . . 116
17.1.9 DeepSeek-V3 and GLM-4.5-Air: native MTP heads . . . 117
17.2 Worked sizing examples . . . 117
17.2.1 Choosing the draft length . . . 117
17.2.2 Break-even . . . 117
17.2.3 How much can a better drafter be worth? . . . 118
17.2.4 Speculative speculative decoding: when does it pay? . . . 118
17.2.5 A sizing checklist . . . 118

18 Open Problems and Research Directions 119
18.1 Evidence gaps . . . 119
18.2 Algorithmic questions . . . 119
18.3 Systems questions . . . 120
18.4 What would change our conclusions . . . 121

19 Conclusions 122
19.1 What the evidence supports . . . 122
19.2 What is not established . . . 122
19.3 Practical guidance . . . 123
19.4 Closing remark . . . 123

A Proofs and Derivations **124**

A.1 Exactness of speculative sampling . . . 124

A.2 Expected tokens per step . . . 124

A.3 Break-even for $\gamma = 1$ . . . 125

A.4 Optimal draft length via the Lambert W function . . . 125

A.5 Jensen effect of heterogeneous acceptance . . . 125

A.6 Variance of tokens per step . . . 126

B Simulation Code and Results **127**

B.1 Summary of experiments . . . 127

B.2 Shared helpers . . . 127

B.3 exp0_walkthrough.py . . . 128

B.4 exp1_lossless.py . . . 128

B.5 exp2_formulas.py . . . 130

B.6 exp3_roofline.py . . . 131

B.7 exp3b_piecewise.py . . . 133

B.8 exp4_trees.py . . . 133

B.9 exp5_ssd.py . . . 135

B.10 exp6_parallel_draft.py . . . 137

B.11 exp7_failure.py . . . 138

B.12 exp9_extras.py . . . 139

B.13 exp10_vocab.py . . . 140

B.14 exp11_ngram.py . . . 140

B.15 exp12_substitutes.py . . . 141

C Glossary and Notation **143**

Bibliography **144**

List of Figures

2.1 Roofline schematic for an H100-class GPU using the assumed efficiency factors of our model ([M]). A single-token decode step sits far to the left of the ridge. A speculative step verifying $\gamma + 1 = 5$ tokens moves right but remains memory-bound; the same request inside a batch of 64 verifying five tokens each is near the ridge and has little spare compute. 19

3.1 Left: distance to the exact target joint for the correct algorithm (green) versus three broken variants (red); dashed line is the sampling noise floor ([M]). Right: empirical versus exact probabilities of all 216 three-token sequences for the correct algorithm; points lie on the diagonal. 26

4.1 Speedup surfaces $S(\alpha, \gamma, c)$ for three draft costs ([D]). Contours mark $1\times$ through $4\times$; the orange curve is the optimal γ for each α. As c grows the useful region shrinks and the optimal γ collapses toward 1–3. 31

4.2 Left: speedup against γ for $c = 0.05$ with the optimum marked. Right: the best achievable speedup (envelope over γ) as a function of α for four draft costs ([D]). 32

4.3 Speedup against γ when the conditional acceptance decays geometrically with position, $\alpha_k = 0.85\rho^{k-1}$, $c = 0.05$ ([D]). Stronger decay (smaller ρ) lowers and flattens the optimum. 33

5.1 Modelled throughput speedup of chain speculation ([M]). Left: $\alpha = 0.8$, $\gamma = 4$, for four context lengths; points are plotted only where $BS\kappa$ fits in free HBM (about 60 GB). Right: ctx $= 512$ for four acceptance rates. The dashed line marks no gain. 38

5.2 Published throughput relative to a non-speculative baseline versus batch size, for EAGLE and EAGLE-3 ([S]). Left: SGLang on H100 with Llama-3.1-8B (MT-Bench, chain of length 3, measured by the SGLang team; Table 3 of Li et al. [23]). Right: vLLM with Llama-3.1-8B (chain of length 2; Table 5 of the same paper, whose text and caption disagree on whether the GPU was an RTX 3090 or an A100). . . 39

5.3 Modelled target step time against batch size for 1, 5, 9 and 17 tokens per request (ctx 512; [M]). The curves share a memory-bound floor of roughly 6 ms and leave it at $B \approx 221/m$. 40

5.4 Adaptive versus fixed draft length ([M]). The green curve selects the best $\gamma \geq 0$ at each batch size. 41

6.1 Left: expected tokens per round for best-first trees versus chains as the node budget grows ([M]). Right: modelled throughput speedup versus batch size for five tree budgets under the roofline model of Chapter 5 ([M]). 47

8.1 Baseline sweeps for Llama-3.1-70B at batch size one on 4×H100 ([S]; Tables 1–2 of the Saguaro paper, appendix B.4). Left: SGLang 0.5.9, top-k=3 branch of the grid. Right: vLLM 0.16.0 with chain lookahead K. The dashed line is the autoregressive baseline. 57

9.1 Illustrative cost model contrasting sequential and parallel drafting ([M]). Left: modelled speedup against depth K for $\alpha = 0.78$ (an effective rate implied by P-EAGLE's reported acceptance length, Table 4.6) and single-layer pass cost $c_1 = 0.04$; the sequential drafter costs Kc_1, the parallel drafter costs κc_1 with mild per-depth decay $\rho = 0.97$. Right: ratio of best parallel to best sequential speedup as a function of c_1. 62

10.1 Left: coverage of target probability mass by a static versus context-aware candidate vocabulary in a synthetic setting ([M]; Proposition 10.1 caps acceptance by this coverage). Right: tokens per round for prompt-lookup drafting on real text with the text acting as a deterministic target ([M]; Section 10.4); mean over 60 standard-library modules (code) and four paper texts (prose). 67

11.1 Schematic timelines ([D]; drawn by us, not measured). Autoregressive decoding, speculative decoding (verifier waits for the drafter and vice versa), and SSD in which the drafter runs concurrently on separate hardware and a cache hit eliminates the drafting wait. 74

11.2 Left: gain of SSD over SD with the same drafter, $\alpha = 0.8$, $K = 5$, as a function of the hit probability and of the drafter's relative cost ([M]). Right: modelled speedup versus batch size for a slow backup (primary reused) versus an instantaneous but poor backup; the dotted line is the closed-form crossover $b^\star = 1.61$, the numerical crossover is at $b = 2$ ([M]; toy parameters $E_{\text{hit}} = 4.5$, $E_{\text{miss}} = 1$, $p_{\text{hit}} = 0.97$, $T_p = 0.5$). 77

11.3 Left: fan-out per number of accepted tokens k for $B = 24$, $\alpha = 0.8$, $K = 5$, $r = 0.6$: uniform allocation, the geometric allocation of Theorem 11.6, and the numerical optimum ([M]). Right: hit probability against cache budget B for several r; solid curves geometric, dashed uniform. 79

11.4 Saguaro sampling toy ([M]): acceptance, round hit rate and modelled speedup versus the downweighting constant C (axes reversed so that stronger bias is to the right) for fan-outs 4 and 16. 80

11.5 Re-plot of Table 11.4 ([S]; our plot of the published numbers). 83

13.1 Re-plots of published numbers ([S] left; [V] right). Left: EAGLE-3 in SGLang (EAGLE-3 paper, Llama-3.1-8B, H100, MT-Bench) and the two batch-size endpoints reported in the text of the vLLM study for EAGLE (intermediate points were not retrieved, so lines between endpoints are not data). Right: ratio of P-EAGLE to EAGLE-3 throughput at concurrency 1 and 64 from the vLLM blog (GPT-OSS-20B, B200); the blog reports 1.55, 1.55 and 1.69 at $c = 1$ and 1.05, 1.23 and 1.25 at $c = 64$ for MT-Bench, HumanEval and SPEED-Bench. 95

15.1 Toy failure-mode calculations ([M]). Left: acceptance versus temperature for a single noisy-copy draft (non-monotone in this toy; not a general law). Middle: relaxed acceptance $\min\{1, \lambda p/q\}$, per-token acceptance versus the total-variation distance of the emitted distribution to the target (labels are λ). Right: bias from drafting at temperature T_d while verifying with q at temperature 1. 108

List of Tables

1.1 Dated milestones in the review window, as reported by the cited pages. Dates are taken from arXiv version stamps or event pages. 17

3.1 Monte-Carlo test of exactness ([M]). TV is total-variation distance between the empirical and exact joint of the first three tokens (4×10^5 samples). The χ^2 test uses the 163 cells with expected count ≥ 5. The last row is the noise floor: the TV obtained by drawing 4×10^5 samples directly from the exact joint. 25

3.2 Left: bias from verifying with q at $T = 1$ while drafting at temperature T_d. Right: relaxed acceptance $\min\{1, \lambda p/q\}$ with the standard residual (our illustrative relaxation, not a specific published rule). Exact single-step calculations on a toy Zipfian vocabulary ([M]). 26

4.1 Monte-Carlo check of $\mathbb{E}[N] = (1 - \alpha^{\gamma+1})/(1 - \alpha)$ ([M]). α is the exact $\sum \min(p, q)$ of the constructed distributions. 29

4.2 Reproduction of the free-draft table of Leviathan et al. [20]. "Ours" is computed from the formulas ([D]); "published" values are as printed in the paper ([S]). All six rows agree to two decimals. 30

4.3 Optimal integer draft length and speedup $S(\alpha, \gamma^\star, c)$ from (4.3) ([D]; computed by enumeration, $\gamma \leq 32$). The $c = 0$ row is truncated at the grid limit of $\gamma = 32$. . . . 31

4.4 Continuous optimum from (4.5) versus exhaustive integer search ([D], checked numerically with scipy; $\gamma \leq 199$). 32

4.5 Effect of heterogeneity at fixed mean acceptance $\bar{\alpha} = 0.7$, $\gamma = 5$, $c = 0.05$ ([M]). Speedup is computed from the mean tokens per round and the constant round time. 34

4.6 Effective i.i.d. acceptance rates implied by published acceptance lengths ([D], by numerical inversion of (4.2)); the acceptance lengths are [S] except where the source is a vendor blog ([V]). 34

5.1 Check of Theorem 5.1: $\alpha = 0.8$, $\gamma = 4$, $n^\star = 221.4$ ([M] versus [D]). 37

5.2 First batch size at which the modelled speedup falls below 1, for context 512 ([M]). Under the weights-only theorem the corresponding values would be $B_{\text{be}} = \mathbb{E}[N]\, n^\star/(\gamma + 1)$. 38

5.3 Sensitivity of the break-even batch size to the assumed FLOP efficiency ($\alpha = 0.8$, $\gamma = 4$, ctx 512; [M]). 39

5.4 Throughput relative to no speculation, SGLang, H100, Llama-3.1-8B, MT-Bench, chain draft of length 3 ([S]; EAGLE-3 paper Table 3, measured by the SGLang team). 40

6.1 Recursive rejection sampling with k i.i.d. candidates ([M]versus [D]). TV is the distance of the output histogram from p (3×10^5 trials). 44

6.2 Expected tokens per round (Eq. (6.2)) of the best-first tree versus a chain with the same number of verified draft nodes (chain truncated at depth 10), under the rank model with $a = 0.6$, $b = 0.3$ ([M], exact enumeration). 46

6.3 Modelled throughput speedup of best-first trees (rows: number of verified nodes) across batch sizes, ctx 512 ([M]). 47

7.1 Speculative sampling with Chinchilla 70B at batch size one, $K = 4$ ([S]; Table 1 of Chen et al. [6]). The task metric is unchanged to within noise (e.g. HumanEval pass rate 45.1% autoregressive versus 47.0% speculative). 50

7.2 Taxonomy and headline reported results. Speedups are *as reported by the authors under their own settings* ([S] unless marked) and are not comparable across rows. "AR": autoregressive baseline. "Exact": distribution-preserving verification as described by the authors. 52

8.1 EAGLE-3 paper, Table 1, Vicuna-13B, temperature 0, mean over MT-Bench, HumanEval, GSM8K, Alpaca and CNN/DailyMail ([S]; extracted from the paper's table, where the "Mean" columns were unambiguous). τ is the mean acceptance length. 56

9.1 P-EAGLE hidden-state ablation ([S]; Table 3 of Hui et al. [15]): GPT-OSS-20B target, 4-layer drafter, 20 epochs on OpenCodeInstruct, HumanEval, speculation length 5. 60

9.2 Training overhead at sequence length 2048, $K = 8$ ([S]; Table 2 of Hui et al. [15]). Data loading measured on 128 examples; epoch time on UltraChat (200K examples), 8×H200. 61

9.3 Acceptance length at speculation depth $K = 5$ ([S]; Table 9 of Hui et al. [15]); values as printed (one decimal). The maximum possible with the bonus token is 6.0. 61

10.1 SpecVocab results on Spec-Bench with Qwen3-8B ([S]; Tables 1–2 of Williams et al. [56]); SGLang, one GPU; mean over five seeds. "EAGLE-3" is a third-party checkpoint, "Repro." is the authors' own EAGLE-3 draft trained with their recipe on UltraChat (464K target-generated examples); FR-Spec and VocabTrim are applied to the reproduction with a 32K vocabulary. The last column is our ratio against the reproduction ([D]). 66

10.2 Prompt-lookup drafting on real text as a deterministic target ([M]). Mean tokens per round $E[N]$ including the bonus token, with the fraction of rounds in which any draft was proposed. Because the target is a fixed text, these are measurements of text repetitiveness, not of any LLM. The code-edit case is an extreme (a purely mechanical rename); real edit tasks change semantics. 69

10.3 Qualitative comparison of drafter families (synthesis, [D]). 72

11.1 Left: stationary hit probability, simulation versus Equation (11.2) ([M]). Right: discrete-event simulation of SSD versus Theorem 11.2 ([M]); the backup has acceptance $\alpha - 0.1$. 76

11.2 Hit probability of uniform versus geometric fan-out allocation versus a numerical optimum, for the power-law model $1 - p_{\text{hit}}(F) = F^{-r}$ ([M]). 78

11.3 Saguaro-sampling toy ([M]): acceptance α, probability that the bonus token is in the cache given a rejection, per-round hit rate, and modelled speedup with $K = 5$, $T_p = 1$, instantaneous backup. 80

11.4 Decode throughput (tokens/s) of autoregressive decoding (AR), SD and SSD (Saguaro) from Appendix B.3 of Kumar et al. [18] ([S]), on 4 H100 for AR and SD and 5 for SSD, batch size 1, greedy. Ratios are as printed in the paper. 82

11.5 Best baseline configurations in the SSD paper's sweeps (decode tokens/s, Llama-3.1-70B/1B, [S]; Appendix B.4). SGLang cells are best over the tested (steps, top-k) grid; "–" for incompatible configurations is omitted. 83

12.1 Approximate drafter memory for Llama-3.1-8B in bf16 ([D]; derived from model dimensions, not measured). The EAGLE-3 head assumes one decoder layer (attention $2d^2 + 2d \cdot d_{kv}$ with $d_{kv} = 1024$, MLP $3 \cdot d \cdot 14336$), a $3d \to d$ fusion projection and a 32K-vocabulary output matrix; the input embedding is assumed shared with the target. The real layer takes a concatenation of two d-dimensional vectors as input, which adds a few tens of millions of parameters that we ignore. 90

13.1 Execution-time breakdown reported by Liu et al. [27] ([S]), vLLM on H100, CNN/DailyMail requests; ranges as stated in the text of the paper. 93

13.2 Average generation length in tokens (mean $\pm$ s.d.) on ShareGPT, Llama-3.1-8B, by batch size ([S]; Table 1 of Liu et al. [27]). Differences are not systematically positive or negative, consistent with numerical nondeterminism. 94

13.3 Claims audit. "Headline" is as stated by the source; "Normalised" is the gain over the strongest comparator the source itself reports in the same setting ([D] when we computed the ratio). All rows batch size 1 unless stated; none independently replicated in the sources we retrieved. 96

13.4 Modelled speedup over autoregressive decoding for sequential versus parallel ($\kappa = 2$) drafters, with and without SSD (hit probability p); $\alpha = 0.8$, $\gamma = 5$, ceiling $E = 3.69$ ([D], closed-form model, not measurements). 97

14.1 Levers for faster decoding and their interaction with speculative decoding (synthesis of earlier chapters and cited sources; [D] unless marked). 103

14.2 Which speculation strategy to try first, by regime (synthesis, [D]). 104

15.1 Failure modes of speculative decoding in serving (synthesis; evidence tags as in the text). 107

16.1 Sources used for community perception, with extent of reading. 111

16.2 Common beliefs about speculative decoding and the evidence in this report. . . . 114

17.1 Speedup $S(\gamma)$ from the closed form for several (α, c) pairs [D]. The final columns give the best integer γ and the continuous optimum $\gamma^\star$. 117

Chapter 1

Introduction

1.1 Why decoding latency is a hard problem

A transformer language model generates text one token at a time. To produce token $t+1$ the model must have consumed token t, so the generation of a length-n response is a chain of n dependent forward passes. Each pass reads the model's parameters (and the accumulated key–value cache) from high-bandwidth memory (HBM) into on-chip memory, performs a modest amount of arithmetic per byte read, and writes back little. On a modern accelerator this arithmetic intensity is far below the machine's balance point, so the step time is set by memory bandwidth, not by floating-point throughput. The consequence is that an accelerator capable of nearly a petaflop per second spends most of each decode step idle with respect to arithmetic. This is the central inefficiency that speculative decoding exploits.

The pressure to fix it has grown over the period this report covers. Reasoning models produce long chains of thought, so response lengths of thousands of tokens are routine; one of the studies reviewed here reports a median combined prompt-plus-generation length of 3,891 tokens and a 90th percentile of 10,800 for a 120-billion-parameter reasoning model on a chat dataset[S] [15]. Agentic workloads chain many such generations, so latency compounds. Interactive products (coding assistants, voice agents) have hard per-token latency requirements. Meanwhile reinforcement-learning post-training spends a large fraction of wall-clock time generating rollouts, which has made decoding speed a training-cost issue as well [50, 58].

1.2 Speculative decoding in one paragraph

Speculative decoding [6, 20] splits each iteration into *drafting* and *verification*. A cheap draft mechanism proposes γ candidate tokens. The large target model then evaluates all γ candidates (plus one extra position) in a *single* forward pass, which costs only slightly more than one ordinary decoding step when the system is memory-bound. A rejection-sampling rule decides how many candidates to keep: it accepts the candidates left to right with a probability that depends on the ratio of target to draft probabilities, and at the first rejection it resamples from a corrected ("residual") distribution. The rule is constructed so that the emitted tokens are distributed *exactly* as if the target had been sampled autoregressively. Speedup arises because each target pass now yields, on average, more than one token.

1.3 The three developments this report focuses on

The original formulation used a small separate language model as the drafter. The years since have been a sequence of attacks on its three costs: poor draft quality (low acceptance), the cost of drafting itself, and the serial dependency between drafting and verification.

EAGLE-3 (training the drafter on its own mistakes). The EAGLE family [21–23] builds a one-layer drafter that conditions on the target model's internal features. EAGLE-3 removes the feature-regression loss that earlier versions inherited, predicts tokens directly, and replaces the target's top-layer feature with a fusion of low-, middle-, and high-level features. A "training-time test" simulates multi-step drafting during training so that the drafter learns to cope with its own imperfect inputs. The authors report that this change lets the drafter benefit from more training data, with speedups of up to 6.5× at batch size one[S] [23].

Parallel drafting (removing the K sequential draft passes). Even a one-layer autoregressive drafter must be run K times to propose K tokens. Parallel drafters (PARD [1], P-EAGLE [15], DFlash [7]) predict several tokens in a single forward pass, trading some per-token accuracy for a lower and, importantly, nearly depth-independent drafting cost. P-EAGLE was integrated into vLLM; its authors report speedups of 1.10–1.36× over autoregressive EAGLE-3 in their paper's vLLM experiments[S], and a vLLM project blog reports up to 1.69× on a single B200 for one model[V].

Speculative speculative decoding (removing the draft/verify dependency). In ordinary speculative decoding the verifier idles during drafting and the drafter idles during verification. SSD [18] runs the drafter on separate hardware *during* verification, and uses that time to pre-compute speculations for many possible verification outcomes (how many tokens will be accepted, and which bonus token will be sampled). If the actual outcome is in the pre-computed cache, the next speculation is available immediately. The Saguaro algorithm adds three optimizations: a geometric fan-out of the cache, a cache-aware sampling scheme, and a batch-size-dependent fallback. The paper reports about 30% average improvement over the strongest speculative-decoding baselines and up to 5× over autoregressive decoding, at batch size one on Llama-3.1-70B[S].

1.4 Scope and timeline of the period under review

The review window is October 2025 to October 2026. Table 1.1 lists the dated events relevant to this report that could be verified from retrieved pages. Older foundational works (2018–2025) are covered in Chapter 7 because the new methods cannot be understood without them.

1.5 Contributions and organization

This report is intended to be usable as a reference for a practitioner who wants to understand *why* these methods work and *when* they do not. Its structure is:

- Chapters 2–6 develop the mathematics from first principles: memory-bound decoding and the roofline model, the speculative sampling theorem and its proof, expected-token and speedup formulas, the batching regime, and tree verification.
- Chapter 7 gives the history and a taxonomy. Chapters 8–11 cover EAGLE-3, other drafter improvements, parallel drafting, and SSD in detail, including proofs and our simulations.
- Chapter 12 covers integration with vLLM, SGLang and TensorRT-LLM; Chapter 13 consolidates the empirical evidence; Chapter 14 compares with diffusion and other parallel

Table 1.1: Dated milestones in the review window, as reported by the cited pages. Dates are taken from arXiv version stamps or event pages.

Date	Event	Source
Oct 2025	Mirror Speculative Decoding (Apple) posted: heterogeneous GPU/NPU, 2.8–5.8× on SpecBench for 14B–66B models[S]	[3]
Dec 2025	SpecBundle / SpecForge v0.2 release of EAGLE-3 draft checkpoints for 12 instruct models[V]; EAGLE-3 on Vertex AI with SGLang[V]	[13, 29]
Dec 31 2025	"Speculative Decoding: Performance or Illusion?" posted (v2 March 2026; MLSys 2026 oral, May 21 2026)[S]	[27]
Feb 1 2026	P-EAGLE posted (arXiv:2602.01469); NeurIPS 2026 poster scheduled for Dec 10 2026[S]	[15, 36]
Feb 5 2026	DFlash posted (block-diffusion drafter)[S]	[7]
Feb 14 2026	SpecVocab posted (v2 July 17 2026)[S]	[56]
Mar 2026	P-EAGLE blog and vLLM v0.16.0 support[V]; SSD / Saguaro posted (arXiv:2603.03251); ConFu posted (arXiv:2603.08899)[S]	[18, 41, 53]
Apr 24 2026	SSD presented as an ICLR 2026 poster[S]	[16]
May 5 2026	Google releases Gemma 4 multi-token-prediction drafters (secondary summary)[V]	[12]
Jun 10 2026	HKUST(GZ) PhD qualifying exam on speculative decoding (survey plus AdaMemSpec)[S]	[58]
Jul 13 2026	vLLM/AMD blog on EAGLE-3 for AMD Instinct GPUs[V]	[54]

decoding; Chapters 15–17 discuss pros, cons, failure modes, community reception, and case studies; Chapters 18–19 give open problems and conclusions.

- The appendices contain extended proofs, the complete simulation code with its real outputs, and a glossary.

1.6 Methodology and limits of this study

The material was gathered from primary sources (arXiv versions of the papers, conference pages, framework documentation) whenever possible. Where only a blog, vendor page, or forum snapshot was available, it is flagged with [V]. Four limits matter:

1. **No real-GPU experiments.** All "simulation" results are of the mathematical models described in the text. The roofline simulations in particular use assumed efficiency factors and should be read as qualitative illustrations.
2. **Partial retrieval.** Some papers were read through text extraction of their PDFs, which garbles multi-column tables; where a table could not be unambiguously reconstructed, only the unambiguous figures are quoted.
3. **Moving documentation.** Framework documentation (vLLM, TensorRT-LLM) changes rapidly; statements reflect the pages as retrieved in October 2026.
4. **Version differences.** The SSD paper's abstract page and its conference page quote different headline figures (about 30% average gain over strong baselines in the arXiv v3 abstract; "up to 2×" on the ICLR page). We report both and attribute each.

Chapter 2

Background: Autoregressive Decoding and the Memory Wall

2.1 Notation

Throughout, $x_{1:t} = (x_1, \ldots, x_t)$ is a token sequence over a vocabulary $\mathcal{V}$ with $|\mathcal{V}| = V$. The *target* model defines conditional distributions $p(\cdot \mid x_{1:t})$ and the *draft* mechanism defines $q(\cdot \mid x_{1:t})$. When the context is clear we write $p(x)$ and $q(x)$. A speculation of length γ (the *lookahead*; the SSD paper writes K) is $\tilde{x}_{t+1}, \ldots, \tilde{x}_{t+\gamma}$. The acceptance rate of a position is α and $\beta = \mathbb{E}[\alpha]$ denotes its expectation over contexts in the notation of Leviathan et al. [20]. c denotes the ratio of the time of one draft forward pass to the time of one target forward pass. $\mathrm{TV}(p, q) = \frac{1}{2} \sum_x |p(x) - q(x)|$ is the total variation distance.

2.2 What a decode step costs

2.2.1 Parameters, FLOPs and bytes

Consider a decoder-only transformer with P parameters held in b-byte precision (here $b = 2$ for BF16/FP16). A forward pass over n new tokens performs approximately $2Pn$ floating-point operations in the dense matrix multiplications (two FLOPs, a multiply and an add, per parameter per token), plus attention operations that scale with the context length S. It must read all bP bytes of weights at least once from HBM, independent of n, plus the key–value (KV) cache entries of the S previous tokens for each sequence in the batch.

For a *batch* of B sequences, each contributing m new tokens, we have $n = Bm$ and, ignoring attention,

$$\text{FLOPs} \approx 2P\,B\,m, \qquad \text{bytes} \approx bP + B\,S\,\kappa, \tag{2.1}$$

where κ is the KV-cache size per token in bytes. For a model with L layers, n_{kv} key–value heads of dimension d_h, stored in b bytes, $\kappa = 2\,L\,n_{kv}\,d_h\,b$. For Llama-3.1-8B ($L = 32$, $n_{kv} = 8$, $d_h = 128$, $b = 2$) this is 131,072 bytes = 128 KiB per token[D], so a single 32,768-token sequence has a 4 GiB cache.

2.2.2 Arithmetic intensity and the roofline

The *arithmetic intensity* of a kernel is $I = \text{FLOPs/bytes}$. A processor with peak throughput F FLOP/s and memory bandwidth W byte/s can sustain at most $\min(F,\ I \cdot W)$ FLOP/s; this is the *roofline model*. The *ridge point* $I^\star = F/W$ separates memory-bound ($I < I^\star$) from compute-bound ($I > I^\star$) execution. The time of a step is lower-bounded by

$$t \;\geq\; \max\left(\frac{\text{bytes}}{W},\ \frac{\text{FLOPs}}{F}\right). \tag{2.2}$$

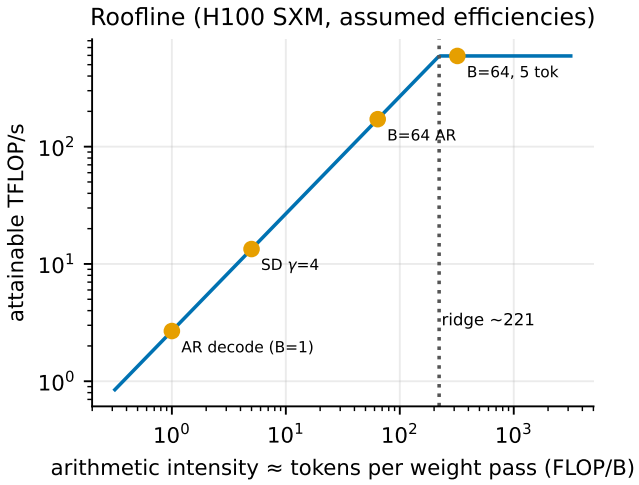


Figure 2.1: Roofline schematic for an H100-class GPU using the assumed efficiency factors of our model ([M]). A single-token decode step sits far to the left of the ridge. A speculative step verifying $\gamma + 1 = 5$ tokens moves right but remains memory-bound; the same request inside a batch of 64 verifying five tokens each is near the ridge and has little spare compute.

Ignoring attention and the KV cache, (2.1) gives, with $b = 2$, $I = 2PBm/(2P) = Bm$. In other words, *the arithmetic intensity of a weight-dominated decode step in BF16 is simply the number of tokens processed in the step*[D]. This clean fact drives the whole field: autoregressive decoding at batch size one has $I = 1$ FLOP/byte; the ridge point of current accelerators is in the hundreds.

Example 2.1 (Ridge point of an H100)**.** The NVIDIA H100 SXM datasheet lists 3.35 TB/s memory bandwidth and 80 GB of HBM, and tensor-core peak figures of 1,979 TFLOP/s for BF16/FP16 marked as "with sparsity"[S] [38]. Dense peak is half that, 989.5 TFLOP/s[D](the footnote means the listed figure assumes 2:4 structured sparsity; halving it is our inference). The ridge point is therefore

$$I^\star \approx \frac{989.5 \times 10^{12}}{3.35 \times 10^{12}} \approx 295 \text{ FLOP/byte}.$$

Under (2.1) the step becomes compute-bound at roughly $Bm \approx 295$ tokens per weight pass[D]. Real kernels reach only a fraction of peak FLOPs and bandwidth; with the assumed efficiencies used later (60% of peak FLOPs, 80% of peak bandwidth) the ridge moves to about 221 tokens[M].

2.2.3 A reality check against published numbers

We can test the memory-bound picture against two published baselines.

Example 2.2 (Llama-3.1-8B on one H100)**.** Weights in BF16 occupy about $2 \times 8.03 \times 10^9 \approx 16.1$ GB. Reading them at 3.35 TB/s takes 4.8 ms, an upper bound of about 208 tokens/s for batch-size-one decoding[D]. The SGLang team's measurement reported in the EAGLE-3 paper for this setting is 158.34 tokens/s without speculation[S] [23], about 76% of the idealised bound[D]. The remainder reflects KV-cache reads, kernel launch and non-GEMM operators, and bandwidth efficiency below 100%.

Example 2.3 (Llama-3.1-70B on four H100s with tensor parallelism)**.** The Saguaro paper reports an autoregressive baseline of 54.7 tokens/s for Llama-3.1-70B with tensor parallelism 4 on H100s[S] [18]. Each GPU holds roughly $140/4 = 35$ GB of BF16 weights, which takes 10.4 ms to

stream, giving an ideal bound of about 96 tokens/s[D]. The measured throughput is about 57% of that bound[D], consistent with all-reduce communication between GPUs adding substantial per-token overhead at small batch size. The same paper notes that tensor-parallel decoding at this scale makes drafting on the same GPUs inefficient, which motivates placing the drafter on a separate device.

2.3 Why batching helps throughput but hurts speculation

Equation (2.1) shows that the weight bytes are amortised over all Bm tokens in a step. Increasing the batch size B is therefore the standard way to raise throughput: at $B = 1$ the step costs bP/W and yields one token; at $B = 64$ the step costs nearly the same and yields 64. Serving systems exploit this with continuous batching [19]. But batching consumes exactly the slack that speculative decoding wants to use. With a batch of B sequences, a speculative step verifying $\gamma + 1$ tokens per sequence has intensity $B(\gamma + 1)$; once $B(\gamma + 1)$ approaches the ridge point, additional verified tokens cost real time, and the rejected ones are pure waste. We make this quantitative in Chapter 5.

2.4 The KV cache and long contexts

For long contexts the bytes term in (2.1) is dominated by the KV cache rather than the weights: $BS\kappa$ exceeds bP when $BS\kappa > bP$, i.e. for the 8B model above when $BS > 16.1$ GB/128 KiB $\approx$ 123,000 token-contexts in flight. In this regime, verifying $\gamma + 1$ tokens per sequence reads the *same* KV cache once for all $\gamma + 1$ queries, so verification costs hardly more than a normal step even at large batch sizes, while attention FLOPs grow only modestly. This observation is the basis of MagicDec [43], which argues (and reports) that speculative decoding can remain useful, even increasingly useful, at large batch sizes for sufficiently long contexts. Its authors estimate for LLaMA-3.1-8B a crossover context length of around 4,000 tokens in their theoretical analysis, above which speedup can grow with batch size[S]. We reproduce the qualitative effect in Section 5.6.

2.5 Mixture-of-experts models

Many 2026 frontier models are mixture-of-experts (MoE). At batch size one only a few experts per layer are active, so the weight bytes read per step are a small fraction of the total parameter count. When verifying $\gamma + 1$ tokens, different tokens may route to different experts, so more experts must be loaded and the step becomes more expensive than a single-token step. Two sources in this review report effects of this kind: the P-EAGLE paper attributes diminishing speedups for larger MoE models at higher concurrency to expert routing overhead scaling with batch size[S] [15], and a secondary summary of Google's Gemma 4 drafter announcement says that, for the MoE variant at batch size one, verifying multi-token drafts can load extra experts and offset much of the gain, so that benefits mainly appear at higher batch sizes[V] [12]. The sign of the batch-size effect can therefore *reverse* for MoE models relative to dense ones, a point we return to in Chapter 15.

2.6 Sampling, temperature and what "lossless" means

Autoregressive sampling draws $x_{t+1} \sim p(\cdot \mid x_{1:t})$, where $p = \mathrm{softmax}(z/T)$ for logits z and temperature T, possibly after top-k or nucleus truncation and renormalisation. Speculative sampling can be applied to the *adjusted* distributions; what matters is that the verifier uses precisely the same adjusted distribution that autoregressive sampling would use (Section 3.6). *Greedy* decoding ($T \to 0$) is a special case in which p is a point mass and a draft token is accepted if and only if it equals the target's arg-max; most framework implementations treat it separately.

"Lossless" has a precise meaning: the distribution of the emitted token sequence equals the target's autoregressive distribution, in exact arithmetic. In practice, floating-point non-associativity makes even autoregressive outputs depend on batch composition and kernels. The MLSys 2026 study of Liu et al. [27] documents that generation lengths for the same prompts differ across batch sizes even without speculation, with a mean of 737 ± 1048 tokens at batch size 1 versus 674 ± 784 at batch size 32 on ShareGPT with Llama-3.1-8B[S]. Distributional equivalence is thus a statement about the algorithm, not a guarantee of bitwise identical outputs.

2.7 Prefill versus decode

Prefill processes the whole prompt in parallel and is typically compute-bound (intensity equals the prompt length), so speculative decoding does not help it; it only accelerates the decode phase. A consequence is that workloads with long prompts and short outputs (classification, short answers) gain little end-to-end. Several of the retrieved sources therefore report decode-only throughput (for example, Saguaro excludes prefill from its measurements[S]), and a note in a vendor-style article cautions that time-to-first-token may increase for some deployments[V] [13].

Chapter 3

Speculative Sampling: Algorithm and Correctness

This chapter develops the exact acceptance/rejection scheme of Leviathan et al. [20] and Chen et al. [6], proves that it leaves the output distribution unchanged, derives the acceptance probability and residual distribution, and shows that the scheme is optimal in a precise sense. All proofs are our own write-ups of the standard arguments; each is labelled [D].

3.1 Single-position speculative sampling

Fix a context and let p and q be the target and draft distributions over $\mathcal{V}$ for the next token. We want one sample $X \sim p$ but are only willing to evaluate p in a way that is *shared* with other positions, and we hold a cheap sample $\tilde{X} \sim q$.

Definition 3.1 (Speculative sampling step)**.** Draw $\tilde{X} \sim q$ and $U \sim \mathrm{Unif}[0,1]$ independently. If $U \le \min\{1,\, p(\tilde{X})/q(\tilde{X})\}$, output $X = \tilde{X}$ (*accept*). Otherwise (*reject*) draw X from the *residual* distribution

$$r(x) = \frac{(p(x) - q(x))_+}{\sum_y (p(y) - q(y))_+}, \qquad (u)_+ = \max(u, 0). \tag{3.1}$$

Lemma 3.2 (Acceptance probability)**.** *The probability that the step accepts is*

$$\alpha = \sum_x \min\{p(x), q(x)\} = 1 - \mathrm{TV}(p, q), \tag{3.2}$$

and the normaliser of the residual is $\sum_y (p(y) - q(y))_+ = \mathrm{TV}(p, q) = 1 - \alpha$.

Proof. $\mathbb{P}[\text{accept} \mid \tilde{X} = x] = \min\{1, p(x)/q(x)\}$ (with the convention that the ratio is $+\infty$ when $q(x) = 0$, which has probability zero under q). Hence $\mathbb{P}[\text{accept}] = \sum_x q(x) \min\{1, p(x)/q(x)\} = \sum_x \min\{q(x), p(x)\}$. For the second identity note $\min\{p, q\} = p - (p - q)_+$, so $\sum_x \min\{p, q\} = 1 - \sum_x (p - q)_+$; and since $\sum_x (p - q) = 0$ we have $\sum_x (p - q)_+ = \sum_x (q - p)_+ = \frac{1}{2} \sum_x |p - q| = \mathrm{TV}(p, q)$. □

Theorem 3.3 (Exactness of one step)**.** *The output X of a speculative sampling step satisfies $X \sim p$.*

Proof. Fix $x \in \mathcal{V}$. The event $\{X = x\}$ decomposes into acceptance of $\tilde{X} = x$ and rejection followed by a residual draw of x:

$$\mathbb{P}[X = x] = q(x) \min\Big\{1, \frac{p(x)}{q(x)}\Big\} + \mathbb{P}[\text{reject}]\; r(x) = \min\{p(x), q(x)\} + (1 - \alpha) \frac{(p(x) - q(x))_+}{1 - \alpha}.$$

Here we used Lemma 3.2 for $\mathbb{P}[\text{reject}] = 1 - \alpha$ and for the normaliser of r. (If $\alpha = 1$ then $p = q$, rejection has probability zero, and the claim is trivial.) Since $\min\{p, q\} + (p - q)_+ = p$ pointwise, we obtain $\mathbb{P}[X = x] = p(x)$. □

Algorithm 1: One round of speculative sampling (Leviathan et al.; Chen et al.).

Input: prefix $x_{1:t}$; draft model q; target model p; lookahead γ

for $i = 1$ **to** γ **do**
  $q_i \leftarrow q(\cdot \mid x_{1:t}, \tilde{x}_{1:i-1})$; $\tilde{x}_i \sim q_i$;
Run the target once on $(x_{1:t}, \tilde{x}_{1:\gamma})$ to obtain $p_i \leftarrow p(\cdot \mid x_{1:t}, \tilde{x}_{1:i-1})$ for $i = 1, \ldots, \gamma + 1$;
$n \leftarrow \gamma$;
for $i = 1$ **to** γ **do**
  draw $u_i \sim \mathrm{Unif}[0, 1]$;
  if $u_i > p_i(\tilde{x}_i)/q_i(\tilde{x}_i)$ **then**
    $n \leftarrow i - 1$; **break**
if $n < \gamma$ **then**
  $y \sim \mathrm{norm}\big((p_{n+1} - q_{n+1})_+ \big)$
else
  $y \sim p_{\gamma+1}$
return $(\tilde{x}_1, \ldots, \tilde{x}_n, y)$; `// between 1 and` $\gamma + 1$ `new tokens`

Example 3.4 (Numerical walk-through)**.** Let $\mathcal{V} = \{1, 2, 3, 4\}$, $p = (0.50, 0.30, 0.15, 0.05)$ and $q = (0.25, 0.45, 0.20, 0.10)$. Then $\min\{p, q\} = (0.25, 0.30, 0.15, 0.05)$ so $\alpha = 0.75$ and $\mathrm{TV} = 0.25$. The positive part $(p - q)_+ = (0.25, 0, 0, 0)$ has total 0.25 (as Lemma 3.2 predicts), so the residual is the point mass at token 1. The output law is $\mathbb{P}[X = 1] = 0.25 + 0.25 \cdot 1 = 0.50$, $\mathbb{P}[X = 2] = 0.30$, $\mathbb{P}[X = 3] = 0.15$, $\mathbb{P}[X = 4] = 0.05$, equal to p. Intuitively the draft *under*-proposes token 1 (0.25 versus 0.50) so whenever a rejection happens (which only occurs on tokens the draft over-proposed) the missing mass is made up by token 1. Our simulation ([M]) with 2×10^5 draws gives acceptance 0.7497 and output frequencies $(0.5012, 0.2998, 0.1493, 0.0497)$, within sampling error of p.

3.2 Speculative sampling of a block of γ tokens

In the full algorithm the drafter proposes an entire block autoregressively, and the target scores the block in one parallel pass.

Two implementation observations deserve emphasis. First, the target probabilities $p_1, \ldots, p_{\gamma+1}$ can all be computed in one forward pass because a causal transformer already produces next-token distributions for *every* position of its input; the verification pass is simply a prefill of γ extra tokens on top of the cached prefix. Second, the final token y is always produced, so every round yields at least one token: speculative sampling never produces fewer tokens per target pass than ordinary decoding.

Theorem 3.5 (Exactness of a round and of the whole generation)**.** *Let $Y_1, \ldots, Y_N$ be the (random-length) token sequence produced by repeating Algorithm 1, each round starting from the sequence generated so far. For every N and every $y_{1:N}$,*

$$\mathbb{P}[Y_{1:N} = y_{1:N}] = \prod_{i=1}^{N} p(y_i \mid y_{1:i-1})$$

(truncating the final round's output to length N if necessary).

Proof. It suffices to prove the following claim about a single round with prefix s: letting $Z_1, Z_2, \ldots$ denote the tokens emitted by the round (accepted drafts followed by y), for every j and every $z_{1:j-1}$ with $\mathbb{P}[Z_{1:j-1} = z_{1:j-1}, \text{ round emits } \geq j \text{ tokens}] > 0$,

$$\mathbb{P}\left[Z_j = z \;\middle|\; Z_{1:j-1} = z_{1:j-1}, \text{ emits } \geq j\right] = p(z \mid s, z_{1:j-1}). \tag{3.3}$$

Given the claim, the chain rule applied token by token (and then across rounds, each of which conditions on the previously emitted tokens as its prefix) yields the stated product.

Proof of the claim. Condition on $Z_{1:j-1} = z_{1:j-1}$ and on the round emitting at least j tokens. The latter event means that all of $\tilde{x}_{1:j-1}$ were accepted (so $\tilde{x}_{1:j-1} = z_{1:j-1}$) and the round did not terminate at position $j-1$ with a resampled token. Hence the round reaches position j with the verification context exactly $(s, z_{1:j-1})$. The draft token $\tilde{x}_j$ was drawn from $q(\cdot \mid s, \tilde{x}_{1:j-1}) = q(\cdot \mid s, z_{1:j-1})$, *conditionally independently* of the previous accept/reject uniforms given $\tilde{x}_{1:j-1}$, and u_j is a fresh uniform. If $j \leq \gamma$, the token emitted at position j is either $\tilde{x}_j$ (if accepted) or a residual draw: precisely a single-position speculative sampling step with $p = p(\cdot \mid s, z_{1:j-1})$ and $q = q(\cdot \mid s, z_{1:j-1})$, which by Theorem 3.3 has law p. If $j = \gamma + 1$ the emitted token is $y \sim p_{\gamma+1} = p(\cdot \mid s, \tilde{x}_{1:\gamma})$, again with the correct context. This proves (3.3).

One subtlety: the event "emits $\geq j$ tokens" depends on the earlier accept decisions, but *not* on the position-j draw, so conditioning on it does not bias $\tilde{x}_j$ or u_j. (Contrast with the position-$(j-1)$ decision, which is already part of the conditioning.) □

Remark 3.6 (What the proof does not need)**.** Nothing in the proof requires the draft to be a language model, to be calibrated, or even to be close to p. It requires only that (i) the draft distribution q used for the *acceptance ratio* is the distribution from which $\tilde{x}$ was actually sampled, and (ii) p is the true target conditional. This is why EAGLE-style feature-conditioned drafters, Medusa heads, n-gram lookups (with a deterministic proposal, i.e. q a point mass), and diffusion drafters are all "lossless" under the same verifier. Accuracy of q affects only speed.

3.2.1 Deterministic drafts

For a deterministic proposal such as an n-gram match, $q = \delta_{\tilde{x}}$. The acceptance probability becomes $\min\{1, p(\tilde{x})/1\} = p(\tilde{x})$ and the residual is $(p - \delta_{\tilde{x}})_+$ normalised, i.e. p with the mass of $\tilde{x}$ removed and renormalised. The expected acceptance of a deterministic proposal is therefore just the target probability of the proposed token, $\alpha = p(\tilde{x})$. Under greedy decoding (p a point mass at $x^\star$) this reduces to "accept iff $\tilde{x} = x^\star$", which is the rule used in most engines' greedy path.

3.3 Optimality of the acceptance rule

Is the acceptance probability $\alpha = 1 - \mathrm{TV}(p, q)$ the best one can do? Consider any joint distribution (coupling) of $(\tilde{X}, X)$ with marginals q and p. The event $\{\tilde{X} = X\}$ is the event that the proposed token can be kept.

Proposition 3.7 (Maximal coupling)**.** *For any coupling π of q and p, $\mathbb{P}_\pi[\tilde{X} = X] \leq 1 - \mathrm{TV}(p, q)$. The speculative sampling step attains equality.*

Proof. For each x, $\mathbb{P}_\pi[\tilde{X} = X = x] \leq \min\{\mathbb{P}[\tilde{X} = x], \mathbb{P}[X = x]\} = \min\{q(x), p(x)\}$. Summing over x gives the bound. The speculative step has $\mathbb{P}[\tilde{X} = X = x] = q(x)\min\{1, p(x)/q(x)\} =$

Table 3.1: Monte-Carlo test of exactness ([M]). TV is total-variation distance between the empirical and exact joint of the first three tokens (4×10^5 samples). The χ^2 test uses the 163 cells with expected count ≥ 5. The last row is the noise floor: the TV obtained by drawing 4×10^5 samples directly from the exact joint.

Variant	TV to exact	χ^2	dof	p-value
Correct (residual $\propto (p-q)_+$)	0.0056	153.3	162	0.675
Resample from p on reject	0.0797	16828.2	162	$< 10^{-16}$
Resample from q on reject	0.1261	126339.1	162	$< 10^{-16}$
Accept every draft token	0.3254	2579148.8	162	$< 10^{-16}$
AR sampling (reference)	0.0045	–	–	–

$\min\{p, q\}(x)$ on accept, and its marginal on X is p by Theorem 3.3, so it is a coupling attaining the bound. □

So no scheme that keeps the *single* proposed token and preserves the target marginal can accept more often. Improvements must come from improving q (raising $1 - \mathrm{TV}$), from offering *multiple* candidates per position (Chapter 6), or from relaxing exactness (Section 3.6).

3.4 Relationship between acceptance and divergences

$\alpha = 1 - \mathrm{TV}(p, q)$ is convenient but rarely measured directly; practitioners train drafters with cross-entropy or KL objectives. Pinsker's inequality, $\mathrm{TV}(p, q) \leq \sqrt{\frac{1}{2}\mathrm{KL}(p\|q)}$, gives the guarantee

$$\alpha \;\geq\; 1 - \sqrt{\tfrac{1}{2}\,\mathrm{KL}(p \,\|\, q)}. \tag{3.4}$$

The bound is loose in the regime of interest (an acceptance of 0.8 only requires $\mathrm{TV} = 0.2$, which Pinsker certifies at $\mathrm{KL} \leq 0.08$ nats; real drafters are typically further away in KL but still obtain $\alpha \approx 0.7$–0.8 because TV is bounded by 1 even when KL is large). It does explain why distillation of the target distribution (forward KL) and training on target-generated data both raise acceptance: the draft learns the target's own conditional distribution rather than the data distribution. The SDSL study [4] fits acceptance as a linear function of draft and target perplexity and finds that the draft's perplexity dominates, with $R^2 \approx 0.60$ over the pairs tested[S], a useful reminder that these relationships are empirical and noisy.

3.5 Monte-Carlo verification

We verify Theorem 3.5 numerically with a context-dependent toy language model: a first-order Markov chain over $V = 6$ tokens with a Dirichlet-random transition matrix P (the target) and a draft $Q = 0.6P + 0.4P'$ with P' an independent random chain. Per-state acceptance rates range from 0.66 to 0.88[M]. We draw 4×10^5 generations with $\gamma = 3$ and compare the empirical joint distribution of the first three emitted tokens (216 cells) with the exact joint $\prod p$. We also run three deliberately incorrect variants.

The correct algorithm passes ($\chi^2 = 153.3$ with 162 degrees of freedom, $p = 0.67$; TV 0.0056 against a noise floor of 0.0045)[M]. All three broken variants fail decisively (TV between 0.08 and 0.33, $p < 10^{-16}$). The first two are the classic implementation bugs of resampling from the wrong distribution on rejection; they are instructive because the TV they induce (0.08 and 0.13)

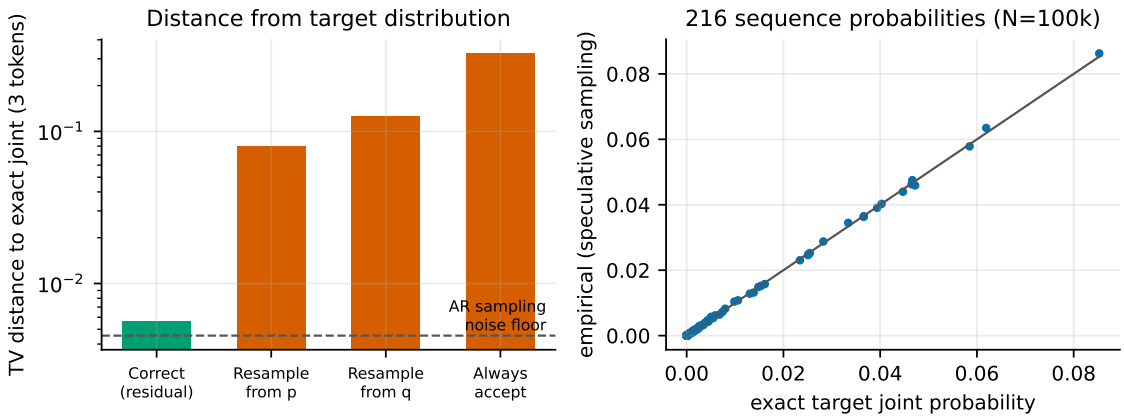


Figure 3.1: Left: distance to the exact target joint for the correct algorithm (green) versus three broken variants (red); dashed line is the sampling noise floor ([M]). Right: empirical versus exact probabilities of all 216 three-token sequences for the correct algorithm; points lie on the diagonal.

Draft sampling T_d	Acceptance	TV to target
1.0	0.537	0.000
0.9	0.467	0.074
0.7	0.373	0.180
0.5	0.353	0.227

λ	Per-token acceptance	TV to target	$E[\text{tokens}]$, $\gamma = 4$
1.0	0.537	0.000	2.06
1.25	0.601	0.063	2.31
1.5	0.659	0.122	2.57
2.0	0.765	0.228	3.14
4.0	0.934	0.397	4.38

Table 3.2: Left: bias from verifying with q at $T = 1$ while drafting at temperature T_d. Right: relaxed acceptance $\min\{1, \lambda p/q\}$ with the standard residual (our illustrative relaxation, not a specific published rule). Exact single-step calculations on a toy Zipfian vocabulary ([M]).

is small enough that a casual qualitative check of generated text would likely not detect it, while a statistical test does.

3.6 Pitfalls: when "lossless" implementations are not

Because exactness depends on using consistent distributions in the ratio, several practical mistakes silently break it. We quantify three with exact single-step calculations on a toy Zipfian vocabulary of $V = 2000$ tokens with a noisy-copy draft ([M]; Appendix B).

Mismatched draft sampling and verification probabilities. If the draft samples at temperature T_d but the verifier uses ratio p/q with q computed at $T = 1$, the output law is no longer p. In our toy, $T_d = 0.9$ gives a TV of 0.074 from the target, $T_d = 0.7$ gives 0.180, and $T_d = 0.5$ gives 0.227 (Table 3.2). Mismatched truncation behaves the same way: drafting from a top-50 truncated draft while verifying against the untruncated q yields a TV of 0.146, whereas using the truncated distribution consistently in the ratio restores exactness (TV $\approx 10^{-16}$) at the cost of lower acceptance (0.388 versus 0.451)[M]. The practical rule: the q in the acceptance ratio must be the *post-processed* distribution actually sampled from.

Relaxed acceptance. Several systems offer "lenient" verification that accepts more draft tokens at the price of exactness. A simple family accepts with probability $\min\{1, \lambda p/q\}$ for $\lambda \geq 1$ and resamples from the standard residual on rejection. In our toy, $\lambda = 1.5$ raises the per-token

acceptance from 0.537 to 0.659 and the expected tokens per round at $\gamma = 4$ from 2.06 to 2.57, but the output is then at TV 0.12 from the target; $\lambda = 4$ pushes acceptance to 0.934 at TV 0.40[M]. The vLLM documentation lists a "FLy" verification mode and states that it is lossy and not distribution-preserving[S] [52]; users who enable it should not assume equivalence with the target.

Numerical precision. Computing p in one precision during verification and q in another can create systematic ratio errors; more generally, the verifier computes p_i with a batched $(\gamma + 1)$-token kernel that may differ numerically from the single-token kernel used in autoregressive decoding. The result is an output law that equals the target *computed by the verification kernel*, which can differ in the low-order bits. This is the origin of the generation-length variability reported by Liu et al. [27] and discussed in Chapter 15.

Chapter 4

Expected Tokens, Speedup, and Optimal Draft Length

4.1 Expected number of tokens per target pass

Let N be the number of tokens emitted by one round of Algorithm 1. Write α_i for the probability that draft position i is accepted *given that positions* $1, \ldots, i-1$ *were accepted*.

Proposition 4.1 (General expected tokens)**.**

$$\mathbb{E}[N] = \sum_{k=0}^{\gamma} \prod_{j=1}^{k} \alpha_j \qquad (\textit{empty product} = 1). \tag{4.1}$$

Proof. $N = 1 + M$, where M is the number of accepted draft tokens. $M \geq k$ iff the first k draft tokens are all accepted, which has probability $\prod_{j \leq k} \alpha_j$. Hence $\mathbb{E}[M] = \sum_{k=1}^{\gamma} \mathbb{P}[M \geq k] = \sum_{k=1}^{\gamma} \prod_{j \leq k} \alpha_j$, and adding the always-present final token gives (4.1). □

Corollary 4.2 (I.I.D. acceptance; Leviathan et al.)**.** *If* $\alpha_j \equiv \alpha < 1$ *for all* j*, then*

$$\mathbb{E}[N] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}, \tag{4.2}$$

and $\mathbb{E}[N] = \gamma + 1$ *if* $\alpha = 1$*. As* $\gamma \to \infty$, $\mathbb{E}[N] \to 1/(1-\alpha)$.

Proof. Geometric sum. The limit follows from $\alpha^{\gamma+1} \to 0$. □

Formula (4.2) is exactly Theorem 3.8 of Leviathan et al. [20] with $\beta = \alpha$; it also appears in the SpecDec performance analysis of Liu et al. [27]. Note the interpretation: α is a *conditional* acceptance rate and the i.i.d. assumption means it does not depend on the position or on the history. Both assumptions are violated in practice (Sections 4.4 and 4.5).

Numerical confirmation. We confirmed (4.2) by Monte Carlo ([M]): for 16 pairs (α, γ) with α set to 0.5, 0.7, 0.8, 0.9 (up to the exactness of the constructed p, q) and $\gamma \in \{1, 3, 5, 8\}$, using 6×10^4 rounds each with context-independent random distributions, the maximum relative error between simulated and predicted $\mathbb{E}[N]$ was 0.50% (Table 4.1).

4.1.1 Distribution and variance of N

$M = N - 1$ is a geometric random variable truncated at γ, so $\mathbb{P}[N = k+1] = \alpha^k(1-\alpha)$ for $0 \leq k < \gamma$ and $\mathbb{P}[N = \gamma + 1] = \alpha^\gamma$. Using $\mathbb{E}[M^2] = \sum_{k=1}^{\gamma}(2k-1)\alpha^k$ we obtain $\mathrm{Var}[N] = \sum_{k=1}^{\gamma}(2k-1)\alpha^k - \left(\sum_{k=1}^{\gamma}\alpha^k\right)^2$. For $\alpha = 0.8$, $\gamma = 5$ this gives mean 3.69 and standard deviation about 2.0 tokens (computed from the formula); a nontrivial fraction of rounds, $\alpha^\gamma = 0.33$, accept the whole draft and the remainder stop earlier. The large variance matters for batched serving:

Table 4.1: Monte-Carlo check of $\mathbb{E}[N] = (1-\alpha^{\gamma+1})/(1-\alpha)$ ([M]). α is the exact $\sum \min(p,q)$ of the constructed distributions.

α (exact)	γ	Monte Carlo	Formula (4.2)	rel. err.
0.500	1	1.501	1.500	0.05%
0.500	3	1.874	1.875	0.04%
0.575	5	2.270	2.267	0.12%
0.500	8	2.006	1.996	0.50%
0.700	1	1.700	1.700	0.00%
0.700	3	2.528	2.533	0.19%
0.700	5	2.940	2.941	0.04%
0.700	8	3.201	3.199	0.07%
0.800	1	1.802	1.800	0.10%
0.800	3	2.956	2.952	0.13%
0.800	5	3.695	3.689	0.15%
0.800	8	4.319	4.329	0.23%
0.900	1	1.902	1.900	0.09%
0.900	3	3.437	3.439	0.07%
0.900	5	4.681	4.686	0.09%
0.900	8	6.130	6.126	0.06%

sequences in a batch finish rounds with different numbers of tokens, which creates ragged KV-cache updates and scheduling complexity (Chapter 12).

4.2 Wall-clock speedup

Let the target forward pass for a block of $\gamma+1$ tokens cost time T (equal to the cost of an ordinary single-token step in the memory-bound regime) and each draft pass cost cT. Drafting γ tokens sequentially costs γcT. The time of one round is $T(1+\gamma c)$ and it emits $\mathbb{E}[N]$ tokens, whereas autoregressive decoding emits one token per T.

Theorem 4.3 (Speedup with draft cost; Leviathan et al.)**.** *Under the i.i.d. model, the expected wall-clock improvement factor is*

$$S(\alpha,\gamma,c) = \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)}. \tag{4.3}$$

Proof. Throughput of speculative decoding is $\mathbb{E}[N]/(T(1+\gamma c))$ tokens per second (by the renewal-reward theorem: rounds are i.i.d. with mean reward $\mathbb{E}[N]$ and constant duration). Autoregressive throughput is $1/T$. Dividing gives (4.3). □

Proposition 4.4 (When can speculation help?)**.** *There exists $\gamma \geq 1$ with $S(\alpha,\gamma,c) > 1$ if and only if $\alpha > c$.*

Proof. For $\gamma = 1$, $S = (1+\alpha)/(1+c) > 1 \iff \alpha > c$. Conversely, suppose $\alpha \leq c$. Then $1+\alpha+\cdots+\alpha^{\gamma} \leq 1+\gamma\alpha \leq 1+\gamma c$ (as $\alpha^k \leq \alpha$ for $k \geq 1$), so $S \leq 1$ for every γ. □

This simple condition is a useful sanity test: a draft whose per-token acceptance is lower than its relative cost can never pay off, however long the lookahead. For example, Liu et al. [27] report the draft/target single-pass time ratio as about 12.5% for the 1B drafter with a 70B target on four GPUs but an estimated 37.5% for the 0.6B drafter with an 8B target on one GPU[S]; the

Table 4.2: Reproduction of the free-draft table of Leviathan et al. [20]. "Ours" is computed from the formulas ([D]); "published" values are as printed in the paper ([S]). All six rows agree to two decimals.

α	γ	Speed (ours)	Speed (published)	Ops (ours)	Ops (published)
0.6	2	1.96×	1.96×	1.53×	1.53×
0.7	3	2.53×	2.53×	1.58×	1.58×
0.8	2	2.44×	2.44×	1.23×	1.23×
0.8	5	3.69×	3.69×	1.63×	1.63×
0.9	2	2.71×	2.71×	1.11×	1.11×
0.9	10	6.86×	6.86×	1.60×	1.60×

latter pairing needs a per-token acceptance well above 0.375 to be useful, which helps explain why they find the draft-model approach noticeably less effective on the smaller model despite similar acceptance rates (75% versus 81%)[S].

4.2.1 Reproducing the original table

Leviathan et al. [20] tabulate the speedup and the extra arithmetic cost for $c = \hat{c} = 0$ (a free draft). Table 4.2 recomputes it from (4.3) and the operation-count factor $(\gamma + 1)/\mathbb{E}[N]$ (the ratio of arithmetic operations per generated token, which is > 1 because rejected tokens are verified).

4.2.2 Operation counts and energy

Speculative decoding is latency-oriented: it *increases* arithmetic work. With $\hat{c}$ the ratio of draft FLOPs per token to target FLOPs per token, one round performs $(\gamma + 1)$ target-token evaluations and γ draft evaluations, so the FLOPs per emitted token are

$$\frac{(\gamma + 1) + \gamma\hat{c}}{\mathbb{E}[N]} \text{ target-token-equivalents,} \tag{4.4}$$

compared with 1 for autoregressive decoding. At $\alpha = 0.8, \gamma = 5, \hat{c} = 0$ this is $6/3.69 = 1.63$, i.e. 63% more arithmetic in exchange for a 3.69× reduction in the number of sequential target passes[D]. In the memory-bound regime this extra arithmetic is nearly free in time; at large batch it is not (Chapter 5). Rejected-token work also consumes energy; the Saguaro paper makes the same point explicitly for SSD, where entire pre-computed chains are discarded on a cache miss[S] [18].

4.3 Optimal draft length

4.3.1 Numerical optimum

For fixed (α, c), S as a function of γ rises, peaks, and then declines as the linear drafting cost γc overtakes the geometrically saturating $\mathbb{E}[N]$. Table 4.3 lists the integer-optimal γ and the corresponding speedup over a grid of (c, α).

Table 4.3: Optimal integer draft length and speedup $S(\alpha, \gamma^\star, c)$ from (4.3) ([D]; computed by enumeration, $\gamma \le 32$). The $c = 0$ row is truncated at the grid limit of $\gamma = 32$.

c	α=0.5	α=0.6	α=0.7	α=0.8	α=0.9	α=0.95
0.0	32 (2.00×)	32 (2.50×)	32 (3.33×)	32 (5.00×)	32 (9.69×)	32 (16.32×)
0.02	4 (1.79×)	6 (2.17×)	8 (2.76×)	11 (3.82×)	19 (6.37×)	31 (9.95×)
0.05	3 (1.63×)	4 (1.92×)	6 (2.35×)	8 (3.09×)	13 (4.67×)	21 (6.60×)
0.1	2 (1.46×)	3 (1.67×)	4 (1.98×)	6 (2.47×)	10 (3.43×)	15 (4.48×)
0.2	1 (1.25×)	2 (1.40×)	3 (1.58×)	4 (1.87×)	7 (2.37×)	10 (2.87×)
0.4	1 (1.07×)	1 (1.14×)	2 (1.22×)	2 (1.36×)	4 (1.58×)	6 (1.77×)

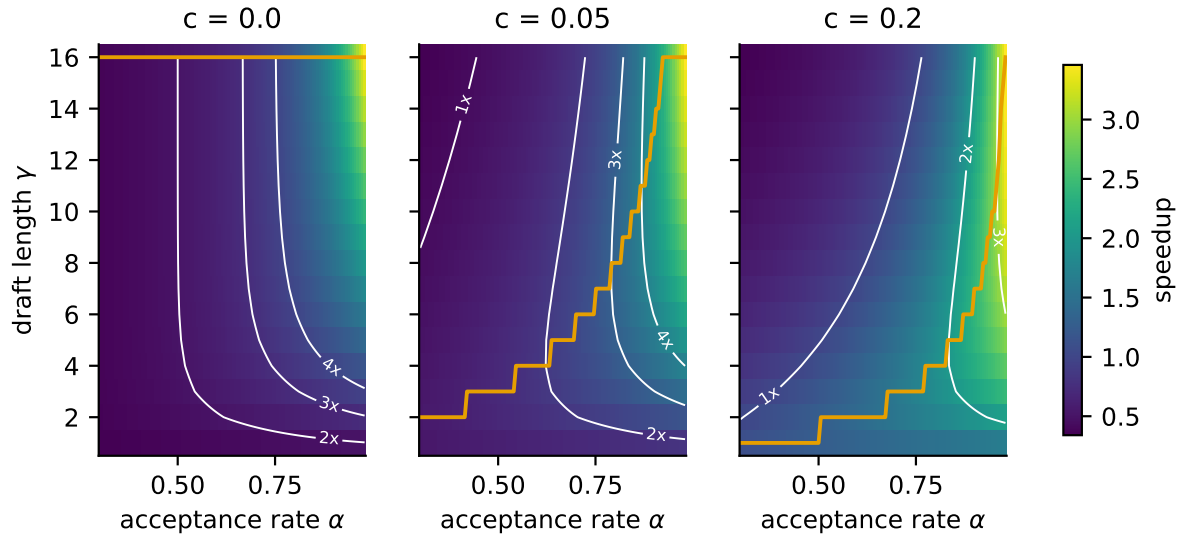


Figure 4.1: Speedup surfaces $S(\alpha, \gamma, c)$ for three draft costs ([D]). Contours mark 1× through 4×; the orange curve is the optimal γ for each α. As c grows the useful region shrinks and the optimal γ collapses toward 1–3.

4.3.2 A closed form via the Lambert W function

Treating γ as continuous and setting $\partial S/\partial\gamma = 0$ in (4.3) gives the first-order condition $\alpha^{\gamma+1}(1 - s\ln\alpha) = 1$ with $s = \gamma + 1/c$ (derivation in Appendix A.4). Substituting $y = -s\ln\alpha$ turns this into $(1+y)e^{-(1+y)} = \kappa/e$ with $\kappa = \alpha^{1/c-1}$, whose relevant solution lies on the lower branch of the Lambert W function:

$$\gamma^\star = \frac{-1 - W_{-1}\big(-\alpha^{1/c-1}/e\big)}{-\ln\alpha} - \frac{1}{c}. \tag{4.5}$$

We verified (4.5) against exhaustive search (Table 4.4); the continuous optimum rounds to (or next to) the best integer in every tested case, and flatness near the optimum means that integer rounding costs at most a few percent of speedup.

Rules of thumb ([D], from Table 4.3). With a cost ratio of $c = 0.05$ (a draft pass costing 5% of a target pass, plausible for a one-layer EAGLE head plus language-model head) the optimal lookahead is 4 at $\alpha = 0.6$, 8 at $\alpha = 0.8$ and 13 at $\alpha = 0.9$. At $c = 0.2$ the optimum falls to 2–7. These optima are for a *chain* draft at batch size one and a memory-bound verifier; they shift lower under load and are bounded in practice by position-dependent acceptance decay, as the next section shows.

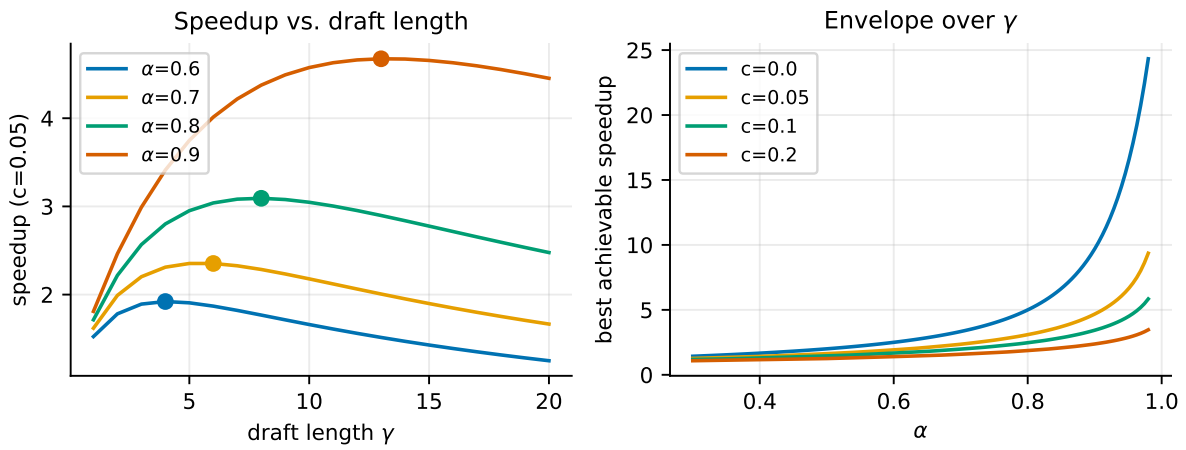


Figure 4.2: Left: speedup against γ for $c = 0.05$ with the optimum marked. Right: the best achievable speedup (envelope over γ) as a function of α for four draft costs ([D]).

Table 4.4: Continuous optimum from (4.5) versus exhaustive integer search ([D], checked numerically with scipy; $\gamma \leq 199$).

α	c	Continuous $\gamma^\star$ (Lambert W)	Best integer γ (grid)	Speedup at best integer
0.6	0.02	5.62	6	2.170×
0.6	0.05	4.06	4	1.921×
0.6	0.1	2.98	3	1.674×
0.6	0.2	1.97	2	1.400×
0.7	0.02	7.61	8	2.758×
0.7	0.05	5.48	6	2.353×
0.7	0.1	4.02	4	1.981×
0.7	0.2	2.70	3	1.583×
0.8	0.02	11.02	11	3.817×
0.8	0.05	7.86	8	3.092×
0.8	0.1	5.76	6	2.470×
0.8	0.2	3.90	4	1.868×
0.9	0.02	19.06	19	6.365×
0.9	0.05	13.29	13	4.674×
0.9	0.1	9.65	10	3.431×
0.9	0.2	6.56	7	2.373×

4.4 Beyond i.i.d.: position decay

Real drafters become less accurate with depth, particularly feature-conditioned ones that must extrapolate on their own outputs, and parallel drafters that predict several tokens at once. EAGLE-3's training-time test is a direct attempt to flatten this decay; the EAGLE-3 paper plots acceptance rates n-α against the number n of draft-generated inputs and reports that EAGLE's acceptance drops significantly as n grows, whereas EAGLE-3's stays almost unchanged[S] [23].

With position-dependent rates, (4.1) applies. A concrete case: the MLSys 2026 study reports, for GLM-4.5-Air with its single released multi-token-prediction (MTP) head reused autoregressively on GPQA-Main, position-wise acceptance of 0.92, 0.68 and 0.38[S] [27]. If these are conditional rates $\alpha_1, \alpha_2, \alpha_3$ then $\mathbb{E}[N] = 1 + 0.92 + 0.92 \cdot 0.68 + 0.92 \cdot 0.68 \cdot 0.38 = 2.78$; if instead they are marginal acceptance probabilities of each position then $\mathbb{E}[N] = 1 + 0.92 + 0.68 + 0.38 = 2.98$[D](the source text does not say which convention it uses). An i.i.d. model with the same

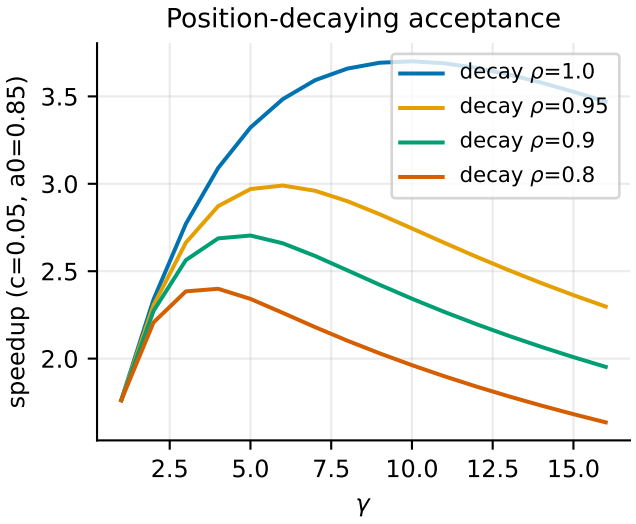


Figure 4.3: Speedup against γ when the conditional acceptance decays geometrically with position, $\alpha_k = 0.85\rho^{k-1}$, $c = 0.05$ ([D]). Stronger decay (smaller ρ) lowers and flattens the optimum.

mean rate $\bar{\alpha} = 0.66$ would predict $\mathbb{E}[N] = 2.38$, so the i.i.d. formula *under*-estimates here, since the first position is so much better than the average. Figure 4.3 shows how decay with depth flattens the speedup curve and moves the optimal lookahead down.

4.5 Beyond i.i.d.: heterogeneity across contexts

The acceptance rate also varies from context to context: code boilerplate is easy, novel reasoning is hard. The MLSys 2026 study observes that acceptance varies at three levels—within a request, across requests, and across datasets[S] [27]. A neat consequence of convexity:

Proposition 4.5 (Heterogeneity helps tokens per round)**.** *Let α be random with mean $\bar{\alpha}$ and let γ be fixed. Then $\mathbb{E}[\mathbb{E}[N \mid \alpha]] \geq \mathbb{E}[N \mid \bar{\alpha}]$.*

Proof. $f(\alpha) = \sum_{k=0}^{\gamma} \alpha^k$ is convex on $[0, 1]$ (a sum of convex monomials), so Jensen's inequality gives $\mathbb{E}f(\alpha) \geq f(\mathbb{E}\alpha)$. □

Hence a dataset-average acceptance rate fed into (4.2) *underestimates* the tokens per round when acceptance is spread across requests (for fixed γ and equal cost per round). Table 4.5 illustrates for $\bar{\alpha} = 0.7$, $\gamma = 5$: a Beta-distributed α with concentration 2 gives $\mathbb{E}[N] = 3.53$ against 2.94 for constant α ([M]). The converse is the opportunity highlighted by adaptive schemes: since easy and hard contexts differ greatly, setting γ per request (as in SmartSpec's goodput-based selection [28], the dynamic trees of EAGLE-2 [22], or ATLAS's confidence-aware controller [50]) can raise speedup further. The Jensen result also warns that comparisons of averaged acceptance rates between methods can hide differences in their *spread*.

4.6 Worked examples

Example 4.6 (A 70B target with a 1B draft)**.** Suppose $\alpha = 0.75$ (the acceptance reported for the 70B setup in Liu et al. [27]), $\gamma = 3$ (the lookahead they use for draft-model SD) and

Table 4.5: Effect of heterogeneity at fixed mean acceptance $\bar{\alpha} = 0.7$, $\gamma = 5$, $c = 0.05$ ([M]). Speedup is computed from the mean tokens per round and the constant round time.

Beta concentration	Mean α	E[tokens] heterogeneous	E[tokens] at mean α	Speedup het. / iid
10^6 (nearly constant)	0.700	2.941	2.941	2.35 / 2.35
20	0.700	3.033	2.941	2.43 / 2.35
5	0.700	3.250	2.939	2.60 / 2.35
2	0.700	3.533	2.941	2.83 / 2.35

Table 4.6: Effective i.i.d. acceptance rates implied by published acceptance lengths ([D], by numerical inversion of (4.2)); the acceptance lengths are [S] except where the source is a vendor blog ([V]).

Source	Dataset	Method	AL	K	Implied α
P-EAGLE blog (B200, GPT-OSS-20B) K=7	HumanEval	P-EAGLE	3.94	7	0.782
P-EAGLE blog K=7	HumanEval	EAGLE-3	3.03	7	0.686
P-EAGLE blog K=7	SPEED-Bench	P-EAGLE	3.38	7	0.727
P-EAGLE blog K=7	SPEED-Bench	EAGLE-3	2.59	7	0.623
P-EAGLE blog K=7	MT-Bench	P-EAGLE	3.7	7	0.760
P-EAGLE blog K=7	MT-Bench	EAGLE-3	3.27	7	0.715
P-EAGLE paper K=5 (Qwen3-Coder-30B)	HumanEval	P-EAGLE	4.5	5	0.883
P-EAGLE paper K=5 (Qwen3-Coder-30B)	HumanEval	EAGLE-3	4.4	5	0.874

$c = 0.125$ (their reported draft/target time ratio). Then $\mathbb{E}[N] = (1 - 0.75^4)/(0.25) = 2.734$ and $S = 2.734/(1 + 3 \cdot 0.125) = 1.99$[D]. The same study measures, for EAGLE on a 70B target with ShareGPT, 1.96× at batch size 1 falling to 1.72× at batch size 32[S], so the order of magnitude agrees; the formula neglects scheduling overhead, rejection sampling and the fact that verification of four tokens costs slightly more than one.

Example 4.7 (Implied acceptance rates from published acceptance lengths)**.** Acceptance lengths τ (mean emitted tokens per round) are routinely reported instead of α. Inverting (4.2) gives an *effective* i.i.d. rate. For the vLLM project's P-EAGLE blog, with lookahead $K = 7$ on GPT-OSS-20B, the reported acceptance lengths of 3.94 (P-EAGLE) and 3.03 (EAGLE-3) on HumanEval correspond to effective rates 0.78 and 0.69; on MT-Bench 3.70 and 3.27 correspond to 0.76 and 0.72[D](Table 4.6). These are only effective summaries (real position-wise rates decay), but they indicate that the gain of P-EAGLE over EAGLE-3 in this benchmark is largest on HumanEval and SPEED-Bench (effective per-token rate up by about 0.10) and smaller on MT-Bench (about 0.045).

4.7 Throughput, latency and goodput

Three different metrics are used in the literature, and speedups are not comparable across them:

- **Per-request latency or tokens/s at batch size one** (used by Leviathan, EAGLE, Saguaro): what a single user experiences when the system is otherwise idle.
- **Aggregate throughput at a given batch size or concurrency** (used by EAGLE-3's SGLang table, P-EAGLE's vLLM table, the MLSys study): tokens/s summed over concurrent requests.
- **Goodput** (SmartSpec [28]): the rate of tokens generated and kept, counting accepted drafts plus the bonus token, as opposed to throughput that counts every token the target scored including rejected drafts.

The SmartSpec paper's own illustration, reported for a LLaMA-7B target with a LLaMA-160M draft on one A100 with acceptance 0.7, is that proposing five tokens helps at a request rate of 4 and gives no benefit beyond about 12, while proposing three tokens still hurts above about 16[S]. The lesson is that a draft length which is optimal at low load can be harmful at high load; we reproduce the effect with our roofline model in Chapter 5.

Chapter 5

Roofline Analysis: Why Speculation Works and When Batching Erases the Gain

Chapter 4 treated the cost of verification as a constant T. This chapter replaces that with a hardware model, derives a closed-form description of how speedup decays with batch size, and then simulates a more detailed model. Everything computed here is our own derivation or simulation; no real GPU was used. The purpose is to understand *shapes* and *crossover orders of magnitude*, not to predict absolute numbers.

5.1 A step-time model

Using (2.2) with the byte and FLOP counts of a dense transformer, the time of a target step that processes m tokens for each of B sequences with context length S is

$$t_{\text{tgt}}(B, m, S) = \max\left\{ \frac{bP + B\,S\,\kappa}{\eta_W\,W},\ \frac{B\,m\,(2P + 4LHd_h\,S)}{\eta_F\,F} \right\}, \tag{5.1}$$

where W, F are peak bandwidth and (dense BF16) FLOP rates, $\eta_W, \eta_F \le 1$ are achieved efficiencies, L is the number of layers, H the number of query heads and d_h the head dimension. The term $4LHd_hS$ is the attention FLOPs per new token ($\mathrm{QK}^\top$ and AV, each $2Sd_h$ per head per layer). A drafter that is a single decoder layer plus a language-model head, with ϕP parameters ($\phi \approx 0.03$ in our runs, in the 2–5% range quoted for EAGLE drafters by the P-EAGLE paper[S]), costs

$$t_{\text{dr}}(B, S) = \max\left\{ \frac{b\phi P + BS\kappa/L}{\eta_W W},\ \frac{2\phi PB}{\eta_F F} \right\} + t_{\text{launch}} \tag{5.2}$$

for one draft token per sequence, with a fixed per-pass launch overhead $t_{\text{launch}} = 0.15$ ms. The chain-speculation speedup in throughput relative to autoregressive decoding at the same (B, S) is then

$$S_{\text{thr}}(B, S; \alpha, \gamma) = \frac{\mathbb{E}[N]\ t_{\text{tgt}}(B, 1, S)}{\gamma\, t_{\text{dr}}(B, S) + t_{\text{tgt}}(B, \gamma + 1, S)}. \tag{5.3}$$

Parameters (all assumptions, [M]). H100 SXM: $W = 3.35$ TB/s, dense BF16 $F = 989$ TFLOP/s (half the sparse datasheet figure); $\eta_W = 0.8$, $\eta_F = 0.6$; model: Llama-3.1-8B shape ($P = 8.03$B, $L = 32$, $H = 32$, $d_h = 128$, $\kappa = 131{,}072$ B). The weights-only ridge is $\eta_F F/(\eta_W W) \approx 221$ tokens per weight pass.

5.2 A closed form for the weights-only regime

Ignoring attention and KV cache ($S = 0$) and the drafter, (5.1) reduces to $t(B, m) = t_0 \max\{1,\ Bm/n^\star\}$ with $t_0 = bP/(\eta_W W)$ and $n^\star = \eta_F F/(\eta_W W)$ (for $b = 2$ bytes per parameter, so that arithmetic intensity equals tokens per pass), i.e. the ridge in tokens per pass (221 above).

Table 5.1: Check of Theorem 5.1: $\alpha = 0.8$, $\gamma = 4$, $n^\star = 221.4$ ([M] versus [D]).

Batch B	Simulator (ctx 0, free draft)	Piecewise formula
1	3.362	3.362
8	3.362	3.362
32	3.362	3.362
44	3.362	3.362
64	2.326	2.326
100	1.489	1.489
148	1.006	1.006
200	0.744	0.744
221	0.674	0.674
300	0.672	0.672
512	0.672	0.672

Theorem 5.1 (Batch-size dependence of speculative speedup; weights-only model)**.** *Let* $\mathbb{E}[N]$ *be the expected tokens per round for lookahead* γ *and assume a free draft. Then*

$$S_{\text{thr}}(B) = \begin{cases} \mathbb{E}[N] & B \le n^\star/(\gamma+1), \\ \mathbb{E}[N]\,\dfrac{n^\star}{B(\gamma+1)} & n^\star/(\gamma+1) < B \le n^\star, \\ \mathbb{E}[N]/(\gamma+1) & B > n^\star. \end{cases}$$

In particular speculation breaks even at $B_{\text{be}} = \mathbb{E}[N]\, n^\star/(\gamma+1)$ *and is a strict slowdown for every* $B > B_{\text{be}}$ *since* $\mathbb{E}[N] \le \gamma + 1$.

Proof. The autoregressive step takes $t_0 \max\{1, B/n^\star\}$ and yields B tokens; the speculative step takes $t_0 \max\{1, B(\gamma+1)/n^\star\}$ and yields $B\,\mathbb{E}[N]$. Taking the ratio of throughputs and examining the three regimes of the two maxima gives the three cases. For $B > n^\star$ both steps are compute-bound and the ratio is $\mathbb{E}[N]/(\gamma+1) \le 1$, with equality only if every draft is accepted ($\alpha = 1$). The break-even point follows from setting the middle case to 1. Since $1 < \mathbb{E}[N] < \gamma + 1$ for $0 < \alpha < 1$, we have $n^\star/(\gamma+1) < B_{\text{be}} < n^\star$, so it indeed lies in the middle regime. □

Reading the theorem. The speedup plateau at $\mathbb{E}[N]$ extends only to $B = n^\star/(\gamma+1)$: with $n^\star = 221$ and $\gamma = 4$ this is $B \approx 44$. Beyond it, speedup declines as $1/B$ until it hits the floor $\mathbb{E}[N]/(\gamma+1)$, i.e. the *inverse of the FLOP overhead* (4.4): a compute-bound system pays for every rejected token. Longer lookahead *shortens* the plateau (it multiplies the tokens per sequence per step) while raising its height only logarithmically. At $\alpha = 0.8$, $\gamma = 4$: $\mathbb{E}[N] = 3.36$, break-even $B_{\text{be}} \approx 149$, and the floor is 0.67.

We validated the theorem against the simulator of (5.1) with $S = 0$ and a free draft (Table 5.1); they agree to all printed digits, as they must since this is an algebraic check of the derivation.

5.3 Full simulation with context and drafting cost

Figure 5.1 shows (5.3) with realistic context lengths and a non-free draft.

Three observations (all [M]). First, at short context ($S = 512$) the model shows a plateau of about 2.76× out to $B = 32$, then decline: 2.47× at $B = 64$, 1.60× at 128, 1.12× at 256 and 0.86× at 512. The decline is slower than the weights-only theorem because the KV cache adds bytes that

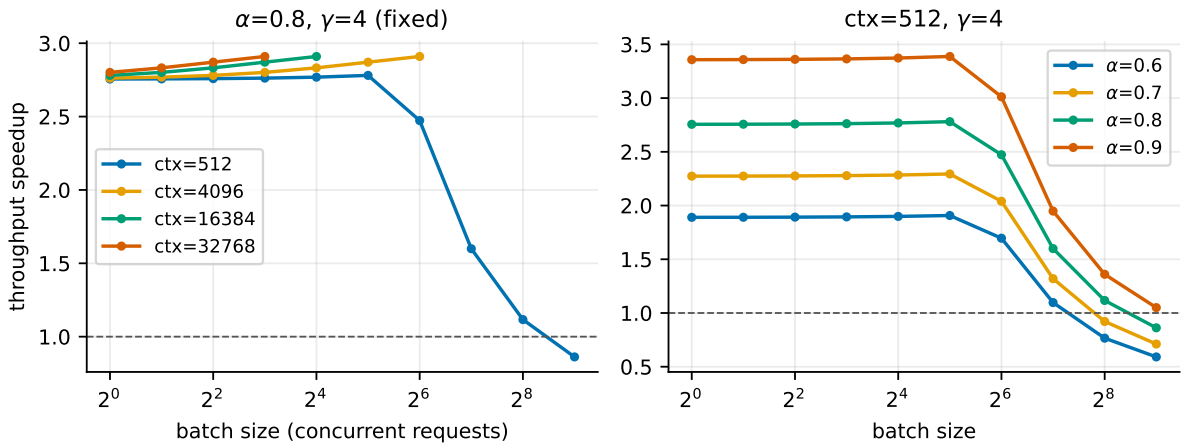


Figure 5.1: Modelled throughput speedup of chain speculation ([M]). Left: $\alpha = 0.8$, $\gamma = 4$, for four context lengths; points are plotted only where $BS\kappa$ fits in free HBM (about 60 GB). Right: ctx $= 512$ for four acceptance rates. The dashed line marks no gain.

Table 5.2: First batch size at which the modelled speedup falls below 1, for context 512 ([M]). Under the weights-only theorem the corresponding values would be $B_{\text{be}} = \mathbb{E}[N]\, n^\star/(\gamma+1)$.

α	$\gamma = 2$	$\gamma = 4$	$\gamma = 6$
0.6	317	151	93
0.7	426	215	136
0.8	607	333	221
0.9	959	609	440

do not scale with m (so the *autoregressive* step is also pushed up by memory traffic). Second, the batch size at which speculation stops helping depends on both α and γ (Table 5.2): 151 for $(\alpha, \gamma) = (0.6, 4)$ against 609 for $(0.9, 4)$, and shrinking as γ grows (333 versus 221 for $\alpha = 0.8$ at $\gamma = 4$ versus 6). Third, at long context ($S \geq 4096$) the modelled speedup *increases* slightly with batch size (2.76× at $B = 1$ to 2.91× at the largest feasible batch), the MagicDec effect discussed in Section 5.6.

Calibration against published batch effects. Our idealised model places the break-even batch in the hundreds, far above the values published for real systems, where gains are already small at batch sizes of a few tens: EAGLE-3 retains 1.38× at batch 64 in SGLang (a chain of length 3), while the older EAGLE falls to 0.88–0.99× for batches from 24 to 64[S](Table 5.4, Figure 5.2); in vLLM, EAGLE-3 is at 1.01× at batch 56[S]. The discrepancy is itself informative. Table 5.3 shows that lowering the assumed achievable FLOP efficiency from 0.6 to 0.15 moves the break-even for $\alpha = 0.8$, $\gamma = 4$ from 333 to 41, and to 20 at $\eta_F = 0.08$ ([M]). Real inference stacks achieve a small fraction of peak tensor throughput at the odd shapes of verification (small M, tile quantisation), non-GEMM kernels and sampling take additional time, and acceptance varies across requests. We do *not* claim to have identified the cause of the published crossovers; we only note that a modest effective efficiency reproduces their order of magnitude.

Table 5.3: Sensitivity of the break-even batch size to the assumed FLOP efficiency ($\alpha = 0.8$, $\gamma = 4$, ctx 512; [M]).

Assumed FLOP efficiency	Crossover batch size
0.6	333
0.3	97
0.15	41
0.08	20

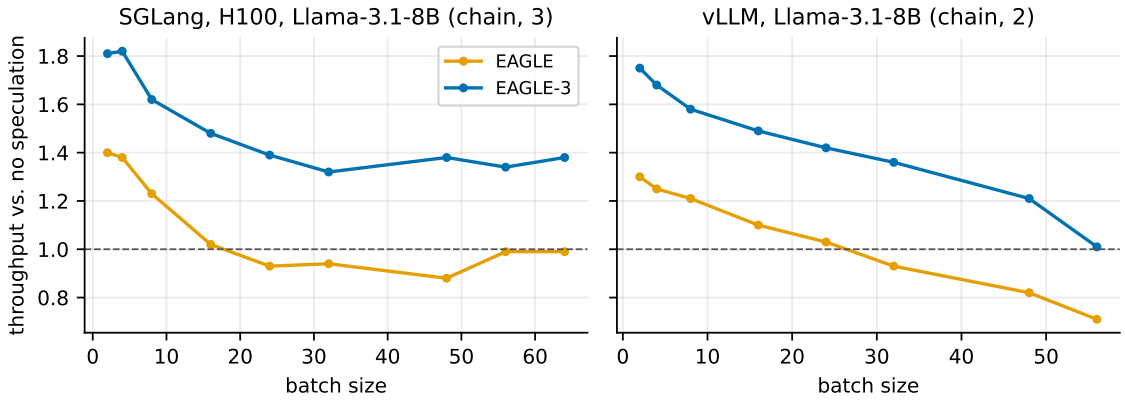


Figure 5.2: Published throughput relative to a non-speculative baseline versus batch size, for EAGLE and EAGLE-3 ([S]). Left: SGLang on H100 with Llama-3.1-8B (MT-Bench, chain of length 3, measured by the SGLang team; Table 3 of Li et al. [23]). Right: vLLM with Llama-3.1-8B (chain of length 2; Table 5 of the same paper, whose text and caption disagree on whether the GPU was an RTX 3090 or an A100).

5.4 Verification time and the step-time curve

Figure 5.3 plots the modelled verification step time against batch size for different numbers of tokens per request. For small B the curves coincide (memory-bound); they separate when the larger token count crosses the ridge, with the 17-token curve peeling off at roughly $B \approx 13$ ($= 221/17$). This picture explains a qualitative finding of the MLSys 2026 study that verification, not drafting, dominates run time: they report verification at 42–95% of execution time across configurations (growing with model size and batch size) while drafting was below 2% for n-gram and 12–20% at batch size one (3–7% at batch 512) for EAGLE/EAGLE-3[S] [27].

5.5 Adaptive speculation under load

The remedy suggested by theory and by SmartSpec [28] is to choose γ per step as a function of the load, including $\gamma = 0$. Figure 5.4 compares fixed $\gamma \in \{2, 5, 8\}$ with the best $\gamma \in \{0, \ldots, 12\}$ at each batch size in our model ($\alpha = 0.75$, ctx 1024). The adaptive curve is the upper envelope: it matches the long-lookahead curves at low load and descends to exactly 1.0 (no speculation) at the highest loads, whereas fixed lookaheads fall below 1. This is the same qualitative behaviour reported by SmartSpec, which found that fixed proposal lengths became counter-productive at high request rates and that its goodput-based adaptation reduced average latency by up to 3.2× relative to non-speculative decoding across settings[S] [28]. TensorRT-LLM's documentation, by contrast, notes that speculation cannot be turned off dynamically there, so that speedups are seen only at low batch sizes[S] [39].

Table 5.4: Throughput relative to no speculation, SGLang, H100, Llama-3.1-8B, MT-Bench, chain draft of length 3 ([S]; EAGLE-3 paper Table 3, measured by the SGLang team).

Batch size	2	4	8	16	24	32	48	56	64
EAGLE	1.40	1.38	1.23	1.02	0.93	0.94	0.88	0.99	0.99
EAGLE-3	1.81	1.82	1.62	1.48	1.39	1.32	1.38	1.34	1.38

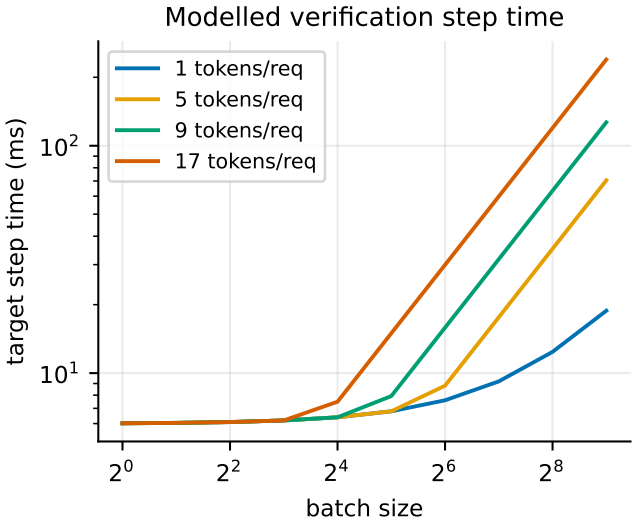


Figure 5.3: Modelled target step time against batch size for 1, 5, 9 and 17 tokens per request (ctx 512; [M]). The curves share a memory-bound floor of roughly 6 ms and leave it at $B \approx 221/m$.

5.6 Long contexts and the MagicDec effect

When the KV cache dominates memory traffic, verification of $\gamma + 1$ tokens per sequence reads the same cache as a one-token step. In (5.1), the bytes term is independent of m while the FLOP term grows with m but is small relative to bytes when S is large. Provided the step stays memory-bound, $S_{\text{thr}} \to \mathbb{E}[N] \,/\, (1 + \gamma t_{\text{dr}}/t_{\text{tgt}})$ and, because the draft's own cost shrinks relative to the target's (its KV cache is smaller per layer), the ratio can even improve with B. Our simulation shows exactly this: at ctx 4,096, 16,384 and 32,768, the modelled speedups at the largest feasible batch sizes (64, 16 and 8) are 2.91× compared with 2.76–2.80× at $B = 1$ ([M]). The effect is small in our model because the draft is cheap; MagicDec makes it large by using a *sparse-KV* version of the target as the drafter, so that drafting is cheap even though context is long, and reports speedups up to 2.51× for LLaMA-3.1-8B at batch sizes 32–256, with examples of 1.42× (4×H100, 32K context, batch 32) and 1.68× (LLaMA-2-7B-32K, 4×H100, 8K, batch 64)[S] [43]. They also report that GPUs with higher compute-to-bandwidth ratio (H100) beat A100 and L40S under identical settings[S], consistent with the model: a higher ridge point delays the compute-bound regime.

Two caveats. The modelled long-context speedups require the KV cache to fit: at 32K context and $\kappa = 128$ KiB each sequence needs 4 GiB, so a single 80 GB GPU holds fewer than 15 such sequences after weights (the feasibility limit in Figure 5.1); MagicDec uses multi-GPU tensor parallelism for its larger batches. And the benefit depends on the drafter's attention being cheaper than the target's, which is a property of the draft design, not of speculation per se.

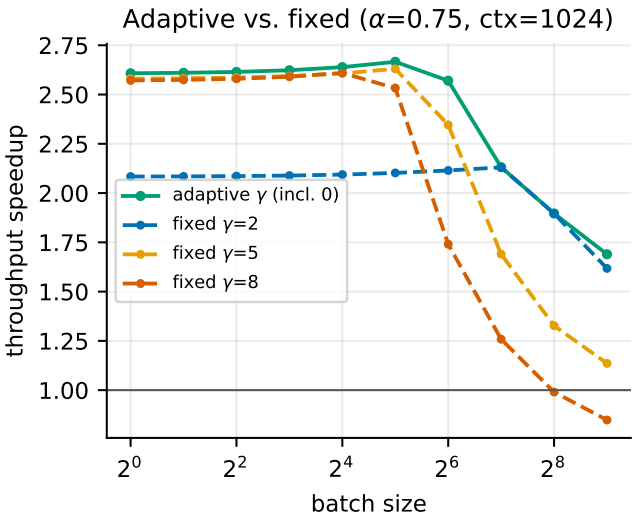


Figure 5.4: Adaptive versus fixed draft length ([M]). The green curve selects the best $\gamma \geq 0$ at each batch size.

5.7 Mixture-of-experts: verification touches more weights

For an MoE layer with E experts and top-k routing, the weights read in a step with n tokens are proportional to the number of *distinct* experts touched. If routing were uniform and independent across tokens, the expected fraction of experts touched by n tokens would be $1 - (1 - k/E)^n$. For $E = 128$, $k = 8$ (a configuration of the Qwen3-30B-A3B family; we use it only as an example) this gives 6.2% of experts for one token, 27.6% for five tokens, 66.6% for seventeen, and 98.5% for 65 ([D]). A five-token verification step therefore reads about 4.4× more expert weight bytes than a single-token step at batch one. Real routing is correlated across neighbouring tokens (which reduces the penalty) and skewed across experts, so this is an upper-bound illustration. It nonetheless matches the qualitative statements in the P-EAGLE paper (smaller gains for the larger MoE model at higher concurrency[S]) and in a secondary summary of the Gemma 4 MTP announcement (batch-one MoE verification loads extra experts[V]). The sign of the batch-size dependence can differ from the dense case: at *large* batch every expert is touched by ordinary decoding anyway, so extra verified tokens add no extra expert bytes, only FLOPs.

5.8 Summary of the regime picture

Where speculative decoding pays

1. Pays most when the verifier is memory-bound for $B(\gamma + 1)$ tokens, i.e. small batch, dense model, short-to-medium context.
2. Breaks even near $B \approx \mathbb{E}[N]\, n^{\star}_{\text{eff}}/(\gamma + 1)$ where $n^{\star}_{\text{eff}}$ is the *effective* ridge (lower than the datasheet ridge in real stacks); strictly hurts beyond that for chain drafts.
3. Can retain or increase its benefit at large batch when long-context KV traffic dominates and the drafter is cheap (MagicDec).
4. MoE models can invert the dependence on batch size.

5. Adaptive γ that includes zero is the safe default for shared serving.

Chapter 6

Multiple Candidates and Token-Tree Verification

A chain of γ draft tokens is wasted from the first rejection onward. If the drafter can cheaply offer several alternatives at the same position, the verifier has more chances to find an acceptable continuation. Organising the alternatives as a *token tree* lets all of them be checked in one target forward pass using a tree-structured attention mask, an idea introduced for serving by SpecInfer [33] and used by Medusa [5], EAGLE and EAGLE-2 [21, 22], Sequoia [8] and, as a dynamic tree, by EAGLE-3 [23]. This chapter gives the mathematics of exact multi-candidate verification, the expected acceptance length of a tree, and the interaction with the batching regime.

6.1 Tree attention in one paragraph

Let the tree $\mathcal{T}$ have root equal to the last verified token and n draft nodes. The target is run once on all $n+1$ tokens with (i) position ids equal to node depth (so that each branch sees correct rotary positions) and (ii) an attention mask in which node v attends to the prefix and to its ancestors only. The output at node v is the target's next-token distribution given the path from root to v. The cost of the pass is that of $n+1$ new tokens, i.e. $m = n+1$ in (5.1). After verification, the KV entries of the accepted path are retained and the others discarded; this requires either gathering entries into contiguous locations or the paged layout of production engines, one source of implementation complexity (Chapter 12). The Saguaro paper notes that custom-mask attention kernels with non-coalesced access patterns can dominate the critical path in its own implementation[S] [18].

6.2 Exact multi-candidate verification

Consider one node. The drafter provides k candidates $X_1, \dots, X_k$ for the next token. We want to output a token $X \sim p$ (the target conditional at that node), preferably equal to one of the candidates.

6.2.1 Recursive rejection sampling

Definition 6.1 (Recursive rejection sampling, RRS)**.** Let $X_1, \dots, X_k$ be i.i.d. draws from q. Set $p^{(0)} = p$. For $i = 1, \dots, k$: accept X_i with probability $\min\{1, p^{(i-1)}(X_i)/q(X_i)\}$ and stop; otherwise set $p^{(i)} = \mathrm{norm}\big(\left(p^{(i-1)} - q\right)_+\big)$ and continue. If all candidates are rejected, output a draw from $p^{(k)}$.

Theorem 6.2 (RRS is exact)**.** *For every $k \geq 0$, the output of RRS has law p. If the candidates are*

Table 6.1: Recursive rejection sampling with k i.i.d. candidates ([M] versus [D]). TV is the distance of the output histogram from p (3×10^5 trials).

k	TV to p (MC)	Accept (MC)	Accept (theory)
1	0.0006	0.6246	0.6247
2	0.0008	0.7785	0.7785
3	0.0012	0.8683	0.8693
4	0.0009	0.9228	0.9229
8	0.0012	0.9905	0.9907

rejected one after another, the probability that at least one is accepted is

$$A_k(p,q) = 1 - \prod_{i=1}^{k} \mathrm{TV}\big(p^{(i-1)}, q\big). \tag{6.1}$$

Proof. Exactness by induction on k. For $k = 0$ the output is a draw from p. Assume the claim for $k-1$ for every target distribution. Run the first stage with target $p^{(0)} = p$ and draft q. As in Theorem 3.3, the first stage accepts $X_1 = x$ with probability $\min\{p,q\}(x)$ and rejects with probability $1 - \alpha(p,q) = \mathrm{TV}(p,q)$. On rejection the remaining procedure is RRS with $k-1$ candidates targeting $p^{(1)} = \mathrm{norm}((p-q)_+)$, which by the induction hypothesis outputs a draw from $p^{(1)}$ (the remaining candidates $X_2, \ldots, X_k$ are independent of the first-stage randomness). Hence

$$\mathbb{P}[\text{output} = x] = \min\{p,q\}(x) + \mathrm{TV}(p,q) \cdot \frac{(p(x) - q(x))_+}{\mathrm{TV}(p,q)} = p(x),$$

as in the one-step proof.

Acceptance probability. Stage i is reached only if the previous $i-1$ stages rejected; since the candidates are independent draws and $p^{(i-1)}$ is a deterministic function of (p, q, i), the conditional probability of rejection at stage i is $1 - \alpha(p^{(i-1)}, q) = \mathrm{TV}(p^{(i-1)}, q)$. Multiplying over stages gives the probability that all are rejected. □

Numerical check ([M]). For a random target/draft pair on a vocabulary of eight tokens, we ran RRS 3×10^5 times for each $k \in \{1,2,3,4,8\}$ (Table 6.1). The empirical output stays within TV of about 0.001 of p (the sampling noise level for this sample size), and the observed acceptance matches (6.1) to within 10^{-3}. Acceptance rises from 0.62 ($k = 1$) to 0.78, 0.87, 0.92 and 0.99 ($k = 8$): diminishing returns, because each additional candidate sees a residual target that is concentrated on tokens the draft is *least* likely to propose.

6.2.2 With-replacement versus without-replacement candidates

RRS as defined draws the candidates *with* replacement, so the same token can appear twice among the candidates of a node; duplicates waste verification budget. Sequoia [8] instead builds trees by sampling *without* replacement and introduces a verification scheme designed to remain correct and to behave well across temperatures; its abstract reports that it improves decoding speed of Llama2-7B, Llama2-13B and Vicuna-33B on an A100 by up to 4.04×, 3.73× and 2.27× respectively, and, in an offloading setting for Llama3-70B-Instruct on a single L40, reduces per-token latency to 0.60 s/token, 9.5× faster than DeepSpeed-Zero-Inference[S]. In the greedy case (p a point mass) none of this matters: a child is accepted iff it equals the target's arg-max, and the tree simply needs to contain that token.

6.2.3 Trees of depth greater than one

A tree is verified recursively: at each node the children are tested by RRS against the target conditional at that node; if one child is accepted the walk descends into it, otherwise the output at that node is the resampled token and the walk ends. By Theorem 3.5 (applied node by node, with the same conditional-independence argument as in the proof there), the emitted sequence has the target law.

6.3 Expected acceptance length of a tree

6.3.1 A path-sum formula

Let $\mathcal{T}$ be a rooted tree (root = last verified token) and for each non-root node v let $a(v)$ be the probability that v is accepted *given that its parent was accepted*. Let $\pi(v) = \prod_{u \in \text{path}(v)} a(u)$ be the probability that the whole path to v is accepted. For the verifier to accept v's path it must accept every ancestor; the events "accept v" for different children of the same node are mutually exclusive.

Proposition 6.3 (Expected tokens of a tree)**.** *The expected number of tokens emitted by a round that verifies tree $\mathcal{T}$ is*

$$\mathbb{E}[N] = 1 + \sum_{v \in \mathcal{T} \setminus \{\text{root}\}} \pi(v). \tag{6.2}$$

Proof. $N = 1 + D$ with D the depth of the accepted path. Write

$$D = \sum_{v} \mathbf{1}[\text{path to } v \text{ accepted and } v \text{ is the accepted node at its depth}].$$

The accepted path contains exactly one node at each depth up to D, and the indicator that the walk visits v is exactly the indicator that the path to v is accepted (the walk follows the unique accepted child at every node). Taking expectations gives $\sum_v \pi(v)$. □

For a chain, $\pi(v_k) = \prod_{j \le k} \alpha_j$ and (6.2) reduces to (4.1). The formula shows that every node is worth its $\pi(v)$, *independent of the other nodes* that were chosen, provided $a(v)$ itself does not depend on its siblings. For RRS this is false (sibling acceptance probabilities interact through the residuals, (6.1)), but for greedy verification—where $a(v) = \mathbb{P}[\text{target arg-max} = v \mid \text{parent accepted}]$ and sibling events are disjoint—it holds exactly, and it is the approximation underlying draft-tree construction in EAGLE-2 (which uses the draft model's confidence as an estimate of acceptance probability; its authors report that the draft is well calibrated[S] [22]).

6.3.2 Optimal trees under a node budget

Suppose we may verify n draft nodes, with $a(v)$ given and ancestors' values at least as large as descendants'.

Proposition 6.4 (Greedy best-first trees are optimal for a modular objective)**.** *If $\mathbb{E}[N] = 1 + \sum_{v \in \mathcal{T}} \pi(v)$ and π is non-increasing along every root-to-leaf path (which holds because each $a(v) \le 1$), then the set of n nodes of largest π is a valid (ancestor-closed) subtree and maximises $\mathbb{E}[N]$ among all subtrees with n nodes.*

Table 6.2: Expected tokens per round (Eq. (6.2)) of the best-first tree versus a chain with the same number of verified draft nodes (chain truncated at depth 10), under the rank model with $a = 0.6$, $b = 0.3$ ([M], exact enumeration).

Nodes n	Best-first tree	Chain (depth cap 10)	Tree depth
1	1.600	1.600	1
2	1.960	1.960	2
3	2.176	2.176	3
4	2.356	2.306	3
6	2.594	2.430	4
8	2.779	2.475	5
12	3.028	2.491	5
16	3.191	2.491	6
24	3.425	2.491	7
32	3.588	2.491	7
48	3.799	2.491	8
64	3.944	2.491	9

Proof. The objective is additive over nodes, so among *all* subsets of n nodes the n largest values maximise it. Ancestor-closure: if v is selected and an ancestor u were not, then $\pi(u) \geq \pi(v)$ means u ties or exceeds v, so some other selected node of value $\leq \pi(u)$ could be swapped; with a consistent tie-breaking rule (prefer shallower nodes) the selected set is ancestor-closed. Thus the unconstrained optimum is feasible and hence optimal for the constrained problem. □

This justifies the best-first expansion used by EAGLE-2 (select the top nodes by estimated path acceptance) and is the continuous analogue of the dynamic-programming construction of Sequoia, which optimises tree *shapes* for a given positional acceptance vector.

6.4 A simulated comparison of trees and chains

We use a rank-based acceptance model: at any node, the target's (greedy) next token equals the draft's rank-r candidate with probability $f_r = a\, b^{r-1}$ for $r = 1, \ldots, R$, independently across nodes, with $a = 0.6$, $b = 0.3$, $R = 6$, so the top candidate matches 60% of the time and the six candidates together cover 85.7% of the target mass ([M]; a toy). The best-first tree under node budget n is built exactly by a priority queue (maximum depth 10).

Table 6.2 shows that with 16 nodes the tree yields 3.19 tokens per round against 2.49 for a chain (which saturates at its depth-10 value), and 64 nodes yield 3.94; a Monte-Carlo run of 2×10^5 rounds on the 16-node tree gave 3.1919 against the formula's 3.1911, validating (6.2)[M]. Trees have steeply diminishing returns in node count: going from 16 to 64 nodes (a fourfold increase in verified tokens) raises $\mathbb{E}[N]$ by 24%.

6.4.1 Trees meet the batch size

The calculation above says nothing about cost. Combining the tree sizes with the roofline model (5.1) (drafting cost proportional to depth, with up to eight tokens per sequence per draft pass), we obtain the modelled throughput speedups of Table 6.3 and Figure 6.1 ([M]).

The ordering flips with load. At $B = 1$ the larger the tree the better (2.02× at 4 nodes to 2.62× at 64). At $B = 16$ the 4-node chain-like tree (2.00×) already beats the 64-node tree (0.78×), and

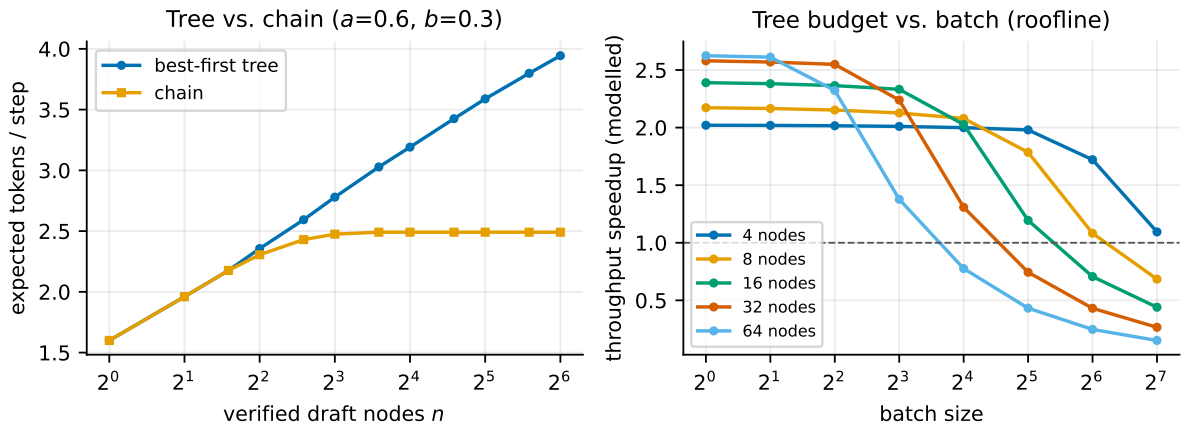


Figure 6.1: Left: expected tokens per round for best-first trees versus chains as the node budget grows ([M]). Right: modelled throughput speedup versus batch size for five tree budgets under the roofline model of Chapter 5 ([M]).

Table 6.3: Modelled throughput speedup of best-first trees (rows: number of verified nodes) across batch sizes, ctx 512 ([M]).

Nodes	B=1	B=2	B=4	B=8	B=16	B=32	B=64	B=128
4	2.02	2.02	2.02	2.01	2.00	1.98	1.72	1.09
8	2.17	2.17	2.15	2.13	2.08	1.79	1.08	0.68
16	2.39	2.38	2.36	2.33	2.03	1.19	0.71	0.44
32	2.58	2.57	2.55	2.24	1.31	0.74	0.43	0.27
64	2.62	2.61	2.32	1.38	0.78	0.43	0.25	0.15

at $B = 64$ every budget of 16 or more nodes is below 1 while the 4- and 8-node trees are still at 1.72× and 1.08×. This agrees qualitatively with the empirical finding of Liu et al. [27] in SGLang v0.5.9: tree-based EAGLE/EAGLE-3 gave slightly higher speedup at batch size 1 (for example, Qwen3-8B on GSM8K: 1.65× for a chain of three, 1.68× for a 6-node tree and 1.85× for a 21-node tree; Llama3-70B on ShareGPT: 1.81×, 1.90×, 2.03×), but by batch size 64 the 21-node tree was below 1× on every workload for both models, while the chain stayed above 1× throughout[S]. They note that the 21-node tree raised the accepted length from 2.25 to 2.92 on Qwen3-8B/GSM8K while the acceptance rate (accepted fraction of verified tokens) fell from 0.415 to 0.095[S]—precisely the *wasted-verification* mechanism of the model.

Trees and production support

As of the documentation retrieved for this report, the TensorRT-LLM page states that tree-structured draft sequences are not supported for EAGLE-3 there (each request gets one linear chain)[S] [39]; the vLLM speculative-decoding page does not describe general tree verification and its only mention of a tree is the suffix-decoding depth setting[S] [52]; and the MLSys study notes that the draft-tree path in the vLLM version it evaluated was not sufficiently optimised for fair comparison, so it used SGLang for tree experiments[S]. The strongest published "EAGLE-3 speedups" (up to 6.5×) are tree-based at batch size one, while chain drafts are what many serving engines actually run at scale.

6.5 Combining trees with the other ideas

Parallel drafters (Chapter 9) produce a chain in one pass; they can also supply the top-k tokens at each position, and DFlash reports comparisons against EAGLE-3 with tree sizes 16 and 60[S] [7]. SSD (Chapter 11) drafts a linear chain for each anticipated verification outcome; its authors note that combining it with token-tree methods amounts to pre-speculating a tree rather than a chain for each outcome, at the cost of a more elaborate attention mask[S] [18]. The joint design space of SSD and trees is, in their words, largely unexplored.

Chapter 7

History and Taxonomy of Speculative Decoding

7.1 Prehistory: blockwise parallel decoding and speculative execution

The idea of generating several tokens and then checking them in parallel predates large language models. Blockwise parallel decoding [46] trained auxiliary prediction heads on a sequence model, proposed a block of future tokens, and verified them with the base model, accepting the longest verified prefix; this was a greedy scheme, and its guarantee was that the output equals greedy decoding. Subsequent encoder–decoder work refined greedy verification, and later surveys credit this lineage with the draft-then-verify template [57]. The analogy invoked by modern authors, including the SSD paper, is *speculative execution* in CPUs: do work that may turn out to be needed, discard it if not [18].

7.2 The founding papers (2022–2023)

Leviathan, Kalman and Matias. Posted in November 2022 (ICML 2023), this paper introduced *speculative decoding* with the probabilistic acceptance rule of Chapter 3, the analysis of Chapter 4 (expected tokens per iteration, the improvement factor with cost coefficient c, the arithmetic overhead), and experiments with T5 models: out-of-the-box latency improvement of 2×–3× for T5-XXL without any change in outputs[S] [20]. Two details of that paper are still instructive. First, even a trivial bigram drafter gives a nonzero acceptance: for English–German translation with T5-XXL (11B) and a bigram model they report $\alpha \approx 0.2$ and a 1.25× speedup at $\gamma = 3$[S], anticipating prompt-lookup and n-gram speculation. Second, they note that an oracle for the draft length could raise the improvement by up to about 60% over the best fixed γ, anticipating adaptive lookahead[S].

Chen et al. (DeepMind). Posted in February 2023, this paper arrived independently at the same acceptance/rejection rule and demonstrated it on Chinchilla 70B with a 4B-parameter, 8-layer draft model trained with the same tokenizer and data, which samples at 1.8 ms/token against 14.1 ms/token for Chinchilla ($c \approx 0.13$) on 16 TPU v4s[S] [6]. Table 7.1 summarises its headline results at batch size one and $K = 4$. The authors remark that, for HumanEval and greedy XSum, the speedup exceeded the hardware's theoretical memory-bandwidth limit for autoregressive sampling (model size divided by total bandwidth)—possible because speculation emits several tokens per weight read—and they hypothesise that HumanEval's larger speedup stems from common code subsequences, shorter tokens, and temperature sharpening both draft and target logits[S]. They also observe that increasing K beyond a point plateaus or even regresses the speedup, with XSum under nucleus sampling optimal at $K = 3$[S].

Table 7.1: Speculative sampling with Chinchilla 70B at batch size one, $K = 4$ ([S]; Table 1 of Chen et al. [6]). The task metric is unchanged to within noise (e.g. HumanEval pass rate 45.1% autoregressive versus 47.0% speculative).

Setting	Mean token time (ms)	Speedup
XSum, nucleus ($p = 0.8$): autoregressive $\rightarrow$ speculative	14.1 $\rightarrow$ 7.52	1.92×
XSum, greedy	14.1 $\rightarrow$ 7.00	2.01×
HumanEval (100-shot), nucleus ($p = 0.95$, $T = 0.8$)	14.1 $\rightarrow$ 5.73	2.46×

7.3 Trees, heads, and self-speculation (2023–2024)

The next wave attacked the need for a separate small model and the single-chain draft.

- **SpecInfer** [33] (May 2023; ASPLOS 2024) used one or several small speculative models to build a *token tree* verified in one pass with tree attention, and showed that the tree verification preserves the output distribution; the authors report 1.5–2.8× over existing serving systems for distributed inference and 2.6–3.5× for offloading-based inference[S].
- **Online speculative decoding** [26] proposed continuously updating the drafter by distillation on observed user queries; the authors report acceptance-rate increases of 0.1 to 0.65 and latency reductions of 1.42–2.17× in a prototype[S], the intellectual ancestor of Together's ATLAS and of drafters trained on-policy.
- **Medusa** [5] (Jan 2024) attached extra decoding heads to the target's last hidden state, each predicting a token several positions ahead, organised the candidates as a tree, and offered "Medusa-1" (frozen backbone, lossless) and "Medusa-2" (joint fine-tuning); the abstract reports over 2.2× and 2.3–2.8× speedups respectively[S]. Medusa also proposed a "typical acceptance" scheme to raise acceptance at non-zero temperature; this relaxes exactness. Hydra [2] made the heads sequentially dependent.
- **Lookahead decoding** [11] (Feb 2024) used Jacobi iteration to generate and verify n-grams without a drafter or data store, reporting up to 1.8× on MT-Bench and up to 4× with multi-GPU scaling on code completion[S].
- **EAGLE** [21] (Jan 2024) autoregressed at the feature level, and **EAGLE-2** [22] (Jun 2024) introduced context-aware dynamic draft trees, reporting speedups of 3.05–4.26×, 20–40% faster than EAGLE[S].
- **Sequoia** [8] gave a dynamic-programming construction of optimal static trees and a robust sampling and verification method.
- **LayerSkip** [10] trained a model with layer dropout and a shared early-exit loss so that its own early layers draft for its later layers (self-speculation), reporting up to 2.16× on CNN/DM summarisation, 1.82× on coding and 2.0× on TOPv2[S].
- **Spec-Bench** [57] provided a unified benchmark and a survey that most later EAGLE-family papers use for evaluation.
- **Systems analysis**: SmartSpec [28] introduced goodput-driven adaptive speculation length for serving; MagicDec [43] showed that long-context batched regimes can still benefit.

7.4 Overlapping drafting and verification (2024–2025)

A second thread targeted the *serial dependency* between drafting and verification rather than draft quality. DSI [49] proposes "speculation parallelism" with multiple verifier instances so that drafting and verification overlap; its authors prove it is faster than both non-speculative and standard speculative inference under their model and report 1.29–1.92× over standard speculative inference in single-node *simulations*[S]. PEARL [25] introduces *pre-verify* (verify the first draft token during drafting) and *post-verify* (continue drafting during verification), reporting up to 4.43× over autoregressive and 1.50× over vanilla speculative decoding[S]. SwiftSpec [47] disaggregates and pipelines drafting and verification with parallel tree generation and tree-aware KV-cache management, reporting an average 1.75× over state-of-the-art speculative-decoding systems and 348 tokens/s for Llama3-70B on eight Hopper GPUs[S]. SpecBranch [44] borrows branch prediction, preparing parallel speculative branches and reporting 1.8–4.5× over autoregressive decoding with 50% fewer rolled-back tokens for poorly aligned models[S]. Mirror-SD [3] launches branch-complete rollouts from early-exit signals in parallel with the target's suffix across heterogeneous accelerators (GPU and NPU), reporting 2.8–5.8× wall-time speedups for 14B–66B models on SpecBench, a 30% average relative improvement over EAGLE-3[S]. SSD (Chapter 11) generalises all of these: its authors state that PEARL and AMUSD prepare only for the all-accepted outcome, SpecBranch is approximately the special case with a single branching point, and SwiftSpec handles only greedy sampling with a just-in-time fallback[S] [18].

7.5 Built-in multi-token prediction and production adoption

DeepSeek-V3 trained with a multi-token-prediction (MTP) objective [9]; the technical report states that the acceptance rate of the second predicted token ranges from 85% to 90% across generation topics and that this yields a 1.8× improvement in tokens-per-second decoding speed[S]. (The EAGLE-3 authors state that EAGLE inspired DeepSeek-V3's MTP, which in turn inspired new architectural designs in EAGLE-3[S] [23].) By 2026 MTP drafters were released alongside open models: the vLLM documentation lists MTP and notes that Gemma 4 assistant checkpoints must be served with the `mtp` method[S] [52], and Google's May 2026 announcement describes drafters that share the target's embeddings, use its last-layer activations and reuse its KV cache[V] [12].

7.6 The review window (Oct 2025 – Oct 2026) in brief

1. **Consolidation of EAGLE-3 as the default learned drafter**, with training frameworks (SpecForge, SpecBundle, Red Hat's Speculators, AMD/vLLM recipes) and platform recipes (Vertex AI, AWS Neuron; Chapter 12).
2. **Parallel and diffusion drafters**: P-EAGLE (Feb 2026), DFlash (Feb 2026), PARD maturing into vLLM support.
3. **Refinements of EAGLE-3's drafter**: SpecVocab (dynamic vocabulary subsets), ConFu (future-aware contemplate tokens).
4. **Asynchrony generalised**: SSD/Saguaro (Mar 2026), Mirror-SD (Oct 2025).

Table 7.2: Taxonomy and headline reported results. Speedups are *as reported by the authors under their own settings* ([S] unless marked) and are not comparable across rows. "AR": autoregressive baseline. "Exact": distribution-preserving verification as described by the authors.

Method	Draft source	Shape	Schedule	Headline reported result
Leviathan; Chen (2022–23)	separate small LM	chain	sync	2–3× (T5-XXL); 1.92–2.46× (Chinchilla 70B)
SpecInfer (2023)	small LM(s)	tree	sync	1.5–2.8× distributed; 2.6–3.5× offload
Medusa (2024)	extra heads	tree	sync	>2.2× (M-1), 2.3–2.8× (M-2)
Lookahead (2024)	Jacobi n-grams	chain/n-gram	sync	up to 1.8× MT-Bench
EAGLE-2 (2024)	feature-conditioned 1-layer	dynamic tree	sync	3.05–4.26×
LayerSkip (2024)	early layers of target	chain	sync	up to 2.16× (CNN/DM)
EAGLE-3 (2025)	fused multi-layer features, direct tokens	dynamic tree	sync	up to 6.5× (bs 1); 1.38× at bs 64 (SGLang)
DSI (2025)	any	chain	multi-verifier overlap	1.29–1.92× over SI (simulation)
PEARL (2025)	draft model	chain	pre/post-verify	up to 4.43× AR; 1.50× over SD
SwiftSpec (2025)	draft model	tree	async, disaggregated	1.75× avg over SOTA SD systems
PARD (2025)	parallel draft model	chain	sync	3.67× (Llama3.1-8B), 1.15× over EAGLE-3
Mirror-SD (2025)	draft + early-exit	branches	heterogeneous async	2.8–5.8×; ∼30% over EAGLE-3
SpecBranch (2026)	draft model	branches	async	1.8–4.5× vs AR
P-EAGLE (2026)	parallel 4-layer EAGLE head	chain	sync	1.10–1.36× over AR EAGLE-3 (vLLM)
DFlash (2026)	block-diffusion drafter	chain/tree	sync	>6×; up to 2.5× over EAGLE-3
SpecVocab (2026)	EAGLE-3 + dynamic vocab	tree	sync	+8.1% throughput over EAGLE-3
ConFu (2026)	EAGLE-3 + future tokens	tree	sync	+8–11% (Llama-3), ∼20% (Qwen3-4B)
SSD/Saguaro (2026)	draft model on separate device + cache	chain per outcome	async speculative cache	∼30% avg over strongest SD; up to 5× AR
Gemma-4 MTP (2026)	MTP drafter sharing KV cache	chain	sync	"up to 3×" (vendor, [V])

5. **Reality checks**: the MLSys 2026 "Performance or Illusion?" study and scaling-law analyses (SDSL).
6. **Speculation for training**: RL rollout acceleration is mentioned as an application in the HKUST survey summary and in the ATLAS post[S][V].

7.7 A taxonomy

Table 7.2 organises the methods discussed in this report along four axes: where the draft comes from, what shape it has, how drafting and verification are scheduled, and whether verification is exact. The HKUST(GZ) survey summary [58] uses a related classification—four drafting paradigms (standalone draft models, learned draft heads, self-speculative methods, training-free heuristic proposals), algorithmic improvements (adaptive length, tree verification, overlapping drafting and verification), and systems concerns—which we have followed where possible.

Chapter 8

The EAGLE Family and EAGLE-3

EAGLE-3 is, as of October 2026, the learned drafter most widely deployed in open-source serving stacks (Chapter 12). This chapter explains the design lineage EAGLE → EAGLE-2 → EAGLE-3, the training-time test, the arithmetic of why the drafter's output layer matters, and the published results. All descriptions of the methods are paraphrases of the cited papers; derivations specific to this report are marked [D].

8.1 EAGLE: autoregression at the feature level

Medusa-style heads predict future tokens from the target's final hidden state in one shot, which is hard because the final hidden state encodes the *next* token, not the one after. EAGLE [21] instead trains a lightweight drafter that performs autoregression over the target's *features*: it takes the target model's top-layer features (the vector just before the language-model head) for the tokens verified so far and predicts the next feature, then applies the target's own frozen LM head to get token probabilities. A key observation of the EAGLE paper is that feature-level autoregression suffers from uncertainty because the sampled token is not visible in the feature sequence; EAGLE resolves this by also feeding the token sequence shifted by one step (the sampled tokens) into the drafter. Because the drafter shares the LM head and can inherit the embedding layer of the target, it is small: the P-EAGLE paper describes a compact single-layer architecture of 2–5% of the target's parameters[S] [15].

EAGLE-2. Using a static draft tree implicitly assumes that acceptance depends only on position. EAGLE-2 [22] observed that acceptance is also context-dependent and that the drafter's confidence scores approximate the acceptance rate with small error, so the tree can be grown adaptively: expand the nodes of highest estimated path acceptance (Proposition 6.4), then prune to a fixed number of tokens before verification. The authors report speedups of 3.05–4.26× across three model series and six tasks, 20–40% faster than EAGLE[S].

8.2 Why EAGLE stopped scaling with data

The EAGLE-3 paper begins from an empirical observation: scaling up the draft model's training data gives EAGLE limited improvement, while modern LLMs gain from more data without extra inference cost [23]. Their analysis is the following.

EAGLE's loss has two terms: a feature-prediction loss ℓ_{fea} (regress the next top-layer feature) and a token-prediction loss ℓ_{token} (cross-entropy of tokens obtained by applying the target's LM head to the predicted feature). Because the ultimate goal is token prediction, the paper argues that feature regression acts as an *extra constraint* that limits the drafter's expressiveness and prevents it from benefiting from more data. Removing the feature loss improves the acceptance rate of the *first* draft token (the paper's Figure 4), but the drafter's step-1 output $\hat{a}_{t+1}$ is then far from the true feature f_{t+1}, so the step-2 input sequence $(f_1, \ldots, f_t, \hat{a}_{t+1})$ is out of distribution

for a drafter trained only on true features, and the acceptance rate of the *second* draft token collapses.

8.3 EAGLE-3: direct token prediction and the training-time test

8.3.1 Two changes

EAGLE-3 makes two coupled changes[S]:

1. **Drop feature prediction; predict tokens directly.** The drafter is trained only with a token-level loss. Its inputs are no longer constrained to be (approximations of) top-layer features.
2. **Fuse features from several layers.** Instead of the top layer alone, take low-, middle- and high-level features l, m, h from the target (each of dimension k, the target hidden size), concatenate them into a $3k$-dimensional vector, and project with a fully connected layer to a k-dimensional fused feature g.

The paper's argument for (2) is that for a full-rank LM head the top-layer feature corresponding to a position is determined by (and aligned with) the logits of the *next* token, so predicting the next-next token from it alone is hard; with feature regression removed, nothing forces the drafter to use only the top layer, and richer intermediate representations are available.

8.3.2 Inference pipeline

Consider prefix "How can" with the target's next token "I". The target pass produces the fused features $g_{\text{how}}, g_{\text{can}}$. *Step 1:* the drafter receives the embedding e_{I} of the sampled token concatenated with g_{can} (and attends to earlier positions), passes it through an FC layer to reduce to dimension k, then through a single decoder layer to produce a_{I}, and applies the LM head to sample the draft token ("do"). *Step 2:* the target has not seen "I" yet, so g_{I} does not exist; the drafter substitutes its own output a_{I} and concatenates it with e_{do}. *Step 3* similarly uses a_{do} with e_{it}, and so on. Draft trees are built as in EAGLE-2 (the dynamic, context-aware tree) and verified with tree attention. The drafter's core is one transformer decoder layer; only self-attention touches context, so the only change needed during training/inference is in the attention masks.

8.3.3 Training-time test

Since at inference the drafter's input alternates between true fused features g and its own previous outputs a, the drafter must be trained on both. The *training-time test* (TTT) simulates the inference procedure during training: after a normal step on a training sequence (the usual causal lower-triangular mask), the drafter's outputs a are fed back as inputs for a second and third simulated step, with attention masks adjusted so that, for the simulated steps, almost all attention is diagonal (a token attends to itself in the previous step's output and to true-feature keys of earlier positions). The paper notes that computing these diagonal terms by full matrix multiplication would waste computation, and uses vector dot products at the relevant positions[S]. The loss at each simulated step is a token-level cross-entropy against the target distribution (as SpecVocab describes it, "a cross-entropy loss across additional timesteps simulated during training"[S] [56]).

In our characterisation TTT is closely related to scheduled sampling, and to the earlier HASS idea of simulating multi-step drafting during training [23]; the EAGLE-3 authors stress the difference in motivation: HASS retains feature prediction and a top-layer feature input, whereas EAGLE-3 removes the constraint altogether, which they say is what enables the data-scaling behaviour.

A heuristic model of the benefit ([D]). Suppose the acceptance of the j-th draft token, conditional on the previous ones being accepted, is $\alpha_j = \alpha_1 \rho^{j-1}$, where $\rho \leq 1$ reflects train/inference mismatch of the drafter's inputs. Then by Proposition 4.1, $\mathbb{E}[N] = \sum_{k=0}^{\gamma} \alpha_1^k \rho^{k(k-1)/2}$; the quadratic exponent shows that even mild per-step degradation ($\rho < 1$) caps the useful lookahead sharply, as Figure 4.3 illustrates in a related parametrisation. EAGLE-3's reported finding that acceptance stays nearly flat as the number of self-generated inputs grows corresponds to $\rho \approx 1$.

8.4 Where drafting time goes: the output layer

The SpecVocab and FR-Spec papers point out that for recent large-vocabulary models the cost of the drafter's *output layer* dominates drafting time. A back-of-envelope calculation shows why.

Example 8.1 (Drafter parameter budget for a Llama-3.1-8B target ([D]))**.** Llama-3.1-8B has hidden size $d = 4096$, MLP width 14,336, eight key–value heads of dimension 128 and a vocabulary of $V = 128{,}256$ (standard published configuration, not retrieved here). One decoder layer has attention parameters $2d^2 + 2 \cdot d \cdot 1024 = 41.9$M and MLP parameters $3 \cdot 4096 \cdot 14336 = 176.2$M, a total of 218M. The full-vocabulary LM head has $dV = 525$M parameters, *2.4 times* the decoder layer. A draft pass in BF16 therefore streams about 0.44 GB for the layer but 1.05 GB for the head. Restricting the head to a 32K vocabulary (as in EAGLE-3 and FR-Spec) cuts the head to 131M parameters (0.26 GB), reducing weight traffic per draft pass from about 1.5 GB to 0.7 GB, *before* adding the FC layers that fuse features. A drafter pass is memory-bound at small batch just like the target, so its time scales with these bytes; this is the fraction of c that vocabulary tricks can attack.

The EAGLE-3 draft model is trained with a reduced vocabulary built from synthetic target data; SpecVocab reports that FR-Spec found a 32K vocabulary to give the best throughput, which EAGLE-3 also uses, and argues that a fixed subset hurts when the next token falls outside it (Section 10.1).

8.5 Published results

8.5.1 Headline speedups and ablation

EAGLE-3 was evaluated on Vicuna-13B, LLaMA-Instruct 3.1 8B, LLaMA-Instruct 3.3 70B and DeepSeek-R1-Distill-LLaMA 8B across MT-Bench, HumanEval, GSM8K, Alpaca and CNN/DailyMail, using strict speculative-sampling acceptance (so output quality is unchanged and not evaluated)[S]. The abstract reports a speedup of up to 6.5× and about 1.4× over EAGLE-2. On Vicuna-13B at temperature 0, the mean (over the five tasks) speedup and acceptance length are: standard speculative sampling 1.92× (acceptance length 2.24); prompt-lookup decoding 1.74× (1.80); Medusa 2.12× (2.51); Lookahead 1.62× (1.67); Hydra 2.80× (3.50); EAGLE 3.05× (3.96);

Table 8.1: EAGLE-3 paper, Table 1, Vicuna-13B, temperature 0, mean over MT-Bench, HumanEval, GSM8K, Alpaca and CNN/DailyMail ([S]; extracted from the paper's table, where the "Mean" columns were unambiguous). τ is the mean acceptance length.

Method	Mean speedup	Mean τ
Standard speculative sampling (Vicuna-68M draft)	1.92×	2.24
Prompt lookup (PLD)	1.74×	1.80
Medusa	2.12×	2.51
Lookahead	1.62×	1.67
Hydra	2.80×	3.50
EAGLE	3.05×	3.96
EAGLE-2	4.22×	4.83
EAGLE-3	**5.51×**	**6.62**

EAGLE-2 4.22× (4.83); EAGLE-3 5.51× (6.62)[S](Table 8.1). Their code-generation result of up to 6.47× with acceptance length 7.54 (Vicuna-13B on HumanEval) is the 6.5× headline[S].

The ablation (LLaMA-Instruct 3.1 8B, MT-Bench) separates the two changes: EAGLE-2 gives 3.16× (τ=4.05); removing the feature constraint gives 3.82× (τ=5.37); adding fused features gives 4.40× (τ=6.13). On GSM8K the corresponding sequence is 3.39×, 3.77×, 4.48× (τ 4.24, 5.22, 6.23)[S]. On MT-Bench, removing the feature constraint accounts for about half to two-thirds of the total improvement over EAGLE-2, with feature fusion supplying the rest: $(3.82-3.16)/(4.40-3.16) = 0.53$ of the speedup gain and $(5.37-4.05)/(6.13-4.05) = 0.63$ of the acceptance-length gain[D].

8.5.2 Scaling with data

The paper reports that with the new architecture the speedup of EAGLE-3 increases with the amount of draft training data (up to 8× the ShareGPT-sized dataset they start from), a trend not observed for EAGLE[S]. EAGLE-3 draft models were trained on about eight times more data than EAGLE[S]. Independent infrastructure work consistent with this: the SpecForge team's SpecBundle used 1.4M samples (a data mix with many coding and math examples) against the roughly 320K samples of the original EAGLE work, regenerating responses with the target so that training data better matches the target's outputs[V] [29]. P-EAGLE's training-cost comparison (Table 9.2) gives a sense of the compute: one epoch of EAGLE-3-style training with TTT on 200K UltraChat examples at sequence length 2048 takes 2.5 hours on 8×H200 (the table is labelled $K = 8$ and notes that TTT requires multiple forward passes per example)[S].

8.5.3 Throughput at larger batch sizes

Tables 5.4 and Figure 5.2 (Chapter 5) reproduce the SGLang and vLLM batch-size results of the EAGLE-3 paper. In SGLang (v0.4.4, one H100, Llama-3.1-8B, chain length 3, MT-Bench) EAGLE-3 gives 1.81× at batch size 2 and still 1.38× at batch size 64, whereas the older EAGLE is at 0.88–0.99× for batch sizes of 24 and above[S]. The authors credit the better acceptance of EAGLE-3 for sustaining a throughput gain longer than EAGLE; both approach 1× as the system becomes compute-bound. In the same paper the SGLang team's batch-size-one throughput on H100 for the 8B model is 158.34 tokens/s without speculation, 244.10 with EAGLE-2 and 373.25 with EAGLE-3[S](2.36× for EAGLE-3[D]).

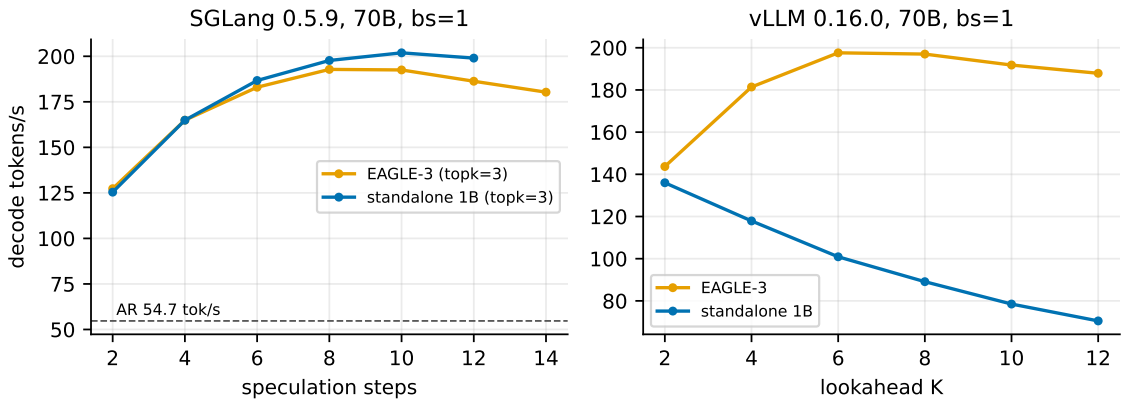


Figure 8.1: Baseline sweeps for Llama-3.1-70B at batch size one on 4×H100 ([S]; Tables 1–2 of the Saguaro paper, appendix B.4). Left: SGLang 0.5.9, top-k=3 branch of the grid. Right: vLLM 0.16.0 with chain lookahead K. The dashed line is the autoregressive baseline.

8.5.4 Compared with a standalone draft model (a counter-example)

The Saguaro paper reports an informative comparison in SGLang 0.5.9 and vLLM 0.16.0 for Llama-3.1-70B on 4×H100 at batch size one: a standalone Llama-3.2-1B drafter *outperformed* an EAGLE-3 head in SGLang[S]. Best-configuration sweeps (Figure 8.1) peak at 192.8 tokens/s for EAGLE-3 (8 steps, top-k 3) against 201.9 for the standalone drafter (10 steps, top-k 3), versus an autoregressive baseline of 54.7 tokens/s; in vLLM, EAGLE-3 reaches 197.6 tokens/s at $K = 6$ while the standalone draft model peaks at 136.0 at $K = 2$ and falls to 70.5 at $K = 12$[S] [18]. The MLSys study likewise found that, with a 70B target, draft-model SD beat EAGLE in most settings because a pretrained 1B model has higher acceptance than an EAGLE head fine-tuned on limited data, while the picture reverses for an 8B target where drafting overhead is a larger fraction of the step[S] [27]. EAGLE-3 is therefore best understood as the strongest *lightweight* drafter for settings where a well-matched small model from the same family is unavailable or too slow, rather than as uniformly superior.

8.6 Summary and critique

- **What EAGLE-3 got right.** It diagnosed a concrete training pathology (an unnecessary regression constraint, train/inference mismatch) and fixed it with a simple procedure; its gains are large in acceptance length (e.g. 4.05 → 6.13 on MT-Bench for the 8B target) and monotone in data.
- **What it leaves open.** Drafting remains K sequential passes; the LM head dominates the pass cost for large vocabularies; training data and its match to deployment traffic matter greatly; and the large batch-size-one numbers use dynamic trees that most production engines do not run.
- **How later work builds on it.** P-EAGLE removes the sequential passes (Chapter 9); SpecVocab shrinks the LM head dynamically; ConFu adds future-oriented signals (Chapter 10); SSD hides the remaining drafting latency behind verification and can use an EAGLE-3 drafter with a modified conditioning (Chapter 11).

Chapter 9

Parallel Drafting: Medusa, PARD, P-EAGLE and Diffusion Drafters

9.1 The cost of sequential drafting

Even the cheapest learned drafter must be evaluated γ times in sequence to produce γ draft tokens, since token $i+1$ is conditioned on token i. In the cost model of Chapter 4 the round time is $T(1+\gamma c)$, so drafting cost grows linearly with lookahead; this is why the optimal lookahead for EAGLE-style drafters in engines is small (the P-EAGLE paper reports that autoregressive EAGLE-3 achieves its best throughput at $K = 3$[S] [15]) even though acceptance would support longer drafts. *Parallel drafting* replaces the γ passes by one: the drafter is run once on a sequence containing γ "mask" positions and predicts all γ future tokens simultaneously. The round time becomes $T(1+\kappa c_1)$ where c_1 is the cost of a single-layer drafter pass and $\kappa \geq 1$ reflects the larger (multi-layer, wider-input) parallel drafter, independent of γ.

9.2 What a parallel drafter can and cannot predict

A parallel drafter cannot see the tokens at the intermediate positions. Formally, to propose position $t+d$ ($d \geq 2$) it must output a distribution q_d that depends only on the context $x_{\leq t}$, whereas the verifier compares the draft token with the *conditional* $p_d = p(\cdot \mid x_{\leq t}, x_{t+1:t+d-1})$, in which the intermediate tokens are the (accepted) draft tokens. Let $X_{t+1:t+d-1}$ be distributed according to the target p (the accepted prefix is distributed as the target prefix by exactness).

Proposition 9.1 (Limits of parallel drafting)**.** *Fix $x_{\leq t}$ and $d \geq 2$. Among all drafters that output q_d depending only on $x_{\leq t}$:*

(a) the expected KL divergence $\mathbb{E}\,\mathrm{KL}(p_d \,\|\, q_d)$ over the accepted prefix is minimised by the marginal *$q_d^\star(\cdot) = \mathbb{E}[p_d] = p(X_{t+d} = \cdot \mid x_{\leq t})$, and equals the conditional mutual information $I_d = I(X_{t+d}; X_{t+1:t+d-1} \mid x_{\leq t})$;*

(b) the expected acceptance of the marginal drafter satisfies $\mathbb{E}[\alpha_d] = 1 - \mathbb{E}\,\mathrm{TV}(p_d, q_d^\star) \geq 1 - \sqrt{I_d/2}$.

Proof. (a) For any q, $\mathbb{E}\,\mathrm{KL}(p_d \| q) = \mathbb{E}\,\mathrm{KL}(p_d \| q^\star) + \mathrm{KL}(q^\star \| q)$ (the standard "compensation identity": $\mathbb{E}[\sum p_d \log p_d] - \sum q^\star \log q$ with $q^\star = \mathbb{E} p_d$). The last term is non-negative with equality iff $q = q^\star$. The first term equals $\mathbb{E}\,\mathrm{KL}(p_d \| \mathbb{E} p_d) = I(X_{t+d}; X_{t+1:t+d-1} \mid x_{\leq t})$, the mutual information. (b) By Pinsker, $\mathrm{TV}(p_d, q^\star) \leq \sqrt{\frac{1}{2}\mathrm{KL}(p_d \| q^\star)}$, and by Jensen (concavity of the square root) $\mathbb{E}\,\mathrm{TV} \leq \sqrt{\frac{1}{2} I_d}$. □

The proposition says that the penalty for not seeing the intermediate tokens is governed by how much information those tokens carry about the target token d steps ahead. For highly predictable text (boilerplate code, structured output), I_d is small and parallel drafting costs little;

for text with strong local dependence (which word follows which), I_d is large and acceptance at depth ≥ 2 is capped. This is consistent with what the literature observes: P-EAGLE needs extra layers and training depth to "compensate" for the missing information (Section 9.5.2), DFlash injects target features into every draft layer, and the MTP head of GLM-4.5-Air, trained for first-token accuracy and reused for later positions, shows position-wise acceptance of 0.92, 0.68, 0.38[S] [27].

9.3 Medusa heads and ParallelSpec

Medusa [5] is the earliest widely used parallel drafter: head k is a small feed-forward network on the target's last hidden state predicting the token k positions ahead; the candidates (top few from each head) form a tree. Medusa-1 freezes the backbone; Medusa-2 co-trains the backbone with the heads. The heads' independence is exactly the marginal predictor of Proposition 9.1, and Hydra [2] added sequential dependencies between heads. ParallelSpec [15] (as described in the P-EAGLE paper, which could not obtain code and re-implemented it) proposed parallel drafting with a single transformer layer, but omits details about the use of target hidden states and does not address memory scaling in long-sequence training[S].

9.4 PARD: target-independent parallel drafts

PARD [1] adapts an *autoregressive* small model into a parallel one with a *conditional drop-token* (COD) training mechanism that retains progressively fewer positions at deeper prediction depths (geometric decay by a retention rate), improving draft-training efficiency by 3× compared with traditional masked-prediction training according to the authors[S]. A distinguishing feature is *target-independence*: one PARD draft model serves an entire family of targets without per-target training. In vLLM, the authors report up to 3.67× speedup on LLaMA3.1-8B, reaching 264.88 tokens/s, 1.15× faster than EAGLE-3[S]. The vLLM documentation lists a "Parallel Draft Model (PARD)" method among its model-based options[S] [52].

9.5 P-EAGLE

P-EAGLE [15] (Amazon, with UC Santa Cruz; NeurIPS 2026 poster[S] [36]) transforms EAGLE from autoregressive to parallel multi-token prediction.

9.5.1 Architecture

The drafter follows the LLaMA-3 architecture with rotary position embeddings. The target model processes the prompt and exposes hidden states from three layers (the paper uses layer indices 2, $L/2$ and $L-1$), concatenated to $3d$ dimensions and projected to d. Position 1 of the drafter is the *next-token-prediction* (NTP) position and operates like standard EAGLE with real hidden states. Positions $2, \ldots, K$ are *multi-token-prediction* (MTP) positions: they lack both the previous token and the previous hidden vector, so the model uses two learned parameters: a *mask token embedding* in place of the unknown token and a single *shared hidden state* h_{shared} in place of the missing hidden vector. The embedding and hidden state are concatenated per

Table 9.1: P-EAGLE hidden-state ablation ([S]; Table 3 of Hui et al. [15]): GPT-OSS-20B target, 4-layer drafter, 20 epochs on OpenCodeInstruct, HumanEval, speculation length 5.

Variant	Acceptance length	Δ
Learnable shared hidden state (baseline)	3.16	–
+ depth-specific encoding	2.85	−9.8%
+ NTP hidden state + depth encoding	2.68	−15.2%
+ NTP hidden state only	2.81	−11.1%
+ regularised NTP hidden state	2.94	−7.0%

position, passed through N transformer layers and the target's frozen LM head. All K tokens are produced in one forward pass.

A notable design finding is that position-specific information is unnecessary: the authors give a theoretical argument that attention with rotary embeddings alone can recover absolute position (their Theorem B.3), so depth-specific encodings are redundant, and an ablation with GPT-OSS-20B and a 4-layer drafter (Table 9.1) shows that all four position-aware variants *underperform* the simple shared hidden state by 7–15%.

9.5.2 Training recipe and capacity compensation

The ablations (LLaMA-3.1-8B target unless noted; HumanEval / MT-Bench acceptance length at speculation depth 5) support the picture of Proposition 9.1: a parallel drafter must compensate for missing information with capacity and training.

- **Depth.** Going from one to two decoder layers raises acceptance length by 33% on HumanEval and 14.5% on MT-Bench; four layers give a further 9.5% on HumanEval (45.7% and 26.1% relative to one layer). In contrast, one layer is optimal for autoregressive EAGLE, where each position receives the previous token and hidden state[S].
- **Unfreezing embeddings.** The mask token needs a learned embedding; unfreezing the inherited embedding table gives a consistent +5% on both benchmarks.
- **Training depth beyond inference depth.** Training with $K_{\text{train}} = 8$ while inferring at $K_{\text{infer}} = 5$ gives +4.1% (HumanEval) and +2.7% (MT-Bench), because supervision at deeper positions regularises the shallower ones.
- **Longer training and sequences.** Acceptance length improves from 3.92 to 4.00 (HumanEval) and 3.04 to 3.18 (MT-Bench) from 20 to 60 epochs, and a gain of about 2% comes from raising maximum training length from 512 to 2048 on a non-reasoning target[S].

9.5.3 Scalable training for long reasoning traces

Parallel-prediction training multiplies the number of positions by the prediction depth K, so attention memory grows with $(nK)^2$. P-EAGLE's training framework has two parts.

Amortised mask construction. The causal structure across prediction depths is position-invariant: the attention pattern for the first n positions is exactly the top-left $(nK) \times (nK)$ sub-matrix of the pattern for any longer sequence. Hence one precomputed maximum-length mask can be sliced for each example in constant time, rather than building an $O((nK)^2)$ mask per example as PARD's COD sampling requires. Table 9.2 gives the reported overhead.

Table 9.2: Training overhead at sequence length 2048, $K = 8$ ([S]; Table 2 of Hui et al. [15]). Data loading measured on 128 examples; epoch time on UltraChat (200K examples), 8×H200.

Method	Load 128 examples	Epoch
EAGLE-3 (with TTT)	14.8 s	2.5 h
PARD (per-example masks)	718.5 s (48×)	12+ h (5×)
P-EAGLE (precomputed masks)	17.5 s	1.8 h

Table 9.3: Acceptance length at speculation depth $K = 5$ ([S]; Table 9 of Hui et al. [15]); values as printed (one decimal). The maximum possible with the bonus token is 6.0.

Model	Dataset	AR EAGLE-3	P-EAGLE (4L)
GPT-OSS 120B	HumanEval / MT-Bench / GSM8K / mean	3.5 / 2.7 / 3.3 / 3.1	3.5 / 2.9 / 3.5 / 3.3
GPT-OSS 20B	HumanEval / MT-Bench / GSM8K / mean	3.7 / 3.4 / 3.9 / 3.7	3.8 / 3.4 / 4.0 / 3.7
Qwen3-Coder 30B	HumanEval / MT-Bench / GSM8K / mean	4.4 / 3.0 / 3.1 / 3.5	4.5 / 3.0 / 3.2 / 3.6

Sequence partitioning. To train on a single sequence longer than memory allows, P-EAGLE splits a sequence into S segments and accumulates gradients across segments ("within-sequence gradient accumulation"), cutting peak attention memory from $O(L^2)$ to $O(L^2/S^2)$ where L is the number of positions. With COD's random position sampling at each depth, a position at depth d depends on position $p-1$ at depth $d-1$ which could fall in a different segment, so the paper assigns segments iteratively across depths: depths 0 and 1 by index, depths ≥ 2 inherit the segment of their dependency. As an illustration, an 8,192-token sequence with $K = 8$ depths and COD retention $r = 0.8$ has $n(1-r^K)/(1-r) \approx 34{,}000$ positions in total ([D]from the paper's formula).

9.5.4 Acceptance length and throughput results

The evaluation (GPT-OSS-120B, GPT-OSS-20B, Qwen3-Coder-30B; trained on UltraChat, GSM8K train and OpenCodeInstruct; maximum training length 8192; 4-layer drafter; evaluated on HumanEval, MT-Bench, GSM8K with speculation depth $K = 5$) first establishes that parallel drafting matches autoregressive EAGLE-3's acceptance length, using an AR baseline strengthened with the Harmonized Context Alignment (HCA) loss. Table 9.3 reports acceptance lengths; P-EAGLE matches or exceeds the baseline in all nine model–dataset pairs, with average gains of +4.5% (120B), +2.5% (20B) and +2.0% (Qwen3-Coder 30B)[S]. The paper stresses that the goal is not higher acceptance but throughput.

vLLM throughput in the paper. On one H200, with chain drafting at concurrency $C = 2$ and 4, the paper reports output tokens per second for $K \in \{3, 5, 7\}$. Autoregressive EAGLE-3 is best at $K = 3$ and degrades as K grows (for the 20B model at $C = 2$ on HumanEval: 929, 912, 833 tokens/s for $K = 3, 5, 7$), whereas P-EAGLE improves with K (1001, 1173, 1205 tokens/s)[S]. Best speedups over the best autoregressive configuration are 1.27–1.36× for the 20B model, 1.04–1.10× for 120B and 1.04–1.17× for Qwen3-Coder-30B at $C = 2$; at $C = 4$ the 20B gains stay 1.24–1.27× while the larger MoE models moderate to 1.03–1.11×, which the authors attribute to expert-routing overhead scaling with batch size[S]. For Qwen-30B on HumanEval at $K = 3$ P-EAGLE is slightly *slower* (0.98× at $C = 2$, 0.92× at $C = 4$) because its four-layer pass can cost more than three one-layer passes at low depth[S]. (The extraction of the full throughput table from the PDF text interleaves columns; the model-block assignment was inferred from the

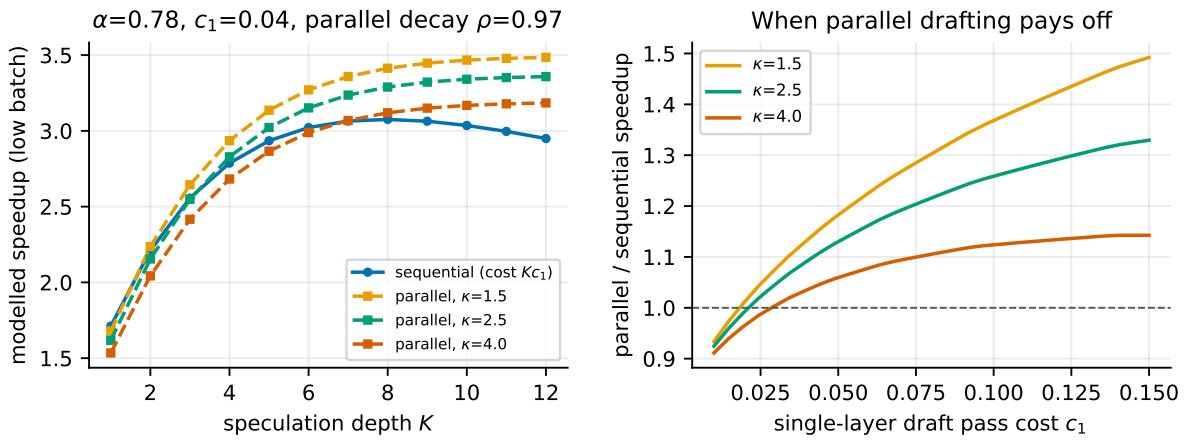


Figure 9.1: Illustrative cost model contrasting sequential and parallel drafting ([M]). Left: modelled speedup against depth K for $\alpha = 0.78$ (an effective rate implied by P-EAGLE's reported acceptance length, Table 4.6) and single-layer pass cost $c_1 = 0.04$; the sequential drafter costs Kc_1, the parallel drafter costs κc_1 with mild per-depth decay $\rho = 0.97$. Right: ratio of best parallel to best sequential speedup as a function of c_1.

speedup ranges quoted in the paper's text.)

The vLLM project blog. The vLLM blog (March 2026) reports P-EAGLE on a single B200 with GPT-OSS-20B across MT-Bench, HumanEval and SPEED-Bench at concurrencies 1–64: 1.05–1.69× over vanilla EAGLE-3, with 55–69% higher throughput at $c = 1$ and 5–25% at $c = 64$; P-EAGLE peaks at $K = 7$ at every concurrency while EAGLE-3 peaks at $K = 3$; at $K = 7$ the acceptance lengths are 3.94 (HumanEval), 3.38 (SPEED-Bench) and 3.70 (MT-Bench) for P-EAGLE versus 3.03, 2.59 and 3.27 for EAGLE-3[V] [53]. It states that support is in vLLM from v0.16.0 (a setting `parallel_drafting`), that pre-trained drafters are available for GPT-OSS-120B, GPT-OSS-20B and Qwen3-Coder-30B-A3B-Instruct, that a P-EAGLE drafter must be specially trained (standard EAGLE heads will not work), and that all numbers come from one B200 and three benchmarks[V]. Cross-checking the two sources shows internally consistent trends (larger gains at low concurrency, EAGLE-3 optimal at $K = 3$, P-EAGLE at larger K) but different hardware and settings, so the 1.69× and the paper's 1.36× should not be compared directly.

9.5.5 An illustrative cost model

To make the trade-off concrete, Figure 9.1 compares the best achievable speedup of a sequential single-layer drafter with that of a parallel drafter whose single pass costs κ times a one-layer pass, taking an effective $\alpha = 0.78$ and a slight decay of acceptance with depth. With $c_1 = 0.04$ the sequential drafter's best speedup is 3.08× at $K = 8$, while parallel drafters with $\kappa = 1.5, 2.5, 4$ reach 3.48×, 3.36× and 3.18× (at the grid limit $K = 12$) ([M]). The parallel drafter wins when κc_1 is less than about $K^{\star}_{\text{seq}} c_1$, and the advantage grows with c_1 (a more expensive base drafter, or a larger LM head). This is a model, not a measurement: the actual κ and acceptance decay of P-EAGLE are not given in these terms by its authors, and the published speedups (1.1–1.4× in the paper) are smaller than the model suggests, in part because the reference EAGLE-3 baselines are already tuned.

9.6 DFlash: block-diffusion drafting

DFlash [7] replaces the autoregressive drafter with a lightweight *block diffusion* model that decodes all masked positions of a block in a single forward pass, conditioned on target features. As summarised from the paper: the default drafter has 5 transformer layers (8 for Qwen3-Coder) and block size 16 (10 for LLaMA-3.1); hidden states from five target layers spread between the second and third-to-last are concatenated and projected into a compact context feature; unlike EAGLE-3, which feeds target features as inputs, DFlash *injects* the fused feature into the key and value projections of *every* draft layer, which the authors argue prevents dilution of the target signal with depth; the draft shares the target's frozen embedding and LM head. Training uses about 800K samples from Nemotron Post-Training Dataset V2 and CodeAlpaca with responses generated by the target, randomly sampled anchor blocks, and exponentially decaying loss weights on later positions (early errors invalidate everything after them)[S].

Reported results: over 6× lossless acceleration and up to 2.5× higher speedup than EAGLE-3 (abstract); for Qwen3 instruct models with thinking off, greedy, in Transformers: about 4.9× average speedup versus about 2.4× for EAGLE-3 with tree size 16; at temperature 1 about 4.1× versus 2.2×; for reasoning mode, about 4.5× and 3.9×; per-task speedups on Qwen3-8B range from roughly 2.8× on LiveCodeBench to about 6× on MATH-500[S]. In SGLang on a single B200, speedups are largest at low concurrency (about 5.1× for Qwen3-8B on Math500 at concurrency 1) and fall to roughly 2.4–3.1× at concurrency 32[S]. The paper reports that a five-layer drafter *without* target features reaches only about 2–3×, supporting the importance of conditioning, and that a drafter trained on 4K contexts degrades beyond 4K but recovers with brief long-context fine-tuning (acceptance length from 3.61 to 6.05 at 16K on hotpotqa)[S].

Reading the DFlash numbers

Baselines are limited to EAGLE-3 (other diffusion drafters were not compared because code was unavailable), most evaluations use the Qwen3 family and LLaMA-3.1-8B, and several headline comparisons are in Transformers, not an optimised serving engine; the reported SGLang numbers show the advantage shrinking with concurrency. The paper's speedups over autoregressive decoding (4–6×) are far larger than those in the P-EAGLE or Saguaro papers for comparable targets, which probably reflects different baselines (AR in Transformers is slower than AR in a CUDA-graph serving engine) and block sizes of 10–16 with high acceptance on math and code; independent replication in a production stack is not yet available in the sources retrieved.

9.7 Summary

- Parallel drafting reduces drafting cost from $O(\gamma)$ passes to $O(1)$, enabling longer lookahead; its price is a mutual-information cap on accuracy at depth (Proposition 9.1) that must be paid for with capacity, conditioning on target features, and training at greater depth.
- The reported gains of P-EAGLE over a tuned AR EAGLE-3 in production-style serving are modest to substantial (1.04–1.36× in the paper; up to 1.69× in a vendor blog at low concurrency) and shrink with concurrency and for large MoE targets.
- Diffusion-style drafters report the largest numbers (DFlash) but with weaker baselines

and limited cross-family evidence.

Chapter 10

Other Drafter Improvements: Vocabulary, Future Signals, MTP, Training-Free and Adaptive Drafters

Chapters 8 and 9 described the two dominant learned-drafter designs of 2026. This chapter surveys the remaining levers that move the two quantities in the speedup formula $S = E[N]/(\gamma c + 1)$ of Chapter 4: the drafter's cost c (vocabulary pruning, weight sharing, lookup-based drafting), its quality α (future-aware conditioning, MTP heads, on-policy and online training) and the choice of γ itself (adaptive length). Wherever possible, claims are tied to a retrieved source; the two small experiments of this chapter (Section 10.1 and Section 10.4) are toy or proxy measurements and are labelled [M].

10.1 The output-layer bottleneck and speculative vocabularies

10.1.1 Why the LM head matters

A drafter that shares the target's tokenizer must produce logits over the full vocabulary $|V|$ using an output matrix $U \in \mathbb{R}^{|V|\times d}$, which costs $d|V|$ multiply-adds and the same number of parameters read per draft token. Chapter 8 computed this for Llama-3.1-8B ($d = 4096$, $|V| = 128{,}256$): $d|V| \approx 525$M parameters, more than twice the roughly 218M parameters of one transformer decoder layer of the same model[D]. For a one-layer EAGLE-style drafter, the output layer therefore dominates the drafter's weight traffic. FR-Spec [59] identified this "output distribution bottleneck" and proposed pruning the draft output matrix to a frequency-ranked subset; VocabTrim (Goel et al., as described in the SpecVocab paper) does the same with token frequencies measured on target-generated data; EAGLE-3 itself trains its drafter's output layer directly on a reduced vocabulary built from target-generated data. According to the SpecVocab authors, FR-Spec found 32K to be the throughput-optimal subset size, a size that EAGLE-3 also uses[S] [56].

10.1.2 A coverage bound

Restricting the draft to a subset K is not free: tokens outside K can never be proposed, so the acceptance rate is capped by the target's mass inside K.

Proposition 10.1 (Coverage cap)**.** *Let q_K be any draft distribution supported on $K \subseteq V$ and let p be the target distribution. Then $\alpha = \sum_x \min\{p(x), q_K(x)\} \le p(K)$, with equality if and only if $q_K(x) \ge p(x)$ for every $x \in K$.*

Proof. For $x \notin K$ we have $q_K(x) = 0$, so $\min\{p(x), q_K(x)\} = 0$. Hence $\alpha = \sum_{x\in K} \min\{p(x), q_K(x)\} \le \sum_{x\in K} p(x) = p(K)$. Equality holds iff $\min\{p(x), q_K(x)\} = p(x)$ for every $x \in K$, i.e. $q_K \ge p$

Table 10.1: SpecVocab results on Spec-Bench with Qwen3-8B ([S]; Tables 1–2 of Williams et al. [56]); SGLang, one GPU; mean over five seeds. "EAGLE-3" is a third-party checkpoint, "Repro." is the authors' own EAGLE-3 draft trained with their recipe on UltraChat (464K target-generated examples); FR-Spec and VocabTrim are applied to the reproduction with a 32K vocabulary. The last column is our ratio against the reproduction ([D]).

Method	Mean accept. length	Mean tokens/s	Speedup vs AR	vs Repro.
Autoregressive	1.00	97.1	1.00×	–
EAGLE-3 (third-party)	4.33	212.7	2.19×	0.90×
EAGLE-3 (Repro.)	4.78	235.1	2.42×	1.00×
+ FR-Spec	4.69	229.8	2.37×	0.98×
+ VocabTrim	4.84	237.6	2.45×	1.01×
+ SpecVocab	5.01	245.2	2.53×	1.04×

pointwise on K. The renormalised target $p(\cdot \mid K) = p/p(K) \geq p$ on K satisfies this, so the bound is tight. □

Two consequences follow. First, the cost of a static subset is largest in contexts whose target mass is concentrated on tokens that are globally rare (names, identifiers, non-English words), because a frequency-ranked K does not contain them. Second, a context-dependent subset K_t can reach the same coverage with a much smaller $|K_t|$, reducing the indexed LM-head cost proportionally.

10.1.3 SpecVocab

SpecVocab [56] (arXiv February 2026, v2 July 2026) predicts, at each drafting step, which vocabulary subset to evaluate. Using the drafter's final hidden state h_t it computes a low-dimensional projection $h'_t = W_{\text{down}} h_t$ with $d' \ll d$, approximate logits $s_t = W_{\text{vocab}} h'_t$ over all of V, selects the top-k indices $\mathcal{K}_t$ (the paper's default is $k = 2048$ versus 32K for static methods), and then computes exact logits only for those rows of the output matrix, $z'_t = U[\mathcal{K}_t, :]h_t$, with a fused kernel so that each embedding row is read once. The ranking module is trained by distillation from the target output distribution jointly with the drafter under the same training-time-test loss as EAGLE-3, $L = L_{\text{TTT}}(p, q) + \lambda L_{\text{TTT}}(p, q_{\text{aux}})$[S]. Because $k \ll |V|$, the dominant cost of the output layer drops from $d|V|$ to $d'|V| + dk$.

Table 10.1 shows what the method buys: a 4% throughput gain over a well-trained EAGLE-3 reproduction and 2–3% over the best static pruning baseline on this model, with higher acceptance in every Spec-Bench category (the abstract reports gains of up to 8.1% in average throughput over EAGLE-3 across the four tested models; we did not retrieve the other models' tables)[S]. Two things deserve emphasis. First, the improvement over the reproduction is modest; the larger gap in Table 10.1 ("third-party" versus "reproduction") is a reminder that *training recipe differences between checkpoints are as large as the algorithmic improvements being claimed*. Second, FR-Spec reduced acceptance length (4.69 versus 4.78) yet recovered most of the loss in throughput, illustrating the α–c trade-off: pruning lowers α and lowers c together.

10.1.4 A toy coverage experiment

To visualise Proposition 10.1, we built synthetic target distributions with $|V| = 32{,}000$ and a global Zipf prior ($s = 1.05$) boosted on 600 randomly chosen tokens per topic (40 topics;

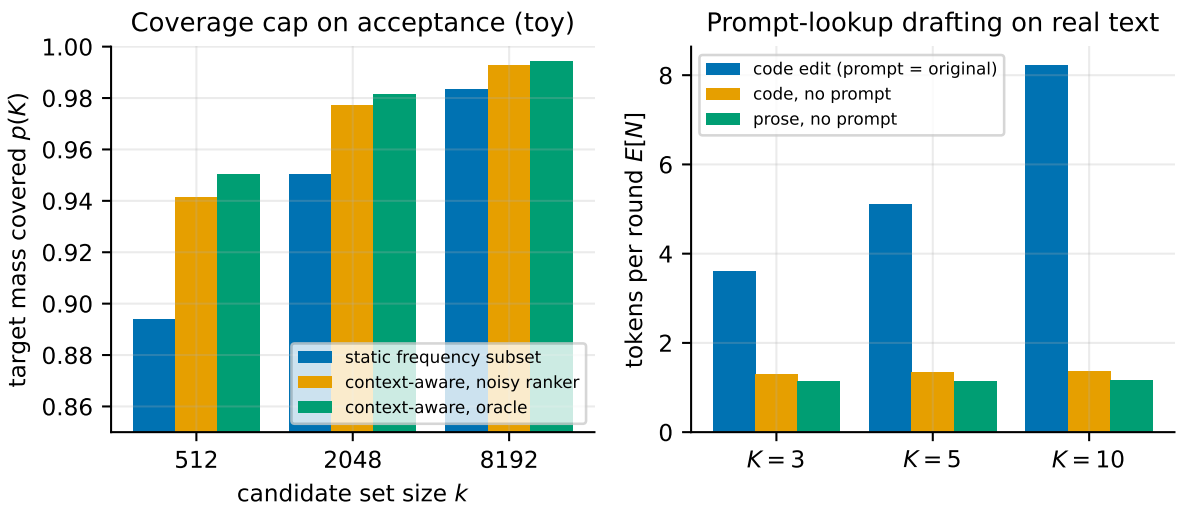


Figure 10.1: Left: coverage of target probability mass by a static versus context-aware candidate vocabulary in a synthetic setting ([M]; Proposition 10.1 caps acceptance by this coverage). Right: tokens per round for prompt-lookup drafting on real text with the text acting as a deterministic target ([M]; Section 10.4); mean over 60 standard-library modules (code) and four paper texts (prose).

400 sampled contexts with log-normal perturbation). We compare the coverage $p(K)$ of (i) a static subset formed by ranking tokens by average mass, (ii) a context-aware subset formed by ranking with Gaussian noise of standard deviation 1 on the log-probabilities (a stand-in for an imperfect learned ranker), and (iii) a per-context oracle top-k ([M]; sim/exp10_vocab.py). The left panel of Figure 10.1 shows that at $k = 2048$ the static subset covers on average 95.0% of the target mass (10th percentile 92.6%) versus 97.7% for the noisy ranker and 98.1% for the oracle; the static subset needs about $k = 8192$ (98.3%) to match the oracle at $k = 2048$. This is a synthetic stress test, not a model measurement: its qualitative message, that context awareness buys roughly a $4\times$ smaller candidate set at equal coverage, depends entirely on the assumed topic structure.

10.2 Future-aware drafting: ConFu

All EAGLE-style drafters condition on the current prefix and on target features of the *verified* prefix; as drafting proceeds, the drafter's own hidden states drift from the target's and acceptance decays with depth (Section 4.4). ConFu [41] (UCLA and Qualcomm AI Research; arXiv March 2026, v3 April 2026) attacks this drift by giving the drafter a signal about where the target's computation is heading. It appends *contemplate* (pause-like) tokens to the target's input and prepends learnable soft-prompt vectors to the target's KV cache (in the manner of BiTA), so that, in the same forward pass used for verification, the target emits an extra hidden state that summarises its "intended direction"; this hidden state is fed to the drafter as an additional input. Because the extra tokens are processed in parallel with the existing ones, the added target-side cost is described as negligible; the target model is frozen, and only the soft prompts, contemplate-token embeddings and the drafter are trained. A mixture-of-experts layer makes the contemplate embedding dynamic (context-dependent), and training uses anchor-token sampling with "future prediction replication"[S].

The reported gains over EAGLE-3 are 8–11% in accepted length and speed on Llama-3.2-3B

and Llama-3.1-8B, and about 20% on Qwen3-4B; for Qwen3-4B averaged over temperatures and draft-node budgets the paper reports +21.0% accepted length and +19.8% speed-up ratio, with a representative cell (temperature 0, 60 draft nodes) improving τ from 3.81 to 4.60 and the speed-up ratio from 2.69 to 3.20[S]. A stress test with 12-token drafts shows heavier tail acceptance than EAGLE-3. Three cautions apply: (i) the models are small (3B–8B) and the evaluation is on Spec-Bench, with a tree draft of 30 or 60 nodes; (ii) speed-up ratios are relative to the authors' own baseline implementation, not to a production engine; (iii) as with P-EAGLE and SpecVocab, no independent replication was found in the sources retrieved. The method is nevertheless conceptually interesting because it explicitly connects speculative decoding with latent-reasoning and pause tokens, and it is orthogonal to parallel drafting: a parallel drafter's missing information (Proposition 9.1) is exactly what a future-oriented signal could supply.

10.3 Multi-token prediction as a built-in drafter

Multi-token prediction (MTP) trains extra prediction modules *jointly with* the base model, rather than fitting a drafter post hoc. DeepSeek-V3's technical report describes an MTP training objective that predicts one additional token at each position, and states that the acceptance rate of the second token ranges from 85% to 90% across topics, which the authors say enables an improvement in decoding speed of about 1.8× (the sentence was truncated in our extraction; the factor and its units should be verified against the report)[S] [9]. Two further points about MTP are important for practitioners.

Position-wise acceptance decays when one head is reused. The released GLM-4.5-Air weights include only the first MTP module; the vLLM study [27] reports that it is trained mainly to predict the first drafted token well and then reused autoregressively for later positions, with position-wise acceptance of $0.92 \to 0.68 \to 0.38$ on GPQA-Main and measured speedups of 1.3–1.8×[S]. Taking the three numbers as conditional per-position acceptance rates, the expected tokens per round with three draft positions is $E[N] = 1 + 0.92 + 0.92 \cdot 0.68 + 0.92 \cdot 0.68 \cdot 0.38 \approx 2.78$; if they are instead cumulative survival probabilities, $E[N] = 1 + 0.92 + 0.68 + 0.38 = 2.98$ ([D]; the extraction does not say which reading is intended). Either way, an i.i.d. model with the first-position rate would overstate the benefit: $\alpha = 0.92$ would give $E[N] = (1 - 0.92^4)/(1 - 0.92) = 3.55$. This is exactly the heterogeneity that Chapter 4 analysed in Proposition 4.5 and Section 4.4.

Open MTP drafters for dense and MoE models. Google announced MTP "assistant" drafters for the Gemma 4 family on 5 May 2026, claiming up to 3× lower latency with unchanged outputs and describing drafters that share the target's embeddings, use its last-layer activations together with token embeddings, and reuse the target's KV cache; for the 26B-A4B mixture-of-experts model the same coverage reports about 2.2× on Apple silicon at batch sizes 4–8, while at batch size 1 verifying multi-token drafts loads additional experts and offsets much of the gain[V] [12]. We read this through a secondary summary, so all numbers are vendor claims; the MoE caveat is consistent with the expert-touch analysis of Chapter 2 and the P-EAGLE observation that MoE gains moderate with concurrency (Chapter 9).

Table 10.2: Prompt-lookup drafting on real text as a deterministic target ([M]). Mean tokens per round $E[N]$ including the bonus token, with the fraction of rounds in which any draft was proposed. Because the target is a fixed text, these are measurements of text repetitiveness, not of any LLM. The code-edit case is an extreme (a purely mechanical rename); real edit tasks change semantics.

	$K = 3$		$K = 5$		$K = 10$	
Case	$E[N]$	drafted	$E[N]$	drafted	$E[N]$	drafted
Code edit (prompt = original)	3.60	97%	5.10	96%	8.23	94%
Code from scratch (no prompt)	1.30	32%	1.33	31%	1.35	30%
Prose (no prompt)	1.14	28%	1.15	28%	1.15	27%

10.4 Training-free drafters: prompt lookup, n-gram and suffix decoding

Prompt-lookup ("n-gram") drafting proposes the tokens that followed the most recent earlier occurrence of the current suffix in the prompt or in the output so far. Its drafting cost is a hash lookup ($c \approx 0$ in the speedup formula), it needs no extra memory on the GPU, and the draft is always a chain. The vLLM study found drafting time below 2% for n-gram and reported that it is generally less effective than learned methods *except* on code editing, where n-gram beat EAGLE and EAGLE-3 for both Llama-3.1-8B and Qwen3-8B on InstructCoder; it also performed comparably to EAGLE-3 (1.50–1.58× versus 1.64–1.80×) on reasoning workloads with Qwen3-8B-Thinking, where long outputs contain repetitive symbolic patterns[S] [27]. LinkedIn reports using n-gram speculation in vLLM for its Hiring Assistant agent, whose long structured outputs repeat skill names and titles from the input, and claims about 4× higher throughput at the same QPS and latency ceiling and a 66% average reduction in P90 end-to-end latency with no measured quality loss[V] [24]; the post (read via an aggregator mirror) states that gains depend on repetitive, structured text. The vLLM documentation lists both prompt lookup and "suffix decoding" (which matches against a cache of previous requests) among its lightweight methods and notes that these give smaller speedups without adding load at peak traffic[S] [52].

10.4.1 A proxy experiment on real text

To quantify *why* n-gram drafting succeeds on code edits and fails on free prose, we simulated prompt lookup (n-gram lengths 2 to 7, longest match first, most recent occurrence, $K \in \{3, 5, 10\}$) using real text as a deterministic "target": the drafter proposes the K tokens following the latest earlier occurrence of the current suffix in the prompt plus accepted history, and a round accepts the longest prefix that matches the true next tokens, plus one bonus token ([M]; sim/exp11_ngram.py). Tokens are regex word and punctuation pieces rather than BPE tokens. We evaluated (a) *code editing*: the prompt is a Python standard-library module (60 modules of 300–1500 lines) and the target is the same module after renaming its most frequent long identifier; (b) *code from scratch*: no prompt, target the module itself; (c) *prose*: no prompt, target 6000 tokens of text from four research-paper extractions. Table 10.2 reports mean tokens per round.

The experiment reproduces the qualitative ordering of the published results: with a prompt that contains the answer, copy-style drafting has very high acceptance (3.60 of the maximum 4 tokens per round at $K = 3$), while without a source to copy from it proposes in under a third of rounds and gains only 14–35% tokens per round. A round yielding $E[N] = 1.14$ offers at most

a 14% gain even with free drafting ($c = 0$ in Chapter 4's formula), and any non-trivial cost of verifying K extra tokens can erase it; this is consistent with the vLLM finding that n-gram is generally less effective than learned drafters outside copy-heavy tasks ([D]). Treat the numbers as an illustration of the mechanism; they say nothing about the repetitiveness of any particular LLM's outputs.

10.5 Adaptive and online drafters

10.5.1 Choosing the lookahead: goodput

SmartSpec [28] defines *goodput* as the rate of tokens that the target actually generates and keeps (accepted drafts plus the bonus token) per unit time, as opposed to throughput that also counts rejected draft tokens verified. It chooses each request's speculation length, from zero (no speculation) upward, to maximise estimated goodput, estimating the accepted length from the recent acceptance rate and the batch step time from a linear model fitted to profiled hardware behaviour; lightly loaded systems speculate more and heavily loaded systems less or not at all. In the paper's illustrative experiment (LLaMA-7B target, LLaMA-160M draft, one A100-80GB, acceptance 0.7), proposing 5 tokens helps at low request rate but gives no benefit beyond about 12 requests per second, and proposing 3 tokens causes slowdowns above about 16; SmartSpec is reported to reduce average request latency by up to 3.2× relative to non-speculative decoding, in a system integrated into vLLM[S]. Our own batch-adaptive calculation in Chapter 5 (Figure 5.4) follows the same logic: the optimal γ is a decreasing function of batch size and reaches zero near the compute-bound regime[M]. The vLLM documentation lists "dynamic speculative decoding" among its scheduling variants, but TensorRT-LLM's documentation states that speculation cannot be turned off dynamically there, so speedups are seen only at low batch sizes[S] [39, 52].

10.5.2 Learning from live traffic

Together AI's ATLAS combines a static speculator trained on a broad corpus, a lightweight adaptive speculator updated from live traffic, and a confidence-aware controller that picks the speculator and the lookahead. The company reports up to about 500 tokens/s on DeepSeek-V3.1 and about 460 on Kimi-K2 in a "fully adapted" state on HGX B200 at batch size 1, a 2.65× speedup relative to standard decoding in one comparison (the "4×" headline uses a different, FP8, baseline at 105 tokens/s), and, in an RL-rollout experiment (Qwen2.5-7B-Instruct-1M, H100), acceptance rising from under 10% to over 80% over about 1,400 steps[V] [50]. All benchmarks are vendor-run on single configurations, and the company's own text says that adaptation works best on narrow, repetitive distributions; results on diverse traffic or during cold start are not reported. The academic ancestor is online speculative decoding [26] (Chapter 7). A 2026 PhD qualifying-exam announcement from HKUST(GZ) describes AdaMemSpec, which reuses recurring token fragments through a "speculation memory" and tunes the drafter's stopping policy online, and reiterates that "acceptance rate alone does not determine end-to-end speed"; this source is an abstract only, without numbers[V] [58].

10.5.3 Scaling laws for drafter size

SDSL [4] (an independent researcher and a University of Hawai'i author; arXiv March 2026) fits acceptance α as an affine function of draft and target perplexity ($R^2 \approx 0.60$, with the

draft perplexity coefficient dominating and measured α between about 0.52 and 0.79) and combines it with the throughput expression $\propto (1-\alpha^{\gamma+1})/[2(M+\gamma N)(1-\alpha)]$ (M, N target and draft parameter counts) to predict a throughput-optimal draft size $N^\star \approx \mu M + M_0$ with $\mu \approx 2.7 \times 10^{-3}$, $M_0 \approx 8.7 \times 10^7$ ($R^2 \approx 0.99$ for the pooled fit)[S]. The paper's headline that the optimal draft is roughly 200× smaller than the target is not fully consistent with its own pooled slope (which implies about 370×); we flag this and do not rely on it. The setting is also narrow (OPT-family-style draft/target pairs trained on comparable data, text-only), and its cost model treats the cost as proportional to parameter count, which Chapter 5 shows fails once the draft is not memory-bound or the output layer dominates.

10.6 Self-speculation and early exit

LayerSkip [10] trains a model with layer dropout and an early-exit loss so that its first layers can draft and its remaining layers verify, sharing weights and the KV cache of the early layers; the paper reports up to 2.16× on summarisation and 1.82× on code (Chapter 7)[S]. Self-speculation avoids the second model and its alignment problem, and one practitioner source recommends watching it as a way to avoid running a separate drafter, noting that it gives up some speed[V] [40]. MagicDec's sparse-KV self-drafting is the long-context analogue: the target drafts for itself with a compressed KV cache, which yields higher acceptance than small independent drafters and is the main device by which the paper obtains speedups at large batch sizes in the long-context regime (Chapter 5)[S] [43].

10.7 A comparison of drafter families

Table 10.3 summarises, qualitatively, how the drafter families trade off the quantities in the speedup formula. The cells are our synthesis of the sources above and of the derivations in Chapters 4–6 and are [D] judgements, not measurements.

10.8 Summary

- Vocabulary pruning exists because the output layer, not the transformer layer, dominates a one-layer drafter's cost; a static subset caps acceptance at its target mass (Proposition 10.1), and context-aware selection (SpecVocab) gains a few percent of throughput over strong baselines.
- Future-aware signals (ConFu) and MTP heads address the same decay of acceptance with depth that parallel drafting suffers from; the published gains are 8–20% over EAGLE-3 on small models in the authors' own implementations.
- Training-free drafters are excellent when the output copies the input and nearly useless otherwise; our proxy experiment quantifies the gap on real text.
- Adaptive speculation length and online adaptation are fundamental in serving because the best γ depends on load; the evidence for online-learned speculators is currently vendor-reported.

Table 10.3: Qualitative comparison of drafter families (synthesis, [D]).

Family	Cost c	Quality α	Main practical limits
Standalone small LM	γ passes of a full small model; large for small targets (37.5% of target step reported for Qwen3-0.6B vs 8B)	Good with pretraining; tokenizer must match	Memory for a second model; weak for 8B targets
EAGLE-3 / feature-level	γ one-layer passes plus LM head	Highest among standard learned drafters; decays with depth	Per-target training; LM head dominates cost
Parallel (P-EAGLE, PARD, DFlash)	One pass independent of γ	Capped by mutual information at depth (Prop. 9.1)	Training complexity; MoE gains shrink with batch
MTP heads (built in)	Minimal if shipped	Excellent first position, decays if one head reused	Only for models that ship MTP
Prompt lookup / suffix	≈ 0	High on copy-heavy tasks, near 1 otherwise (Table 10.2)	Workload-dependent; chain only
Self-speculative	No second model	Moderate	Requires special training; some quality-speed trade-off

Chapter 11

Speculative Speculative Decoding (SSD) and Saguaro

Speculative decoding removes the sequential dependence between *tokens*; it retains a sequential dependence between *drafting and verification*. In each round the verifier idles while the drafter speculates, and the drafter idles while the verifier verifies (Figure 11.1). Kumar, Dao and May [18] (Stanford, Princeton and Together AI; arXiv 2603.03251, v3 of May 2026; ICLR 2026 poster on 24 April 2026[S] [16]) ask whether this dependence can be removed too, and answer with *speculative speculative decoding* (SSD): while verification of round T is in flight, the drafter, running on separate hardware, predicts the likely *outcomes* of that verification and prepares a speculation for each in advance. If the actual outcome is among those prepared, the next speculation is available at once and drafting latency vanishes from the critical path. This chapter explains the framework, the three design problems and their solutions in Saguaro (the optimised algorithm), the theory, the published results, our own simulations that verify the theory numerically, and the limitations.

11.1 Setting and definitions

Let T_p and T_b denote the time taken by the *primary* and *backup* speculators relative to one verification (so that $T = 1$ equals one target step); let K be the speculative lookahead. A *verification outcome* of a round is the pair $v = (k, t^*)$ where $k \in \{0, \dots, K\}$ is the number of accepted draft tokens and t^* the bonus token sampled from the residual distribution $r \propto (p - q)_+$ (or from the target if all tokens were accepted)[S]. The set of possible outcomes has size about $(K+1)|V|$, far too large to precompute exhaustively. The *speculation cache* $\mathcal{S}$ is a map from a subset of outcomes to precomputed speculations; a *hit* occurs when the realised outcome is a key. Let $p_{\text{hit},p}$ ($p_{\text{hit},b}$) denote the probability of a hit given that the previous speculation came from the primary (backup) speculator, and p_{hit} the unconditional hit probability. The authors note an analogy with speculative execution and branch prediction in CPUs[S]; a hit returns the pre-speculated tokens immediately, and a miss invokes a fallback speculator, which reduces to ordinary speculative decoding and is hence lossless.

11.2 Exactness

The SSD paper states that SSD is lossless because a hit returns a speculation that is then verified exactly as in SD and a miss reduces to SD. We make the argument explicit, since the subtlety is that the *choice* of which outcomes to prepare must not leak information about the verifier's coin flips.

Proposition 11.1 (SSD is exact)**.** *Assume that (i) the speculation stored under outcome key v is a sample of the draft's autoregressive chain conditioned on the prefix implied by v, generated with randomness independent of the verifier's accept/residual randomness; (ii) the set of keys depends only on draft-side*

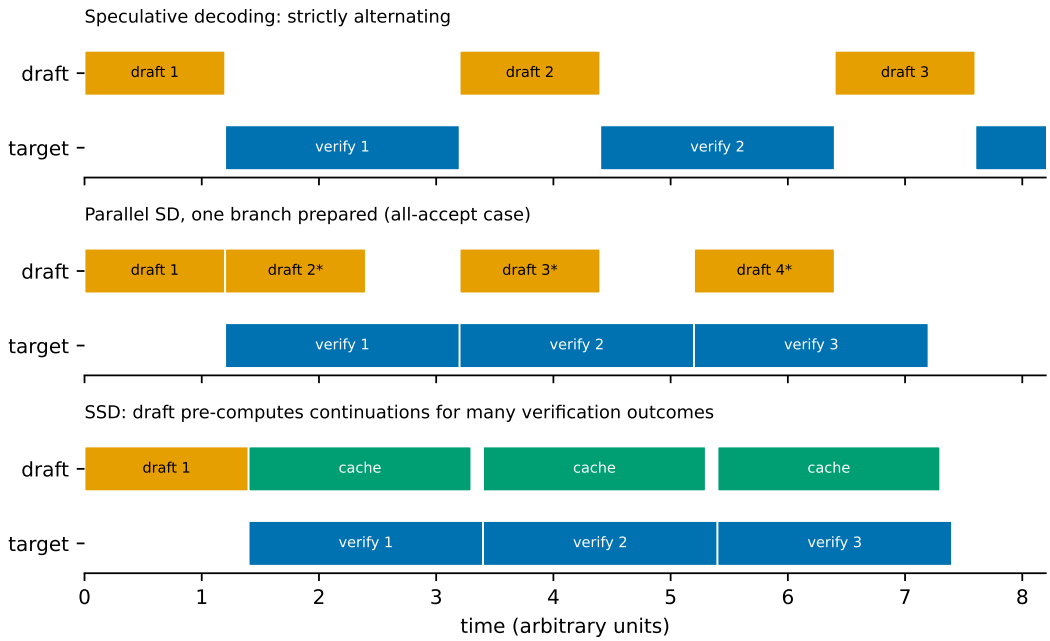


Figure 11.1: Schematic timelines ([D]; drawn by us, not measured). Autoregressive decoding, speculative decoding (verifier waits for the drafter and vice versa), and SSD in which the drafter runs concurrently on separate hardware and a cache hit eliminates the drafting wait.

quantities and on the speculation being verified, not on the verifier's random draws; and (iii) the draft probabilities that the verifier uses in the ratio $\min\{1, p/q\}$ *are exactly those from which the draft tokens were sampled (including any Saguaro reweighting or a trivial uniform backup). Then the emitted token sequence has the law of autoregressive sampling from* p*.*

Proof. By induction on rounds. Suppose the emitted prefix x after round $T-1$ has the correct law. Round T's speculation s is either a hit entry $\mathcal{S}(v)$ or a fallback sample. Conditional on x and on the realised outcome v, (i)–(ii) imply that s is distributed as the draft chain with the stated probabilities: whether v is a key is determined by draft-side quantities independent of the verifier's draws, and the entry stored under v was sampled with independent randomness, so conditioning on a hit does not bias its content. A fallback speculation is also a draft chain given x (a uniform-random backup is a valid draft with q uniform; it merely has low acceptance). By (iii), Theorem 3.5 (exactness of one verification round for an arbitrary draft law) then shows that the tokens emitted in round T have the law of p-sampling given x. Hence the induction closes. □

Assumption (iii) is where a faulty implementation could silently break exactness, in the same way as the mismatched-temperature pitfall of Section 3.6: Saguaro's biased sampling (Section 11.5) changes the draft law, and the verifier must receive the biased probabilities. The paper's system communicates "K-step logits for acceptance" alongside the tokens for this reason[S].

11.3 Speedup theory

Theorem 11.2 (SSD speedup; Theorem 7 of Kumar et al. [18])**.** *Let* E_{hit} *and* E_{miss} *be the expected number of tokens generated per round when the speculation comes from the primary respectively the*

Algorithm 2: The SSD framework (paraphrase of Algorithm 1 of Kumar et al. [18]).

Verifier process (target hardware):
  prefill the prompt; wait for the first speculation s from the speculator
  while *no end-of-sequence* **do**
    $(k, t^*) \leftarrow$ verify s with the target (accept/reject rule of Chapter 3); append the emitted tokens
    send (k, t^*) to the speculator; wait for the next speculation s
  end
Speculator process (separate hardware):
  prefill the prompt; draft the first speculation s and send it to the verifier
  while *no end-of-sequence* **do**
    predict a set of likely outcomes $\mathcal{V}$ of verifying s (Saguaro cache, §11.4)
    for each $v \in \mathcal{V}$, draft the speculation $\mathcal{S}(v)$ in parallel (Saguaro sampling, §11.5)
    receive the realised outcome v_{real}
    if $v_{\text{real}} \in \mathcal{V}$ **then**
      $s \leftarrow \mathcal{S}(v_{\text{real}})$
    else
      $s \leftarrow$ fallback speculation (§11.6)
    end
    send s (with its draft probabilities) to the verifier
  end

backup speculator. The expected speedup of SSD relative to autoregressive decoding (batch size 1) is

$$\text{speedup}_{\text{SSD}} = \frac{p_{\text{hit}} E_{\text{hit}} + (1 - p_{\text{hit}}) E_{\text{miss}}}{p_{\text{hit}} \max(1, T_p) + (1 - p_{\text{hit}})(1 + T_b)}. \tag{11.1}$$

Derivation (ours, [D]). Rounds are i.i.d. in the sense of a regenerative process (the state is whether the next speculation is from the primary or backup speculator, and the chain mixes). By the renewal-reward theorem the long-run token rate is $\mathbb{E}[\text{tokens per round}]/\mathbb{E}[\text{duration per round}]$ (in units of one target step, which is the cost of one autoregressive token). On a hit, the next speculation is ready after $\max(1, T_p)$: the verifier's one unit of work and the drafter's T_p run concurrently, and the round lasts as long as the slower. On a miss, the verifier must wait for the fallback, so the round lasts $1 + T_b$ (the verification just completed plus the fallback drafting); numerator and denominator are the corresponding weighted means. □

The stationary hit probability. The theorem's p_{hit} is an unconditional probability, but hits are not independent across rounds, because a hit leaves the next speculation to the primary and a miss to the backup. The state "previous speculation was primary" is entered exactly after a hit, so the hit indicator is a two-state Markov chain with $P(\text{hit} \mid \text{hit}) = p_{\text{hit},p}$ and $P(\text{hit} \mid \text{miss}) = p_{\text{hit},b}$. Its stationary hit probability h solves $h = h p_{\text{hit},p} + (1 - h) p_{\text{hit},b}$, i.e.

$$h = \frac{p_{\text{hit},b}}{1 + p_{\text{hit},b} - p_{\text{hit},p}} \text{ [D]} \tag{11.2}$$

(not stated in the paper in this form; we derived it and checked it by simulation below). In particular, $h \geq p_{\text{hit},b}$ whenever $p_{\text{hit},p} \geq p_{\text{hit},b}$, and $h = p_{\text{hit},p}$ when both are equal.

Corollary 11.3 (SSD never loses to SD with the same drafter; Corollary 8 of the paper)**.** *Run SSD with primary and backup both equal to the SD drafter $\mathcal{M}$ (so $E_{\text{hit}} = E_{\text{miss}} = E$, $T_p = T_b = T$). Then* $\text{speedup}_{\text{SSD}} = E/[p_{\text{hit}} \max(1, T) + (1 - p_{\text{hit}})(1 + T)] \geq E/(1 + T) = \text{speedup}_{\text{SD}}$*, with strict*

(p_p, p_b)	Simulated p_{hit}	Closed form
(0.9, 0.5)	0.8326	0.8333
(0.8, 0.3)	0.6006	0.6000
(0.7, 0.6)	0.6652	0.6667
(0.95, 0.2)	0.8011	0.8000
(0.6, 0.6)	0.6009	0.6000

α	K	T_p	T_b	DES speedup	Theorem 11.2
0.8	5	0.9	0.25	3.369	3.371
0.7	6	1.1	0.1	2.552	2.552
0.85	4	0.7	0.3	3.075	3.076

Table 11.1: Left: stationary hit probability, simulation versus Equation (11.2) ([M]). Right: discrete-event simulation of SSD versus Theorem 11.2 ([M]); the backup has acceptance $\alpha - 0.1$.

inequality iff $p_{\text{hit}} > 0$ *and* $T > 0$.

Proof. $\max(1, T) \le 1 + T$, with equality iff $T = 0$; therefore the denominator is at most $1 + T$. □

For $T \le 1$ the gain over SD is $(1 + T)/(1 + T - p_{\text{hit}}T)$ ([D]): it equals one at $p_{\text{hit}} = 0$, grows with p_{hit}, and approaches $1 + T$ as $p_{\text{hit}} \to 1$. With a drafter whose step costs 25% of the target's ($T = 0.25$), SSD can *at most* yield a 1.25× gain over SD by hiding the draft; any larger gain must come from a better speculation $E_{\text{hit}} > E_{\text{SD}}$ made possible by the free time budget (a larger drafter or longer lookahead). This is what the paper's second corollary expresses.

Corollary 11.4 (Speedup sandwich; Corollary 9 of the paper)**.** *If* $T_p < 1$ *and the backup is instantaneous* ($T_b = 0$)*, then*

$$\frac{(1 + T_{\text{SD}})E_{\text{hit}}}{E_{\text{SD}}}\, p_{\text{hit}} \;\le\; \frac{\text{speedup}_{\text{SSD}}}{\text{speedup}_{\text{SD}}} \;\le\; \frac{(1 + T_{\text{SD}})E_{\text{hit}}}{E_{\text{SD}}}.$$

Derivation (ours). With $T_p < 1$ and $T_b = 0$ the denominator of (11.1) is $p_{\text{hit}} + (1 - p_{\text{hit}}) = 1$, so $\text{speedup}_{\text{SSD}} = p_{\text{hit}}E_{\text{hit}} + (1 - p_{\text{hit}})E_{\text{miss}}$, while $\text{speedup}_{\text{SD}} = E_{\text{SD}}/(1 + T_{\text{SD}})$. Dropping the non-negative term $(1 - p_{\text{hit}})E_{\text{miss}}$ gives the lower bound; assuming $E_{\text{miss}} \le E_{\text{hit}}$ (the backup is no better than the primary) gives the upper bound. □

Batch size. At batch size b the whole batch is verified together, so a round is a "hit" for the batch only if every sequence hits; the paper's Corollary 16 replaces p_{hit} in the denominator by p_{hit}^b (under independence across sequences) and shows that the speedup approaches $[p_{\text{hit}}E_{\text{hit}} + (1 - p_{\text{hit}})E_{\text{miss}}]/(1 + T_b)$ as $b \to \infty$[S]. This is the quantitative reason that the fallback matters.

11.3.1 Numerical verification of the theory

We simulated the SSD timing model to check these formulas ([M]; sim/exp5_ssd.py). First, a Markov-chain simulation of 4×10^5 rounds reproduces the stationary hit probability (11.2) (Table 11.1). Second, a discrete-event simulation of 2×10^5 rounds with geometric token acceptance (rejection probability per position $1 - \alpha$, lookahead K), hit/miss dynamics given by the Markov chain, and the round durations above reproduces Theorem 11.2 to within 0.002 in all three configurations (Table 11.1); the residual difference is Monte-Carlo noise. These checks validate our reading of the theorem and the derivation above; they do not test the paper's *empirical* claims.

Example 11.5 (Worked example)**.** Take $\alpha = 0.8$, $K = 5$, so $E = (1 - 0.8^6)/(1 - 0.8) = 3.689$ tokens per round, a drafter with $T_{\text{SD}} = 0.25$ and $p_{\text{hit}} = 0.9$ with the same drafter as backup.

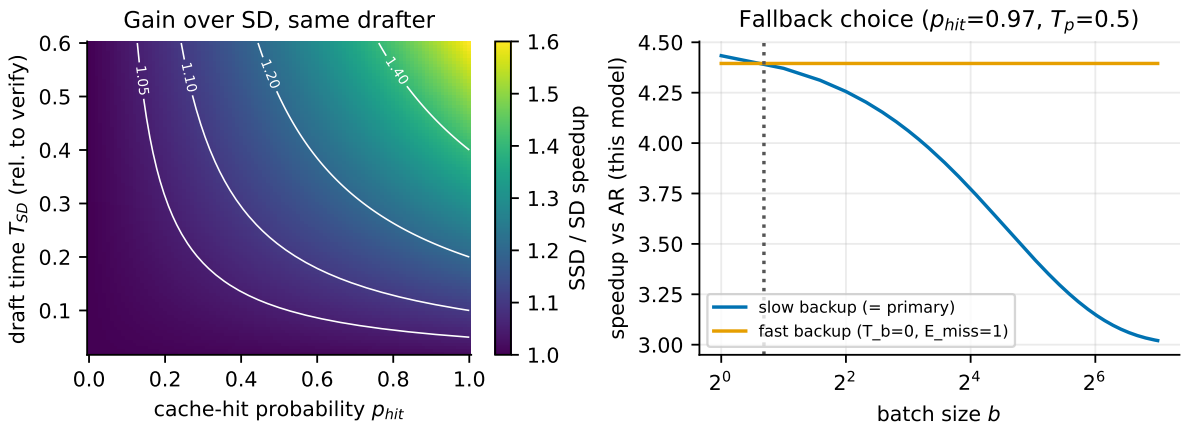


Figure 11.2: Left: gain of SSD over SD with the same drafter, $\alpha = 0.8$, $K = 5$, as a function of the hit probability and of the drafter's relative cost ([M]). Right: modelled speedup versus batch size for a slow backup (primary reused) versus an instantaneous but poor backup; the dotted line is the closed-form crossover $b^\star = 1.61$, the numerical crossover is at $b = 2$ ([M]; toy parameters $E_{\text{hit}} = 4.5$, $E_{\text{miss}} = 1$, $p_{\text{hit}} = 0.97$, $T_p = 0.5$).

Then SD gives $3.689/1.25 = 2.951\times$, while SSD gives $3.689/(0.9 \cdot 1 + 0.1 \cdot 1.25) = 3.599\times$, a $1.22\times$ gain; the sandwich upper bound with an identical drafter is $E(1 + T_{\text{SD}})/E = 1.25$, i.e. $3.689\times$ over AR. To gain more, SSD must use the freed time to improve E_{hit}, e.g. by drafting with a larger model that is $T_p \leq 1$ slow ([M], sim/results/exp5_ssd.json).

The left panel of Figure 11.2 maps the gain over SD with the same drafter as a function of p_{hit} and T_{SD}; the right panel is discussed in Section 11.6.

11.4 Challenge 1: predicting verification outcomes (the Saguaro cache)

11.4.1 The fan-out problem

Given a cache budget B, which outcomes should be prepared? An outcome with k accepted tokens requires a guess of the bonus token. Saguaro takes the top-F_k tokens in the draft logits at lookahead position k, excluding the sampled draft token at that position (a rejected draft token has $p < q$ and hence zero residual mass, so it cannot be the bonus token); F_k is the *fan-out* at position k[S]. Because each prepared outcome requires drafting a full length-K continuation, the budget is the total $\sum_k F_k \leq B$, essentially the number of parallel branches the drafter can finish before verification returns.

11.4.2 Optimal geometric fan-out

Theorem 11.6 (Geometric fan-out; Theorem 12 of Kumar et al. [18])**.** *Suppose the primary speculator has per-token acceptance rate α and a power-law cache hit rate, $1 - p_{\text{hit}}(F) = F^{-r}$ for some $r > 0$. Then the hit-rate-maximising fan-outs under $\sum_{k=0}^{K} F_k \leq B$ follow a capped geometric series*

$$F_k = F_0\, \alpha^{k/(1+r)}\ (k < K), \qquad F_K = F_0\, \alpha^{K/(1+r)} (1 - \alpha)^{-1/(1+r)},$$

with F_0 chosen so that $\sum_k F_k = B$.

(α, K, r, B)	Uniform p_{hit}	Geometric p_{hit}	Numerical optimum
(0.8, 5, 0.6, 24)	0.5647	0.5822	0.5821
(0.7, 5, 0.6, 24)	0.5647	0.5808	0.5807
(0.8, 5, 1.0, 24)	0.7500	0.7633	0.7633
(0.9, 7, 0.6, 48)	0.6587	0.7017	0.7015
(0.8, 5, 0.6, 96)	0.8105	0.8181	0.8181

Table 11.2: Hit probability of uniform versus geometric fan-out allocation versus a numerical optimum, for the power-law model $1 - p_{\text{hit}}(F) = F^{-r}$ ([M]).

Derivation (ours; relaxes integrality). The number of accepted tokens is distributed as $\pi_k = \alpha^k(1-\alpha)$ for $k < K$ and $\pi_K = \alpha^K$. Given k, the probability that the bonus token is in the cache is $1 - F_k^{-r}$. Maximising $\sum_k \pi_k(1 - F_k^{-r})$ is equivalent to minimising $\sum_k \pi_k F_k^{-r}$ subject to $\sum_k F_k = B$. The Lagrangian condition $-r\pi_k F_k^{-r-1} = -\lambda$ gives $F_k \propto \pi_k^{1/(1+r)}$. For $k < K$ this is $\propto \alpha^{k/(1+r)}(1-\alpha)^{1/(1+r)}$; for $k = K$ it is $\propto \alpha^{K/(1+r)}$. Normalising by the common factor $F_0 = c(1-\alpha)^{1/(1+r)}$ yields the stated series (the capped term has the extra factor $(1-\alpha)^{-1/(1+r)}$). The objective is convex in F for $r > 0$, so the stationary point is the global minimiser. □ □

The theorem formalises the intuition that fan-out should be spent where verification outcomes are likely: positions k near the end of the lookahead are reached only if many tokens are accepted, so few branches should be guessed there, except for the capped last position where all tokens were accepted. The paper reports empirically that the rejection rate $1 - p_{\text{hit}}(F)$ falls approximately as a power law in F (both for Llama-3 and Qwen-3), that the geometric allocation improves hit rate and end-to-end speed over a uniform allocation, and that the advantage grows with sampling temperature[S].

Our numerical check. We computed $p_{\text{hit}} = \sum_k \pi_k(1 - F_k^{-r})$ for the geometric allocation, a uniform allocation, and a numerical optimiser (SLSQP, continuous F_k) under the same budget ([M]; Table 11.2). The geometric allocation matches the numerical optimum to four decimals in every case, as it must by the derivation above, and beats the uniform allocation by 0.8–4.3 percentage points of hit rate, with the largest margin at the longer lookahead ($K = 7$, $B = 48$). Note that the table uses idealised parameters (power-law hit rates, continuous fan-out) and says nothing about real hit rates.

Prediction accuracy. The paper reports that using the top draft logits to predict the bonus token reaches up to 90% accuracy in the best case[S]; the figure depends on temperature, batch size and fan-out, and we did not retrieve per-setting values.

11.5 Challenge 2: balancing hit rate and acceptance (Saguaro sampling)

The bonus token is, whenever a draft token was rejected, sampled from the residual $r(t) \propto \max(p(t) - q(t), 0)$. The residual is hard to predict at higher temperatures because it depends on the target. The paper's key observation is that the residual is also a function of the draft distribution q: lowering $q(t)$ raises $r(t)$. Saguaro sampling therefore *downweights* the draft's

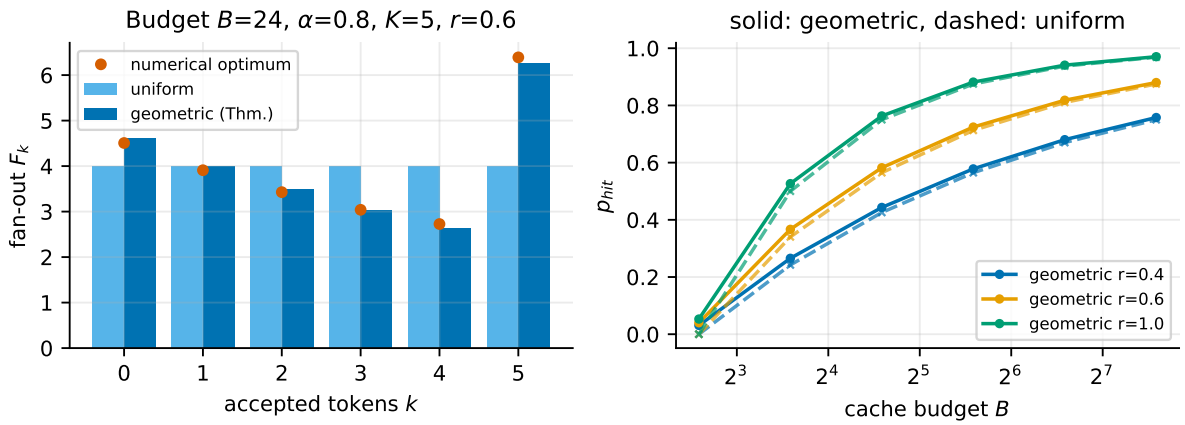


Figure 11.3: Left: fan-out per number of accepted tokens k for $B = 24$, $\alpha = 0.8$, $K = 5$, $r = 0.6$: uniform allocation, the geometric allocation of Theorem 11.6, and the numerical optimum ([M]). Right: hit probability against cache budget B for several r; solid curves geometric, dashed uniform.

probability on the F tokens it will cache, which concentrates the residual on exactly those tokens and so makes the bonus token more likely to be in the cache by construction.

Definition 11.7 (Saguaro sampling; Definition 14 of the paper)**.** For draft logits z, fan-out F and downweighting constant $C \in [0, 1]$, the sampling distribution is $\sigma_{F,C}(z) \propto C \exp(z_t)$ if $t \in \text{top}_F(z)$ and $\exp(z_t)$ otherwise.

The price is a smaller acceptance rate $\alpha = \sum_t \min\{p, q\}$, because q moves away from p. The paper's Theorem 15 states that for fixed F the hit rate increases monotonically as $C \to 0$; an appendix construction shows distributions p, q where biasing the draft in this way yields net speedup, with acceptance unchanged and hit rate strictly increased[S]. Since the verifier uses the biased q in its ratio, the scheme remains lossless (Proposition 11.1).

11.5.1 A toy replication of the trade-off

We reproduced the qualitative trade-off on a synthetic target (Zipf, $V = 2000$, logits with exponent 1.1) and a noisy-copy draft (Gaussian logit noise with standard deviation 0.9) for fan-outs $F \in \{4, 16\}$ ([M]; sim/exp5_ssd.py; Table 11.3, Figure 11.4). The round hit rate was computed from the residual's mass on the cached tokens (and the target's mass on them when all $K = 5$ tokens are accepted), and the model speedup uses an instantaneous backup returning one token and $T_p = 1$. Reducing C from 1 to 0.4 at $F = 16$ raises the hit rate from 0.375 to 0.618 while acceptance falls from 0.713 to 0.649; the modelled speedup peaks at $C = 0.4$ (2.01×) and declines again for smaller C as acceptance collapses (1.69× at $C = 0$). For $F = 4$ the optimum moves to $C \approx 0.1$ (1.74×): the fewer tokens are cached, the more aggressively one must downweight them. The interior optimum is the same qualitative phenomenon as in the paper, but our numbers come from a toy distribution and an idealised timing model and should not be compared with the paper's empirical curves.

11.6 Challenge 3: handling cache misses (Saguaro fallback)

At large batch sizes and temperatures misses are nearly certain, and because the entire batch waits for the slowest fallback, a slow just-in-time fallback erases the asynchrony benefit.

F	C	α	$P(\text{bonus} \in \text{cache} \mid \text{reject})$	Round p_{hit}	Model speedup
4	1.0	0.713	0.135	0.157	1.32×
4	0.8	0.708	0.200	0.210	1.42×
4	0.6	0.694	0.289	0.284	1.54×
4	0.4	0.671	0.389	0.371	1.65×
4	0.25	0.644	0.470	0.446	1.72×
4	0.1	0.606	0.550	0.526	1.74×
4	0.05	0.591	0.576	0.553	1.74×
4	0.0	0.575	0.601	0.579	1.73×
16	1.0	0.713	0.357	0.375	1.76×
16	0.8	0.706	0.434	0.438	1.87×
16	0.6	0.687	0.530	0.519	1.96×
16	0.4	0.649	0.639	0.618	2.01×
16	0.25	0.600	0.720	0.699	1.97×
16	0.1	0.528	0.794	0.781	1.84×
16	0.05	0.494	0.818	0.807	1.77×
16	0.0	0.456	0.840	0.833	1.69×

Table 11.3: Saguaro-sampling toy ([M]): acceptance α, probability that the bonus token is in the cache given a rejection, per-round hit rate, and modelled speedup with $K = 5$, $T_p = 1$, instantaneous backup.

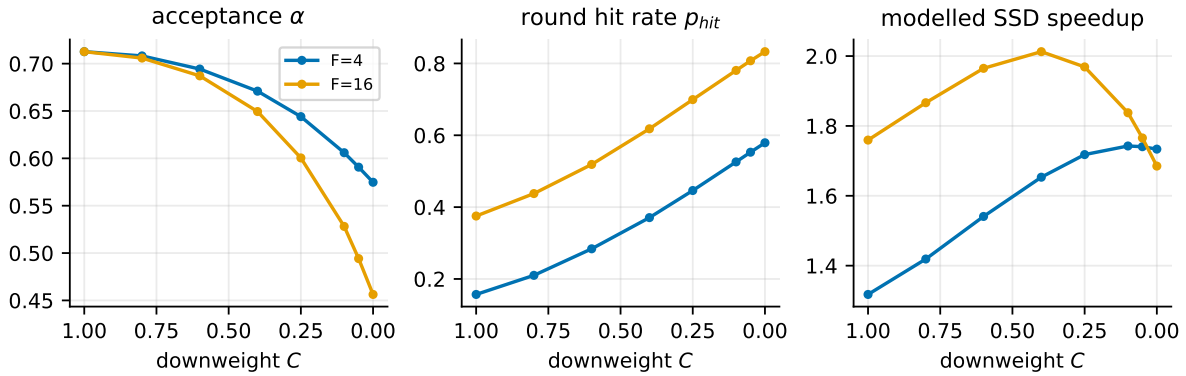


Figure 11.4: Saguaro sampling toy ([M]): acceptance, round hit rate and modelled speedup versus the downweighting constant C (axes reversed so that stronger bias is to the right) for fan-outs 4 and 16.

Saguaro's fallback sets the backup speculator equal to the primary at small batch size, and switches to a very fast, lower-quality speculator (the paper's implementation returns random tokens; it suggests n-gram methods as a better alternative) for $b > b^{\star}$[S].

Theorem 11.8 (Optimal fallback; Theorem 17 of the paper, with our closed form)**.** *Let the slow backup equal the primary ($E_{\text{miss}} = E_{\text{hit}}$, $T_b = T_p < 1$) and let the fast backup have $T_b = 0$ and expected tokens $E^{\text{fast}}_{\text{miss}}$. With $h = p_{\text{hit}}$, $\bar{E} = hE_{\text{hit}} + (1-h)E^{\text{fast}}_{\text{miss}}$ and batch hit probability h^b, the slow backup is better iff $b < b^{\star}$ where*

$$b^{\star} = \frac{\ln\left[(1 + T_p - E_{\text{hit}}/\bar{E})/T_p\right]}{\ln h}.$$

Derivation (ours). From Corollary 16, the slow-backup speedup is $E_{\text{hit}}/[h^b + (1-h^b)(1+T_p)] = E_{\text{hit}}/(1 + T_p - h^b T_p)$, while the fast backup gives $\bar{E}/[h^b + (1-h^b)] = \bar{E}$, independent of b (with $T_p < 1$). Equating, $1 + T_p - h^b T_p = E_{\text{hit}}/\bar{E}$, hence $h^b = (1 + T_p - E_{\text{hit}}/\bar{E})/T_p$; since h^b decreases with b and the slow-backup speedup increases with h^b, the slow backup wins for smaller b. □

With the toy parameters of Figure 11.2 ($E_{\text{hit}} = 4.5$, $E^{\text{fast}}_{\text{miss}} = 1$, $h = 0.97$, $T_p = 0.5$) the closed form

gives $b^\star = 1.61$, and the numerical crossover is at $b = 2$ ([M]). These parameters were chosen to illustrate the mechanism; they are not the paper's. The paper's empirical result is that a fast random-token backup outperforms a slow neural speculator as batch size grows, as the theorem predicts, and that with this strategy Saguaro retains about a 20% advantage over SD at larger batch sizes despite spending more compute per batch element[S]. Figure 6 (right) of the paper also projects that dedicating more devices to speculation (a larger fan-out) increases the hit rate and reduces latency at batch size 16, with diminishing returns[S].

11.7 Implementation and cost

11.7.1 System design

The authors implement Saguaro in a custom PyTorch inference engine with PagedAttention, continuous batching, tensor parallelism, BF16, `torch.compile` and CUDA graphs; the target is split over four H100s and the drafter has a fifth, separate H100. The devices communicate once per round via NCCL: the target sends the outcome keys (sequence id, accepted-prefix length, recovery token), the current sequence lengths, draft block tables for KV addressing and per-row temperatures; the drafter returns a cache-hit bitmap, the K speculative tokens per sequence and the K-step logits needed for acceptance. No KV cache is transferred between devices and the target never sees the speculation cache[S]. During speculation, all $F(K+1)$ branches of each sequence are decoded in parallel using a custom sparse attention mask in which each branch attends to the verified trunk and its own forking path, with FlashAttention where possible and FlashInfer for custom-mask multi-query paths. The authors note that materialising these masks and the sparse, non-coalesced memory access of the custom-mask kernels dominate the critical path and limit how many steps can be drafted profitably, so that *most of the end-to-end speedup comes from hiding draft latency rather than from increasing lookahead*[S]. This agrees with our observation in Section 11.3 that the pure latency-hiding gain is bounded by $1 + T_{\text{SD}}$.

11.7.2 Overheads

Appendix C of the paper gives the cost accounting. With c the drafter's relative FLOPs per pass, SSD decodes $BK(K+1)F$ draft tokens per round versus BK in SD, i.e. a factor $(K+1)F$ more draft FLOPs; for example, $K = 5$ and $F = 4$ is a $24\times$ increase in draft computation ([D], from the formula). Entire pre-speculated chains that are not used are wasted compute, in addition to the rejected tokens already wasted by SD. The cache stores $O(BFK(K+1)(V+1))$ values and the paper says this amounts to hundreds of megabytes; communication per round is $O(B)$ bits from the target and $O(BKV)$ bits for the logits from the drafter over NVLink, which the authors say is not a bottleneck[S]. The extra draft device adds one GPU to the target's four; the authors claim that the throughput–latency Pareto frontier improves *after* accounting for the additional hardware, with the largest gains at low batch size[S]. Because the extra hardware is a real cost, any comparison with SD should be read in terms of devices as well as latency; we return to this in Chapter 14.

Table 11.4: Decode throughput (tokens/s) of autoregressive decoding (AR), SD and SSD (Saguaro) from Appendix B.3 of Kumar et al. [18] ([S]), on 4 H100 for AR and SD and 5 for SSD, batch size 1, greedy. Ratios are as printed in the paper.

Model	Dataset	AR	SD	SSD	SSD/SD	SSD/AR
Llama-3.1 70B / 1B	HumanEval	54.7	176	283	1.60×	5.17×
	UltraFeedback	54.7	138	215	1.55×	3.93×
	Alpaca	54.7	145	224	1.55×	4.10×
	GSM8K	54.7	188	301	1.60×	5.50×
	Average	54.7	161.8	255.8	1.58×	4.68×
Qwen3 32B / 0.6B	HumanEval	88.8	146	222	1.52×	2.50×
	UltraFeedback	88.8	122	174	1.43×	1.96×
	Alpaca	88.8	127	185	1.47×	2.08×
	GSM8K	88.8	152	234	1.54×	2.64×
	Average	88.8	136.8	203.8	1.49×	2.29×

11.8 Combining with EAGLE-3 and token trees

Appendix E of the paper explains that SSD composes with EAGLE-3 with one nuance: an SSD drafter begins round $T+1$ before round T has finished verifying and thus does not have the target activations from round T that an EAGLE-3 drafter normally conditions on. With lookahead K it therefore conditions on up to $2K$ self-generated activations, lowering acceptance for the latter K tokens, which the authors suggest mitigating by training the drafter to retain acceptance under longer periods of self-conditioning[S]. In their implementation the target sends accepted counts and the target activations of the relevant positions to the drafter over NCCL. Token-tree methods can be combined by pre-speculating a tree for each outcome instead of a chain, which requires a more elaborate mask. The authors say the joint design space is largely unexplored[S]; as of October 2026 we found no published result that does it.

11.9 Published results

11.9.1 Main results

The headline evaluation uses Llama-3.1-70B-Instruct with Llama-3.2-1B as draft (TP=4 for the target) and Qwen3-32B with Qwen3-0.6B, batch size 1, greedy decoding, 128 prompts from each of HumanEval, Alpaca, GSM8K and UltraFeedback with 512 generated tokens, vanilla sampling and decode-only throughput; vLLM v0.16.0 and SGLang v0.5.9 are baselines with radix caching disabled, and the baseline lookahead is five draft tokens[S]. Table 11.4 reproduces the throughput table of Appendix B.3.

11.9.2 How strong are the baselines?

The comparison that matters is not SSD versus the authors' own SD (1.58× and 1.49×) but versus the strongest tuned open-source baseline. Appendix B.4 reports sweeps over lookahead and tree size in SGLang and over lookahead in vLLM for Llama-3.1-70B/1B (Table 11.5; Figure 8.1 in Chapter 8 plots the same numbers). The best SGLang standalone configuration (10 steps, top-$k = 3$ tree) reaches 201.9 tokens/s and the best SGLang EAGLE-3 configuration (8 steps, top-$k = 3$) 192.8; in vLLM, EAGLE-3 is best at $K = 6$ (197.6) while standalone SD *declines*

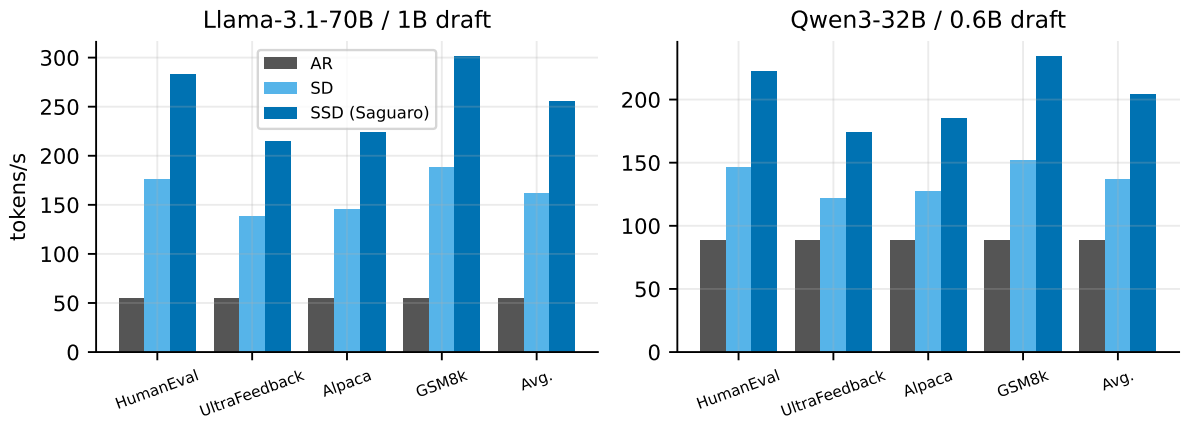


Figure 11.5: Re-plot of Table 11.4 ([S]; our plot of the published numbers).

Table 11.5: Best baseline configurations in the SSD paper's sweeps (decode tokens/s, Llama-3.1-70B/1B, [S]; Appendix B.4). SGLang cells are best over the tested (steps, top-k) grid; "–" for incompatible configurations is omitted.

Engine	Method	Best (tok/s)	Configuration
SGLang 0.5.9	Standalone draft	201.9	10 steps, top-k 3
SGLang 0.5.9	EAGLE-3	192.8	8 steps, top-k 3
vLLM 0.16.0	EAGLE-3	197.6	$K = 6$
vLLM 0.16.0	Standalone draft	136.0	$K = 2$ (decreasing in K)
–	SSD (Saguaro), own engine	255.8	average over four datasets

monotonically with K from 136.0 at $K = 2$ to 70.5 at $K = 12$, a pattern on which the text we retrieved does not comment[S]. Dividing the SSD average of 255.8 by the best baseline (201.9) gives 1.27×, consistent with the paper's claim of being on average about 30% faster than the strongest speculative baselines ([D], our ratio; note that the paper's baseline sweeps are for one model pair and the "average" is over datasets). Two observations follow. First, the authors' own SD (161.8, chain, $K = 5$) is comparable with the SGLang chain results at $K \in \{4, 6\}$ (159.4 and 176.7 at top-$k = 1$), confirming that their SD is a fair baseline, but the best SGLang *tree* configuration (201.9) is 11% faster than the best SGLang chain configuration (182.1 at 8 steps), and tuned trees are the right yardstick for "strongest SD" ([D], our ratio). Second, a standalone 1B draft with a tree beats EAGLE-3 in SGLang for this target; the paper notes that Llama-3.2-1B outperforms EAGLE-3 as a draft for Llama-3.1-70B in SGLang[S], a reminder that a pretrained small LM can beat a trained feature-level head when the target is large (cf. the vLLM study's finding that draft models win on the 70B target[S] [27]).

11.9.3 Independent reproduction attempts

We found one secondary report: an F22 Labs blog post describing a small reproduction on two RTX 4090 GPUs with Qwen3-1.7B as target and Qwen3-0.6B as draft at batch size 1 and greedy decoding, obtaining 201.78 tokens/s versus 73.68 for vLLM autoregressive decoding (2.74×), with 18–21 GB of VRAM per GPU versus 5.4 GB for vLLM[V] [30]. The blog is not peer reviewed and the comparison is against autoregressive decoding rather than against SD, so it supports the existence of a speedup on consumer hardware but not the paper's SSD-versus-SD claim. The author notes that gains shrink at larger batch sizes, that the official implementation

targets H100/H200 (sm90) and that $k = 2$ or 3 worked best in their runs[V]. We did not find an independent replication of the paper's H100 results.

11.10 Relation to concurrent and earlier overlap methods

The SSD authors position the work against earlier attempts to overlap drafting with verification: AMUSD and PEARL speculate the next round during verification but prepare only for the all-accepted outcome; DSI overlaps with multiple logical verifier copies; Mirror-SD uses early-exit signals and heterogeneous devices; SwiftSpec and SpecBranch prepare a token tree branching off the speculation being verified, but SwiftSpec treats only greedy sampling with a just-in-time fallback that fails at large batch and temperature, and SpecBranch is, in their view, approximately the special case with a single branching point and the regular speculator as fallback[S] [18]. The earlier papers' own claims are reported in Chapter 7; we have not independently verified the authors' characterisation of competing work. SSD's distinctive contributions are the *general framework* (the outcome-prediction formulation), the three analyses of cache shape, sampling and fallback, and an implementation evaluated against production engines.

11.11 Critical assessment

Strengths. (i) The framework is clean and the theory is simple and verifiable (we reproduced Theorem 11.2 to three decimals by simulation). (ii) It attacks the one remaining serial dependency, which grows in relative importance as target models speed up through better kernels and higher acceptance. (iii) The system is exact (Proposition 11.1) and does not change the target.

Weaknesses and limits.

1. *Extra hardware.* The headline result uses five GPUs versus four. Per-GPU throughput is therefore *lower* than the raw ratios suggest; the paper's Pareto claim is the relevant evidence, but we lack its numbers. A fair comparison would also consider using the fifth GPU for a larger tensor-parallel degree or for another replica.
2. *Batch size and temperature.* Misses become inevitable at large batch size (hit probability enters as p_{hit}^b) and temperature, so the gain is largest at batch size one and greedy decoding, exactly the setting of the headline numbers. The paper evaluates batch scaling but its limitations section states that, like SD generally, the method is ineffective for throughput-bound workloads such as large-scale RL rollouts or offline data generation[S].
3. *Bounded latency-hiding gain.* With the same drafter, the gain over SD is at most $1 + T_{\text{SD}}$ (Section 11.3); for EAGLE-style drafters with T_{SD} of 10–20% the available gain is 10–20%. Larger gains require better speculation, which is limited by custom-mask attention overheads in the authors' own account.
4. *Engineering complexity.* A custom engine with cross-device synchronisation, a sparse-mask multi-query attention path and page-table rollback; the official code targets Hopper. Neither the vLLM nor the TensorRT-LLM documentation we retrieved lists SSD among its supported methods (Chapter 12).

5. *Version drift in headline claims.* The ICLR poster page's abstract says "up to 2× faster than optimized speculative decoding baselines" while the arXiv v3 abstract says "on average 30% faster than the strongest speculative decoding baselines"[S]; the main-text average gain over the authors' own SD is 1.58× (Llama) and 1.49× (Qwen). These are different baselines and should not be conflated.
6. *Limited model coverage.* Two target families (Llama-3.1-70B, Qwen3-32B), dense models only, one hardware generation.

Open questions. How does SSD interact with MoE targets, where verification width changes the expert-touch fraction? Can a feature-level drafter be trained to be robust to $2K$ steps of self-conditioning? What is the best split of a fixed GPU budget between target tensor parallelism and speculation devices? These questions are echoed in Chapter 18.

11.12 Summary

SSD in five lines

- SSD overlaps drafting with verification by pre-computing speculations for likely verification outcomes on a separate device; a cache hit hides drafting latency ([S]).
- Speedup is $[p_{\text{hit}} E_{\text{hit}} + (1 - p_{\text{hit}}) E_{\text{miss}}] / [p_{\text{hit}} \max(1, T_p) + (1 - p_{\text{hit}})(1 + T_b)]$; we verified it by discrete-event simulation ([M]) and derived the stationary hit probability $p_b/(1 + p_b - p_p)$ ([D]).
- Three designs matter: geometric fan-out (we proved it by a Lagrangian), Saguaro sampling (a hit-rate versus acceptance trade-off with an interior optimum in our toy) and a batch-dependent fallback ($b^\star$ closed form).
- Reported results are up to 5× over AR and about 1.3–1.6× over SD on Llama-3.1-70B/Qwen3-32B at batch size 1 on 4+1 H100 ([S]), not independently replicated on H100s.
- The gain over SD with an identical drafter is bounded by $1 + T_{\text{SD}}$; it costs an extra device and shrinks at large batch and temperature.

Chapter 12

Serving Systems: vLLM, SGLang, TensorRT-LLM and the Training Ecosystem

Algorithms matter in production only to the extent that serving engines implement them correctly and fast. This chapter summarises what the public documentation, engineering posts and papers retrieved for this report say about speculative-decoding support in the main open engines as of October 2026, what the constraints are, how drafters are trained at scale, and what a practitioner should instrument. Documentation changes quickly: every statement below is tied to a dated source, and "not mentioned" in a page we read is not evidence that a feature is absent.

12.1 What an engine must do

A speculative decoding implementation inside a continuous-batching engine has to solve problems that do not appear in the algorithmic description of Chapter 3:

1. **Variable tokens per step.** Different sequences in a batch accept different numbers of draft tokens, so the engine must handle ragged per-sequence progress, positions, and stop conditions in one batched forward pass.
2. **KV-cache allocation and rollback.** The verifier writes $\gamma + 1$ positions per sequence; rejected positions must be freed or overwritten before the next step. The SSD paper describes the same requirement in its engine as "a host-side post-processing step after each verification"[S] [18]; TensorRT-LLM's documentation states that space for rejected draft tokens is freed before the next iteration[S] [39].
3. **Draft-model state.** A separate draft needs its own weights and KV cache; an EAGLE-style drafter needs the target's hidden states, so the engine must expose them from the verification pass.
4. **CUDA graphs and overhead.** Drafting loops of γ small kernels are CPU-launch-bound unless captured; the vLLM study attributes 3–12% of total time to engine overhead (scheduling and bookkeeping) and finds that sampling (rejection) time is negligible, below 1.7%[S] [27]. SpecVocab's authors point out that the cost of large vocabularies can be hidden by CPU overhead in naive implementations[S] [56].
5. **Tree attention.** Tree verification needs custom masks and position ids; kernels differ between engines (FlashInfer for tree configurations versus FlashAttention-3 for chains in SGLang according to the vLLM study[S]).
6. **Interactions with other features.** Structured (grammar-constrained) decoding, prefix caching, chunked prefill, pipeline parallelism, quantised KV caches, and multi-LoRA serving all touch the verification pass.

12.2 vLLM

vLLM's documentation (read October 2026) groups methods into three families[S] [52].

- *Model-based* (said to give the largest latency reductions): EAGLE, multi-token prediction (MTP), standalone draft models, the Parallel Draft Model (PARD), LiLiCorr and hidden-state extraction.
- *Lightweight*: n-gram (prompt lookup) and suffix decoding, which give smaller speedups without adding load at peak traffic.
- *Verification and scheduling variants*: dynamic speculative decoding, adaptive verification (currently listed for one named method), FLy verification (explicitly lossy and not distribution-preserving), and an experimental custom-proposer interface.

The page states that its guidance targets medium-to-low QPS, memory-bound workloads, that gains vary by model, traffic, hardware and sampling settings, and that the standard rejection sampler is theoretically lossless subject to hardware numerical precision. It also warns that changing the batch size can shift log-probabilities (so outputs can differ across runs) and lists version constraints: pipeline parallelism did not compose with speculative decoding in vLLM versions up to 0.15.0, draft models were unsupported up to 0.10.0, heterogeneous-vocabulary drafting ("TLI") supports only greedy draft sampling, and Gemma 4 assistant checkpoints must use the "mtp" method[S]. The configuration keys include the method, number of speculative tokens, draft tensor-parallel size, a `parallel_drafting` flag, the rejection-sampling method and the n-gram/suffix parameters. The page does not describe general tree verification (the only tree it mentions is the suffix-decoding depth setting) and does not list MLP speculators[S]. The SSD paper likewise says that tree-verification code paths in the vLLM version it used (0.16.0) are known to be incomplete, which is why it sweeps only the lookahead for vLLM baselines[S] [18].

P-EAGLE in vLLM. Per the project blog of March 2026, parallel drafting is available from vLLM v0.16.0 by setting `"parallel_drafting": true` with method `eagle3` and a P-EAGLE drafter; pre-trained drafters were published for GPT-OSS-120B, GPT-OSS-20B and Qwen3-Coder-30B-A3B-Instruct; implementation details include a fused Triton kernel that builds the drafter's input on the GPU, mapping rejected tokens to padding slots in the KV cache, and extending the CUDA-graph capture range by $K \times$ `max_num_seqs`[V] [53]. As noted in Chapter 9, the post warns that serving GPT-OSS-20B with EAGLE drafters needed a one-line patch pending a release and that all numbers come from one B200 and three benchmarks.

vLLM on AMD Instinct. A vLLM/AMD post (July 2026) describes an EAGLE-3 pipeline in which vLLM generates on-policy training data, supplies hidden states to the trainer and checks checkpoints by measured acceptance length; AMD trained a draft for MiniMax-M3 whose average acceptance length is 2.80 over eleven SPEED-Bench domains (coding highest at 3.32, roleplay lowest at 2.01), roughly flat from 1K to 32K context[V] [54]. Measured on MI355X with random-prompt 1K/1K microbenchmarks, Kimi K2.5 throughput rose by 1.69–2.00$\times$ and MiniMax M2.5 by 1.38–1.79$\times$ versus no speculation, with the largest gains at low concurrency. The post itself lists caveats: random-prompt throughput microbenchmarks rather than application workloads, different vLLM builds for the BF16 and FP8 sweeps, and the observation that acceptance length excludes drafting overhead so does not equal speedup[V]. EAGLE-3 *training* support on Instinct was described as planned.

12.3 SGLang and the SpecForge ecosystem

SGLang is the engine in which EAGLE-3 is most thoroughly integrated according to the sources we read, and is the baseline engine that the SSD paper found to usually outperform vLLM[S] [18]. Google Cloud's Vertex AI team describes productionising EAGLE-3 with SGLang: they credit SGLang's tree-attention kernel, which verifies branching draft trees in parallel, and its zero-overhead overlap scheduler (to which they attribute a further 10–20% speed-up, a vendor claim), report an EAGLE head of about 2–5% of the target model's size trained on target-generated data with the chat template applied and prompt tokens masked from the loss, and quote roughly 2–3× decoding speedups for Llama-70B-class models and for Llama 4 Scout 17B-16E in their benchmarks (the hardware and benchmark set-up are not stated in the post)[V] [13]. The training data pipeline takes the point of Chapter 8 that drafter quality depends on data that match the target's own output distribution.

The LMSYS *SpecBundle & SpecForge v0.2* post (December 2025) releases production-oriented EAGLE-3 draft checkpoints for twelve instruct-tuned models spanning roughly 8B to 1T parameters (including Llama 3.1 8B, Llama 3.3 70B, Llama 4 Scout, several Qwen3 and Qwen3-Coder models, Ling-flash-2.0 and Kimi-K2-Instruct), trained on a 1.4M-sample Perfect-Blend dataset with target-regenerated responses, compared with about 320K samples (ShareGPT and UltraChat) in the original EAGLE papers; it claims up to 4× end-to-end speedups over standard decoding baselines and up to 10× faster data regeneration than v0.1, without stating the hardware or batch size behind the "up to 4×"[V] [29]. The SpecForge training framework has an arXiv paper (2603.18567, ICML 2026 poster listing); we read only the listing[S] [45]. Red Hat's *Speculators* library (a Technology Preview in Red Hat AI Inference 3.5, with the vLLM project) supports only EAGLE-3 as of the page we read and is explicitly not recommended for production use[S] [42]. AWS publishes Neuron tutorials for deploying EAGLE-3 with vLLM on Trainium for Llama 3.1 8B and GPT-OSS-120B (we saw the tutorial titles in search results but did not read their contents).

DFlash reports its main serving numbers in SGLang on a single B200 (largest speedups at concurrency 1, falling at concurrency 32; Chapter 9), and the SSD paper's authors report that in their experiments SGLang usually outperforms vLLM for speculative baselines[S].

12.4 TensorRT-LLM

NVIDIA's documentation (TensorRT-LLM 1.2.0rc7) distinguishes *one-model* implementations, where the drafter is part of the target model code and the whole drafting loop can be captured in one CUDA graph (fastest at very low latency, but without dynamic draft lengths and with fewer supported model/algorithm combinations), from *two-model* implementations, where drafting runs in the PyExecutor (more flexible, slower)[S] [39]. Supported combinations as documented: one-model EAGLE-3 (Llama 4 Maverick), MTP and EAGLE-style MTP (DeepSeek-V3/R1); two-model EAGLE-3 (Llama 4 Maverick, Llama 3.1 8B, Llama 3.3 70B), draft/target with a separate model, NGram, and user-provided drafters. Documented constraints deserve attention: tree-structured drafts are not supported for EAGLE-3 (a single linear chain of fixed maximum length per request); speculation cannot be turned off dynamically, so speedups only appear at low batch size; two-model methods do not support the overlap scheduler (it is disabled automatically); *only greedy sampling is supported* (a draft token is accepted only if it equals the target's top choice); the number of MTP layers must currently equal the maximum draft length;

draft and target should share a tokenizer[S]. The restriction to greedy verification is a significant departure from the stochastic acceptance rule of Chapter 3; at nonzero temperature this would simply be a different (lossy-in-spirit or inapplicable) policy, so users who need sampling should check whether a given release supports it.

NVIDIA's technical blog on Llama 3.3 70B (December 2024 measurements; TensorRT-LLM 0.15.0, Model Optimizer 0.21) reports draft-target speculative decoding on one H200 with FP8, batch size 1, giving 51.14 tokens/s without a draft and 181.74, 161.53 and 134.38 tokens/s with Llama 3.2 1B, Llama 3.2 3B and Llama 3.1 8B drafts (3.55×, 3.16×, 2.63×); the smaller the draft, the larger the speedup, as the c-dependence of Chapter 4 predicts[V] [37]. The blog lists Medusa, EAGLE and lookahead among supported techniques but publishes numbers only for draft-target; its measurements predate EAGLE-3, and the setup (batch size one, FP8 70B on a single H200) is a best case for speculation.

12.5 llama.cpp, Ollama and edge runtimes

Speculative decoding is also reaching local runtimes. A June 2026 r/LocalLLaMA thread announced that EAGLE-3 support had been merged into llama.cpp after about six months of development, describing EAGLE-3 as "similar to MTP, but different"; the post had 229 points and we read a snapshot of 14 of 44 comments, none of which presented benchmarks, and one of which reported a roughly 38% *slowdown* with acceptance below 20% for Gemma 4 31B on an M2 Max (Chapter 16)[V] [51]. Google's Gemma 4 MTP drafters list support in Hugging Face Transformers, MLX, vLLM, SGLang, Ollama and LiteRT-LM, with on-device models using a clustered embedder to shrink the softmax[V] [12]. Unified-memory devices pose a distinct problem for the roofline argument of Chapter 5: the draft and target share one memory system and bandwidth, so the draft's bytes per token directly extend the bandwidth-bound step.

12.6 Memory accounting

Table 12.1 gives an order-of-magnitude accounting of the extra memory a drafter adds, for Llama-3.1-8B in bf16 ($d = 4096$, 32 layers, 8 KV heads of dimension 128, $|V| = 128{,}256$). The vLLM study reports that EAGLE/EAGLE-3 add under 10% to static weights and per-token KV memory, that n-gram adds nothing, and that draft-model overhead depends on the chosen draft[S] [27]; our arithmetic is consistent with this: the study measures EAGLE-3's static overhead at 5.3% and its per-token KV overhead at 4 KiB (3.1%) for Llama-3.1-8B, against our 5% and 3%.

The per-token KV cost of a standalone draft model matters at long context and large batch: a 1B draft with 16 layers and 8 KV heads of dimension 64 stores $2 \cdot 16 \cdot 8 \cdot 64 \cdot 2 = 32$ KiB per token, a quarter of the 8B target's 128 KiB, whereas an EAGLE head with a single layer stores 4 KiB. That is the arithmetic reason that, under MagicDec's long-context analysis, small draft *models* can have a memory footprint of 38–140% of the target's at long context when their KV cache is not compressed[S] [43].

Table 12.1: Approximate drafter memory for Llama-3.1-8B in bf16 ([D]; derived from model dimensions, not measured). The EAGLE-3 head assumes one decoder layer (attention $2d^2 + 2d \cdot d_{kv}$ with $d_{kv} = 1024$, MLP $3 \cdot d \cdot 14336$), a $3d \to d$ fusion projection and a 32K-vocabulary output matrix; the input embedding is assumed shared with the target. The real layer takes a concatenation of two d-dimensional vectors as input, which adds a few tens of millions of parameters that we ignore.

Component	Parameters	Weights (bf16)	KV per token
Target Llama-3.1-8B (32 layers)	≈8.0 B	≈16 GB	128 KiB
EAGLE-3 head, one layer, 32K vocab	≈0.4 B	≈0.8 GB (5%)	4 KiB (3%)
of which output matrix	134 M	0.27 GB	–
Full-vocabulary output matrix (for comparison)	525 M	1.05 GB	–
Llama-3.2-1B standalone draft	≈1.2 B	≈2.5 GB (15%)	32 KiB (25%)
n-gram / suffix	0	0	0 (CPU hash table)

12.7 An instrumentation checklist

Because published speedups vary so widely across systems, teams should measure on their own traffic. From the derivations in Chapters 4–5 and the failure modes in Chapter 15 we recommend logging at least the following (our synthesis, [D]):

1. *Acceptance length by draft position and by request class.* The mean alone hides decay with depth (Section 4.4) and heterogeneity (Proposition 4.5); the vLLM study observed variability within requests, across requests and across datasets[S].
2. *Step-time breakdown* into drafting, verification, rejection sampling and engine overhead, so that c and the verification-width cost can be estimated; the vLLM study's breakdown found verification dominating at 42–95% of the time[S].
3. *Goodput and tail latency versus load* (tokens per second *kept*, p50/p95/p99 time-between-tokens), not throughput of verified tokens; SmartSpec's definition of goodput exists for exactly this reason[S].
4. *Speculation-off A/B* at the load levels where you operate, since TensorRT-LLM cannot switch speculation off dynamically and vLLM's own documentation positions the feature for medium-to-low QPS.
5. *Output drift checks*: since batched verification kernels may differ in low-order bits, compare generation-length and quality distributions with speculation on and off; the vLLM study found generation lengths vary across batch sizes even without speculation[S].
6. *Constrained decoding paths*: one practitioner source recommends disabling speculation for grammar-constrained generation[V] [40]; test your own stack for correctness there.

12.8 Summary

- All three major open engines document speculative decoding, but with different method coverage and constraints (tree verification, sampling modes, dynamic disablement); EAGLE-3 is the most common learned drafter, and the training ecosystem (SpecForge, SpecBundle, Speculators, AMD/vLLM) is maturing around it.
- Parallel drafting (P-EAGLE, PARD) is in vLLM; SSD and DFlash are, in the sources we read, research implementations (DFlash reports SGLang numbers; SSD uses its own

engine).

- Memory overhead is small for EAGLE-style heads (our arithmetic: about 5% of weights for an 8B target) and larger for standalone drafts at long context.

Chapter 13

Consolidated Empirical Evidence

Earlier chapters reported results where they arose. This chapter pulls the evidence together, asks how far the numbers can be compared, and describes what the one independent, production-engine study of speculative decoding found. It ends with a model-based argument about whether the 2026 advances (parallel drafting and SSD) are complements or substitutes, and with a list of gaps in the evidence.

13.1 How strong is the evidence?

The sources behind the numbers in this report fall into five tiers, and the conclusions of this chapter are weighted accordingly.

1. **Independent measurement in a production engine**: one study, the MLSys 2026 oral "Speculative Decoding: Performance or Illusion?" (UC Berkeley; vLLM v0.10.1.1 on H100) [27]. It is independent in the sense that its authors did not propose any of the evaluated methods; it was supported in part by an NVIDIA gift.
2. **Author-reported measurements in research prototypes or tuned engines**: the EAGLE family, P-EAGLE, DFlash, SpecVocab, ConFu, PARD, SSD. Typically baselines are the authors' own implementations or one engine version; none of the 2026 methods has an independent replication in the sources we retrieved.
3. **Vendor and project blogs**: NVIDIA, Google (Gemma 4, Vertex AI), Together AI (ATLAS), LMSYS (SpecBundle), vLLM/AMD, LinkedIn. These contain real measurements but with incompletely stated setups; several report "up to" figures.
4. **Practitioner commentary**: blog posts and a Reddit thread, anecdotal.
5. **Our own derivations and simulations** ([D], [M]): internally consistent, but not measurements of real systems.

13.2 The independent study in detail

Liu et al. evaluate draft-model SD, EAGLE, EAGLE-3, MTP and n-gram in vLLM on Llama-3.1-8B-Instruct, Llama-3-70B-Instruct (4×H100, TP=4), Qwen3-8B (with and without thinking) and GLM-4.5-Air (106B), with all default optimisations enabled (CUDA graphs, continuous batching, chunked prefill), temperature 0, three draft tokens per step unless stated, and six workloads (CNN/DailyMail, ShareGPT, InstructCoder, GSM8K, AIME22–24, GPQA-Main)[S]. Their main findings are as follows.

1. Every variant helps, less at higher load. Across all workloads every method outperformed the no-SD baseline, with speedup decreasing as batch size grows and the effect amplified for larger targets: for Llama-3.1-8B on GSM8K EAGLE's speedup falls from 1.73× at batch

Table 13.1: Execution-time breakdown reported by Liu et al. [27] ([S]), vLLM on H100, CNN/DailyMail requests; ranges as stated in the text of the paper.

Component	Share of execution time
Verification (target forward)	42–95% across all settings; rises with model size and batch size
Drafting, n-gram	<2%
Drafting, EAGLE / EAGLE-3	12–20% at batch 1; 3–7% at batch 512
Drafting, draft model	Llama-3-70B/1B: 21% (batch 1) to 3% (batch 512); Qwen3-8B/0.6B: ≈47% to 16%
Rejection sampling	<1.7%
Engine overhead	3–12%; decreases with model and batch size

size 1 to 1.21× at 128; on ShareGPT, EAGLE's speedup falls by 4.3% (1.68× to 1.61×) from batch 1 to 32 for the 8B model and by 14% (1.96× to 1.72×) for the 70B model, which is already compute-bound at modest batch because it runs on four GPUs[S]. Reasoning workloads, with long outputs, show speedups comparable to ordinary tasks: Qwen3-8B-Thinking with EAGLE-3 achieves 1.64–1.80× on AIME22–24 and GPQA-Main versus 1.50–1.58× for n-gram; GLM-4.5-Air with its built-in MTP head achieves 1.3–1.8× on GPQA-Main[S].

2. Which drafter wins depends on the target size. Draft-model SD is best on the 70B target (draft/target step-time ratio about 12.5%, acceptance 75% on GSM8K) but loses on 8B targets: with Qwen3-0.6B drafting for Qwen3-8B the acceptance rate on GSM8K was even higher (81%) yet the speedup was substantially lower, because the draft's step is an estimated 37.5% of the target's. EAGLE/EAGLE-3 sit in between, and n-gram wins only on code editing (InstructCoder), where overlap between prompt and output is high[S]. In our notation this is exactly the α–c trade-off of Chapter 4: speedup depends on c through the relative, not absolute, draft cost.

3. Where the time goes. Verification dominates execution: 42–95% of total time across methods, rising with model size and batch size. Drafting is below 2% for n-gram, 12–20% at batch size 1 and 3–7% at batch size 512 for EAGLE and EAGLE-3, 21% (70B) or about 47% (Qwen3-8B) at batch size 1 for draft models (falling to 3% and 16% at batch size 512), sampling below 1.7%, and engine overhead 3–12%[S](Table 13.1).

4. Trees help only at low load. In SGLang (v0.5.9) with tree depth fixed at 3, tree-style EAGLE/EAGLE-3 gains a little at batch size 1 (Qwen3-8B on GSM8K: chain 1.65×, 6-node tree 1.68×, 21-node tree 1.85×; Llama-3-70B on ShareGPT: 1.81×, 1.90×, 2.03×), but the benefit vanishes quickly: by batch size 64 the 21-node tree falls below 1× on every workload for both models whereas the chain stays above 1×. Accepted length rises with tree width (2.25 → 2.51 → 2.92 on Qwen3-8B GSM8K) but the acceptance rate (accepted per verified draft token) falls (0.415 → 0.300 → 0.095)[S]. This is the experimental counterpart of the tree-versus-batch analysis of Chapter 6, where our model produced the same qualitative ordering ([M]).

5. Memory overhead is small. EAGLE adds 3.1% (EAGLE) or 5.3% (EAGLE-3) of static weights for Llama-3.1-8B and 1.4% for Llama-3-70B; per-token KV overhead is 4 KiB per token (3.1% for the 8B target, 1.3% for the 70B target). A draft model costs 1.8% extra static memory for

Table 13.2: Average generation length in tokens (mean ± s.d.) on ShareGPT, Llama-3.1-8B, by batch size ([S]; Table 1 of Liu et al. [27]). Differences are not systematically positive or negative, consistent with numerical nondeterminism.

Batch size	1	8	16	32	64	128
no SD	737 ± 1048	722 ± 994	708 ± 932	674 ± 784	687 ± 852	714 ± 930
n-gram	750 ± 1100	676 ± 810	675 ± 818	697 ± 887	696 ± 911	739 ± 1050
EAGLE	731 ± 1010	709 ± 928	686 ± 853	693 ± 857	709 ± 924	709 ± 924
EAGLE-3	736 ± 1046	704 ± 916	686 ± 853	706 ± 917	709 ± 924	709 ± 924

Llama-3-70B/Llama-3.2-1B but 7.3% for Qwen3-8B/0.6B, and raises Qwen3-8B's per-token KV from 144 to 256 KiB (1.77×)[S]. These values agree with our parametric arithmetic in Table 12.1.

6. Acceptance varies widely and the "lossless" output is not bit-identical. Acceptance length varies within a request, across requests and across datasets: n-gram has heavy-tailed, bursty acceptance (standard deviations 2–5× those of EAGLE-based methods and occasional runs above 15 tokens in code edits), EAGLE-style methods concentrate around 2–4 tokens per step, and draft models give the longest median but widely spread acceptance[S]. For Llama-3.1-8B on ShareGPT the mean generation length differs between configurations even for the same prompts and greedy decoding, with no consistent sign (Table 13.2). The authors attribute this to kernel-level nondeterminism and floating-point variation, not implementation bugs, and therefore use token throughput, not request latency, as their metric[S].

7. Overlap drives n-gram. Binning InstructCoder requests by BLEU-4 between prompt and output, n-gram's relative speedup over EAGLE or EAGLE-3 rises monotonically with overlap and exceeds it above a BLEU threshold (around 0.6 for Llama-3.1-8B) at every batch size; with proposal length 3 n-gram is up to 53% better, and with length 5 up to 100% better[S]. Our proxy experiment (Table 10.2) shows the same mechanism with a deterministic target.

8. Oracle upper bounds. An oracle that proposes exactly the number of tokens the target would accept at each step (so no verification is wasted) gives on InstructCoder with n-gram at batch size 1 about 2.75× versus about 2.1× for the best fixed proposal length of 5; the gap widens with batch size. An oracle that chooses between n-gram and EAGLE at each position adds up to a further 2.2× over the best fixed strategy, with the largest headroom on InstructCoder and little on GSM8K[S]. These are idealised bounds that assume a flawless predictor and ignore the cost of maintaining EAGLE's KV cache when it is switched on and off; the authors present them as research opportunities. The paper's introduction mentions that adaptive combinations could reach 4.9× over no SD; we did not locate the corresponding table in the extracted text and so treat that figure as an author claim[S].

13.3 Speedup versus load across sources

Figure 13.1 puts the load dependence from three independent sources on one page. The qualitative result is robust: all three show a speedup that decays with batch size (or concurrency), though by different amounts: EAGLE-3 in SGLang decays slowly (1.81× at batch 2 to 1.38× at batch 64), EAGLE in vLLM decays faster and approaches 1.2× at batch 128 on GSM8K, and the

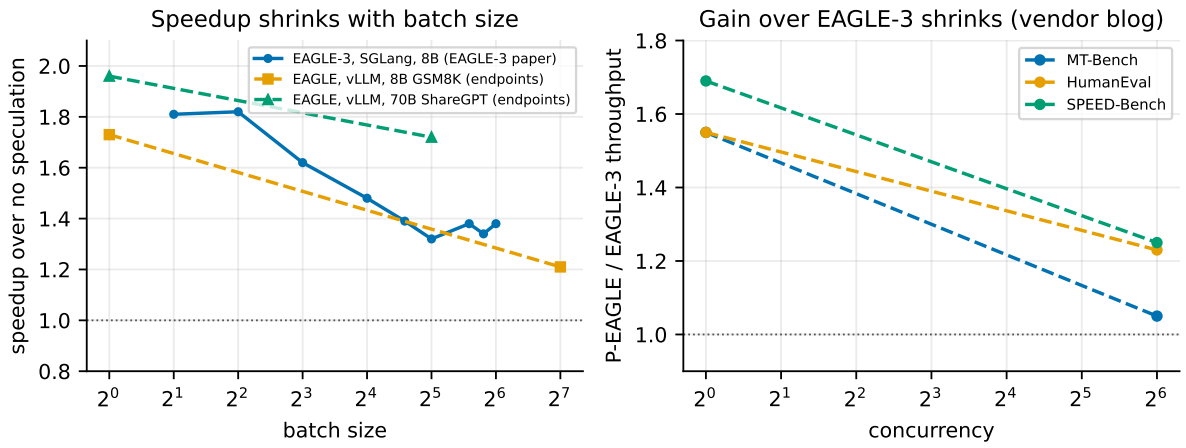


Figure 13.1: Re-plots of published numbers ([S] left; [V] right). Left: EAGLE-3 in SGLang (EAGLE-3 paper, Llama-3.1-8B, H100, MT-Bench) and the two batch-size endpoints reported in the text of the vLLM study for EAGLE (intermediate points were not retrieved, so lines between endpoints are not data). Right: ratio of P-EAGLE to EAGLE-3 throughput at concurrency 1 and 64 from the vLLM blog (GPT-OSS-20B, B200); the blog reports 1.55, 1.55 and 1.69 at $c = 1$ and 1.05, 1.23 and 1.25 at $c = 64$ for MT-Bench, HumanEval and SPEED-Bench.

advantage of P-EAGLE over EAGLE-3 shrinks from 55–69% at concurrency 1 to 5–25% at 64. The first-order explanation is the one derived in Chapter 5: when the target step crosses the ridge, verification of the extra tokens is no longer free. The second-order differences (EAGLE-3 vs. EAGLE, MoE vs. dense, TP degree) are consistent with higher acceptance prolonging the useful regime, with MoE expert-touch overhead, and with large-TP targets being compute-bound sooner.

13.4 A claims audit

Table 13.3 lists the headline claim of each major 2025–2026 method together with the information needed to interpret it, and a "normalised" figure that expresses the claim relative to the strongest same-setting comparator in the same source. The normalised column is the right quantity to compare across rows; the headline column is what appears in abstracts.

Three patterns emerge. (i) Headline numbers in abstracts compare against weaker baselines than the normalised column: 6.5× becomes 1.4× over the immediately preceding method, 5× becomes 1.27× over the strongest tuned SD. (ii) The advances of 2026 deliver gains in the range 1.04–1.4× over the preceding state of the art in their own settings. (iii) Throughput under load is rarely reported by the authors of the new methods (P-EAGLE and SSD are exceptions with partial results), and every independent source shows the effect eroding with batch size.

13.5 Parallel drafting and SSD: complements or substitutes?

Both P-EAGLE-style parallel drafting and SSD claim to remove drafting cost from the critical path. Are their gains additive? A closed-form model suggests mostly not.

Proposition 13.1 (SSD as drafting-cost scaling)**.** *Use the same drafter as primary and backup, $T_p = T_b = T \leq 1$, with unconditional hit probability p. Then* $\text{speedup}_{\text{SSD}}(T, p) = E/[1 + (1 -$

Table 13.3: Claims audit. "Headline" is as stated by the source; "Normalised" is the gain over the strongest comparator the source itself reports in the same setting ([D] when we computed the ratio). All rows batch size 1 unless stated; none independently replicated in the sources we retrieved.

Method	Headline	Setting	Strongest comparator	Normalised
EAGLE-3 (paper)	up to 6.5×	Vicuna-13B HumanEval; strict sampling; research code	EAGLE-2 (about 1.3–1.4× lower)	≈1.4× over EAGLE-2
SSD / Saguaro	up to 5× AR; "∼30% over strongest SD"	Llama-3.1-70B, 4+1 H100, own engine	tuned SGLang standalone tree 201.9 tok/s vs SSD 255.8	1.27× ([D])
P-EAGLE (paper)	1.10–1.36× (abstract; 1.04–1.36× in the detailed results)	vLLM, H200, $C \leq 4$	AR EAGLE-3 at its best K	1.04–1.36×
P-EAGLE (vLLM blog)	1.05–1.69×	B200, c=1–64, GPT-OSS-20B	EAGLE-3 at best $K = 3$	1.05–1.69× ([V])
DFlash	">6×"; up to 2.5× over EAGLE-3	Qwen3, mostly Transformers backend; some SGLang	EAGLE-3 (tree size 16)	about 2× in the Transformers-backend averages (4.9 vs 2.4×); the SGLang advantage shrinks at concurrency 32
SpecVocab	up to +8.1% over EAGLE-3	Qwen3/OLMo 2, SGLang, 1 GPU	EAGLE-3 reproduction	+4.3% on Qwen3-8B ([D])
ConFu	8–11% (Llama-3), ∼20% (Qwen3-4B)	small models, authors' implementation	EAGLE-3	as headline
PARD	up to 3.67×	Llama-3.1-8B vLLM, AMD	EAGLE-3	1.15×
Gemma-4 MTP	"up to 3×"	vendor claim; MoE 2.2× at batch 4–8 Apple silicon	none stated	unknown ([V])
ATLAS	"up to 4×" (vendor)	B200 bs 1; fully adapted	static speculator; FP8 baseline	2.65× vs standard decoding ([V])
LinkedIn n-gram	∼4× throughput (vendor)	production QPS ceiling; structured outputs	none stated	unknown ([V])

$p)T] = \text{speedup}_{\text{SD}}\big((1-p)T\big)$*: SSD acts like SD with the drafter cost scaled by* $1-p$*. Its gain over SD is bounded by* $1+T$*, and SSD's speedup is bounded above by the ceiling* E *for every* T*.*

Proof. By Theorem 11.2 with $E_{\text{hit}} = E_{\text{miss}} = E$, $\max(1, T) = 1$: the denominator is $p + (1-p)(1+T) = 1 + (1-p)T$, which is the SD denominator with T replaced by $(1-p)T$. The ratio to $E/(1+T)$ is $(1+T)/(1+(1-p)T) \leq 1+T$, with equality at $p = 1$; and $E/(1+(1-p)T) \leq E$. □

A parallel drafter reduces T itself (from γc_1 to κc_1). Both therefore push the denominator towards 1 and the speedup towards the ceiling E, and the benefit of adding one to the other is the product of the residual drafting costs. Table 13.4 computes the speedup for $\alpha = 0.8$, $\gamma = 5$ ($E = 3.69$), single-layer pass cost $c_1 \in \{0.04, 0.08, 0.15\}$, a parallel drafter with $\kappa = 2$ and hit probabilities 0.6 and 0.9 ([D]; sim/exp12_substitutes.py). For $c_1 = 0.08$: sequential SD gives 2.64×; parallel drafting alone gives 3.18× (+21%); SSD alone ($p = 0.9$) gives 3.55× (+35%); both give 3.63×, only 2.4% above SSD alone, because the ceiling is 3.69×. Notably, SSD with $p = 0.6$ on a sequential drafter and parallel drafting alone give the same 3.18×, an instance of the scaling identity $1 - p = \kappa/\gamma = 0.4$. The argument has limits: SSD can also use its free time budget for a better drafter (raising E_{hit} above E), which parallel drafting does not; and the

Table 13.4: Modelled speedup over autoregressive decoding for sequential versus parallel ($\kappa = 2$) drafters, with and without SSD (hit probability p); $\alpha = 0.8$, $\gamma = 5$, ceiling $E = 3.69$ ([D], closed-form model, not measurements).

c_1	Drafter	Drafting time T	SD	SSD $p = 0.6$	SSD $p = 0.9$
0.04	sequential	0.20	3.07	3.42	3.62
0.04	parallel	0.08	3.42	3.57	3.66
0.08	sequential	0.40	2.64	3.18	3.55
0.08	parallel	0.16	3.18	3.47	3.63
0.15	sequential	0.75	2.11	2.84	3.43
0.15	parallel	0.30	2.84	3.29	3.58

model ignores the extra hardware that SSD uses.

13.6 Does "lossless" hold in practice?

Three pieces of evidence bear on quality. First, the algorithms are provably distribution-preserving (Chapter 3); the main EAGLE-3 evaluation uses strict acceptance and does not evaluate output quality for this reason[S]. Second, the vLLM study shows that outputs are not bit-identical to autoregressive outputs in practice (Table 13.2), consistent with vLLM's own documentation, which warns that batch size can shift log-probabilities and that the rejection sampler is lossless "subject to hardware numerical precision"[S] [52]. Third, several widely deployed modes are *not* distribution-preserving: relaxed or "typical" acceptance (Medusa), vLLM's lossy FLy verification, and TensorRT-LLM's greedy-match-only acceptance rule. LinkedIn states that internal evaluations showed no quality degradation for n-gram speculation[V], and Google states accepted tokens match the target's greedy decoding[V]; neither is an independent evaluation. For most users the practical position is: exact mode is trustworthy up to numerical nondeterminism that exists anyway; lossy modes need task-level evaluation.

13.7 Gaps in the evidence

1. **No independent replication** of P-EAGLE, DFlash, SSD, SpecVocab or ConFu in the sources retrieved; the only reproduction we found (F22 Labs for SSD on consumer GPUs) is a secondary, small-scale blog.
2. **Large-batch, multi-user, MoE-at-scale** measurements are scarce: most headline results are batch size one. Exceptions are the vLLM study (up to batch 128 or 512 for breakdowns), P-EAGLE (concurrency up to 4 in the paper, up to 64 in the blog) and the EAGLE-3 batch tables.
3. **Cost and energy per token** are almost never reported. SSD uses an additional GPU; speculation spends extra FLOPs on rejected tokens (Chapter 5); a tokens-per-second-per-dollar or per-joule evaluation across load levels is missing from the literature we retrieved.
4. **Real traffic.** Benchmarks are curated (MT-Bench, HumanEval, GSM8K, Spec-Bench, SPEED-Bench, AIME). Distribution shift between drafter training data and production traffic is discussed only in vendor sources (ATLAS; Vertex), without numbers.

5. **Joint benchmarking under the same engine and hardware** of EAGLE-3, P-EAGLE, DFlash, MTP and SSD does not exist; cross-paper comparison relies on ratios to different baselines.

Chapter 14

Speculative Decoding versus Other Routes to Fast Inference

Speculative decoding is one of several ways to reduce the latency or cost of generating tokens. This chapter places it among the alternatives, in particular diffusion language models (which predict many tokens in parallel by construction), asynchronous and disaggregated designs, quantisation, and the simple option of changing the batch policy. The aim is to say where each lever acts, how it interacts with speculation, and which claims in the literature can be compared.

14.1 Where each lever acts

Using the step-time model of Chapter 5, one decoding step costs

$$t = \max\{\text{bytes}/\text{bandwidth},\ \text{FLOPs}/\text{compute}\}$$

plus overheads. Levers differ in which term they reduce and in what they do to the ridge point $n^\star$, the number of tokens per weight pass at which the step becomes compute-bound.

The quantisation-ridge arithmetic. In the weights-only model, $n^\star = \eta_F F\, b/(2\eta_W W)$ for b bytes per parameter (Chapter 5 sets $b = 2$). Changing from $b = 2$ to $b = 0.5$ with the compute path left in 16 bit divides $n^\star$ by 4: the same hardware that could verify about 221 tokens per weight pass for free in 16 bit can do so for about 55 with 4-bit weights. Quantisation therefore *consumes* some of the idle compute that speculation would otherwise exploit, although the quantised AR baseline is itself faster in absolute terms. Whether speculation on top of quantisation pays off must be settled by measurement; both the NVIDIA FP8 results (3.55× for Llama-3.3-70B with a 1B draft at batch size 1 on an H200) and the AMD results with MXFP4/FP8 builds (Chapter 12) show gains at low concurrency[V].

14.2 Diffusion language models

14.2.1 What they are and what is claimed

Diffusion LLMs (dLLMs) generate text by iteratively denoising a block or sequence of masked tokens with a bidirectional (non-causal) transformer, predicting many tokens per forward pass. Inception Labs' Mercury Coder technical report presents "a new generation of commercial-scale LLMs based on diffusion", parameterised as Transformers trained to predict multiple tokens in parallel; it reports, from independent measurements by Artificial Analysis on prompts of about 1,000 input and 1,000 output tokens, throughputs of 1109 tokens/s for Mercury Coder Mini and 737 tokens/s for Mercury Coder Small on NVIDIA H100s, and says these outperform speed-optimised frontier models by up to 10× on average at comparable quality on code

benchmarks[V] [17]. The report is a company document (the throughput measurement is by a third party, but the quality comparison and framing are the vendor's), and the models and their size are not disclosed in a way that allows a like-for-like comparison with an open autoregressive model.

14.2.2 Conceptual comparison

Speculative decoding *preserves* the target model's output distribution and accelerates it. A dLLM *is* the model: its quality has to be established on its own, and there is no equivalent of the exactness theorem relative to an autoregressive model. This distinction makes direct speed comparisons problematic: a diffusion model with a given throughput and a speculative-decoded AR model with a given throughput are different models whose quality differs.

The compute cost can be compared in a simple model ([D]). Let P be the parameter count and measure the arithmetic per *emitted* token.

- Autoregressive: $2P$ FLOPs per token, one sequential pass per token.
- Chain speculation with lookahead γ and expected tokens $E[N]$: the target performs $(\gamma+1)\cdot 2P/E[N]$ FLOPs per emitted token (plus the draft's FLOPs) and $1/E[N]$ sequential passes per token. At $\alpha = 0.8$, $\gamma = 5$: $E[N] = 3.69$, so 0.27 sequential target passes per token and $1.63\times$ the FLOPs of AR.
- Block diffusion with block size B, T denoising steps per block, prefix attention served from a KV cache and all block tokens retained: about $2PT$ FLOPs per emitted token and T/B sequential passes per token. With $B = 16$ and $T = 4$ this is 0.25 passes per token and $4\times$ the FLOPs of AR.

In this accounting the two routes have similar sequential depth per token (0.27 versus 0.25 passes) while the naive diffusion cost is higher in FLOPs ($4\times$ versus $1.63\times$), which makes sense because speculation re-uses a causal model's verification of a *given* draft whereas denoising re-evaluates all B positions at each step. The comparison is very sensitive to the assumed T, B and the fraction of denoised tokens that are finalised at each step (confidence-based unmasking can reduce T), so we draw no conclusion about which route is cheaper; the distinguishing questions are (i) quality at that step count, (ii) the cost of bidirectional attention and KV-cache semantics, and (iii) batch behaviour: both consume the idle compute of a memory-bound step and both lose their advantage when the system becomes compute-bound. We are not aware of a published head-to-head under identical hardware, engine and quality constraints.

14.2.3 Diffusion as a drafter

DFlash (Chapter 9) takes the hybrid route: a small block-diffusion model drafts a block in one pass, conditioned on the target's features, and the target verifies exactly. This retains exactness (the target is an AR model) and exploits diffusion's parallelism only for the cheap component. The SSD paper lists diffusion LLMs among the alternative draft architectures that SSD could use to speed up the drafter[S] [18], and the HKUST(GZ) survey summary lists diffusion and non-autoregressive generation among open directions[V] [58].

14.3 Asynchronous and disaggregated designs

Several systems remove the draft from the critical path by hardware placement, which relates SSD to a broader family.

- **SwiftSpec** (ByteDance Seed; arXiv June 2025) redesigns speculative decoding as an asynchronous and disaggregated pipeline: draft and target run on separate devices with parallel tree generation, tree-aware KV-cache management, and fused latency-optimised kernels, so that tensor parallelism and speculation can be used simultaneously; across five model families and six datasets the authors report an average 1.75× speedup over state-of-the-art speculative decoding systems[S] [47].
- **Mirror-SD** (Apple; arXiv October 2025) launches branch-complete rollouts from early-exit signals in parallel with the target's suffix, maps draft and target to heterogeneous accelerators (GPU and NPU) and pairs this with speculative streaming so that the draft emits multiple tokens per step; on SpecBench with 14B–66B models it reports 2.8–5.8× wall-time speedups, a 30% average relative improvement over EAGLE-3[S] [3].
- **SpecBranch** (ICLR 2026) prepares parallel speculative branches with adaptive draft lengths, reporting 1.8–4.5× over AR and 50% fewer rollback tokens for poorly aligned models[S] [44].
- **PEARL** and **DSI** overlap drafting and verification with pre/post-verify or multiple verifier instances (Chapter 7) [25, 49].

All of these rely on extra devices or heterogeneous compute for their latency gains, so their cost-effectiveness should be judged per device rather than per request. This is the subject of the next section.

14.4 Per-device accounting

SSD's headline results use five GPUs for SSD versus four for AR and SD. A simple per-device normalisation of the published table (Table 11.4) gives, for Llama-3.1-70B/1B (averages over four datasets): AR $54.7/4 = 13.7$, SD $161.8/4 = 40.5$, SSD $255.8/5 = 51.2$ tokens/s per GPU, so SSD's advantage over SD shrinks from 1.58× to 1.26× per GPU; for Qwen3-32B/0.6B: AR $88.8/4 = 22.2$, SD $136.8/4 = 34.2$, SSD $203.8/5 = 40.8$ tokens/s per GPU, an advantage over SD of 1.19× per GPU ([D]). SSD therefore still wins per device at batch size one on these numbers, but the margin is substantially smaller than the raw ratio, and per-GPU throughput at batch size one is in any case a poor measure of cost because an operator who needs throughput would batch more requests on the same four GPUs instead (at which point speculation's advantage decays; Chapter 5). The paper's claim that SSD improves the throughput-latency Pareto frontier even after accounting for the extra hardware is the relevant statement, but the numerical frontier was not retrievable[S].

14.5 A decision guide

The following guide (our synthesis of the evidence in Chapters 5–13; [D]) maps regimes to options. It is a starting point for experiments, not a prescription.

14.6 Summary

- Speculation is one member of a family of ways to convert idle compute in memory-bound decoding into lower latency; diffusion models and asynchronous designs pursue the same trade with different mechanisms and different guarantees.
- Only speculative decoding (in its exact form) leaves the target distribution untouched; diffusion LLMs are different models, so speed comparisons must be paired with quality comparisons, and none that is controlled was found.
- Asynchronous designs buy latency with devices; per-device accounting reduces SSD's published advantage over SD from 1.58× to about 1.26× on the Llama setting.
- Quantisation lowers the ridge point and therefore consumes some of the headroom that speculation exploits; the net effect needs measurement.

Table 14.1: Levers for faster decoding and their interaction with speculative decoding (synthesis of earlier chapters and cited sources; [D] unless marked).

Lever	What it changes	Interaction with speculation
Speculative decoding	Fewer sequential target passes per token, at the price of extra FLOPs	Baseline of this report; exact; headroom shrinks as the step becomes compute-bound
Weight-only quantisation (e.g. 4-bit weights, 16-bit activations)	Fewer weight bytes per pass, same FLOPs	Lowers the ridge $n^\star$ in proportion to bytes per parameter: 4-bit weights give $n^\star \approx 221/4 \approx 55$ tokens, so verification of $\gamma + 1$ tokens becomes costly at smaller batch; speculation still helps at small batch
FP8 weights and activations	Half the bytes and (on H100) twice the peak FLOPs	Ridge essentially unchanged ($\times\frac{1}{2}$ bytes, $\times 2$ FLOPs); NVIDIA's measurements combine FP8 with speculation[V] [37]
Tensor parallelism (more GPUs)	Divides bytes and FLOPs per GPU, adds communication	Targets already on 4 GPUs reach the compute-bound regime at lower batch sizes per GPU (vLLM study: 70B speedup falls faster with batch than 8B)[S] [27]
KV-cache compression, sparse attention	Fewer KV bytes per step	Interacts positively at long context: MagicDec's sparse-KV drafting[S] [43]
Mixture-of-experts	Fewer active parameters per token at batch 1	Verification touches more experts per step (expected touched fraction $1 - (1 - k/E)^n$ for n tokens), eroding the benefit at small batch
Larger batches / continuous batching	Amortises weight reads over sequences	Direct competitor: batching raises throughput, speculation lowers per-request latency; the two compete for the same idle compute
Disaggregated prefill/decode	Isolates prefill compute from decode bandwidth	Orthogonal; SSD proposes an analogous disaggregation of speculation endpoints[S] [18]
Diffusion / block LMs	Different model: predicts many tokens per pass	Replaces rather than accelerates the AR model; hybrids use a diffusion drafter (DFlash)

Table 14.2: Which speculation strategy to try first, by regime (synthesis, [D]).

Regime	Try first	Why / evidence
Interactive, batch $\leq$ 4–8, dense model, long outputs	EAGLE-3 or MTP (if the model ships one); parallel drafter (P-EAGLE) if available for the model	Largest reported gains; P-EAGLE adds 55–69% over EAGLE-3 at concurrency 1 in a vendor blog[V]
Copy-heavy outputs (code editing, structured extraction, RAG answers quoting context)	n-gram or suffix decoding	Near-zero cost; best method on InstructCoder when overlap is high[S]
70B+ target with a good same-family small model	Standalone draft model with a short tree or chain	Draft model beat EAGLE on 70B in two independent sources[S]
Moderate load (batch 16–64)	Chain, short γ, adaptive length; avoid wide trees	Trees fall below $1\times$ at batch 64 in vLLM study; goodput-based adaptation (SmartSpec)[S]
High load, throughput-bound (offline batch, RL rollouts)	Usually no speculation; evaluate quantisation, batching	Verification cost dominates; speculation "generally ineffective for throughput-bound workloads"[S]
Long-context large batch	Sparse-KV self-drafting (MagicDec-style)	Speedup can grow with batch beyond a critical length[S]
Lowest possible batch-one latency with spare GPUs	Asynchronous drafting (SSD, SwiftSpec)	Hides draft latency; costs a device; gains bounded by $1 + T_{\text{SD}}$
MoE target at small batch	Benchmark before deploying	Expert-touch penalty; Gemma-4 MoE and P-EAGLE results suggest gains only at higher batch[V]

Chapter 15

Advantages, Disadvantages and Failure Modes

This chapter consolidates what the preceding chapters imply for a practitioner who must decide whether and how to deploy speculative decoding. We separate (i) properties established by proof, (ii) properties established by measurement in at least one source, and (iii) risks that follow from the mechanism but for which we found only anecdotal or no direct measurements. Evidence tags are used throughout.

15.1 Advantages

1. **Exactness (in the standard mode).** The standard acceptance/rejection scheme preserves the target distribution exactly (Theorem 3.5); our Monte-Carlo checks agree with the proof ([M], Chapter 3). This is the central difference from quantisation, distillation, pruning, or diffusion language models, whose outputs differ from the original model's[D].
2. **Large latency gains in the memory-bound regime.** Batch-size-one results from independent and author sources range from about 1.2× to above 3× over a well-optimised autoregressive baseline in production engines (vLLM study: roughly 1.2–2.0× across variants, workloads and batch sizes[S]; plain SD in the SSD paper's own engine: 1.5–3.0× over AR, i.e. 161.8/54.7 and 136.8/88.8 from Table 11.4[D]), and much larger in research prototypes (EAGLE-3 up to 6.5×[S]). Table 13.3 shows how to normalise such claims.
3. **Small memory footprint for learned heads.** EAGLE-style heads add about 3–5% static weights and 3% KV bytes per token for an 8B target (Chapter 12; vLLM study[S]).
4. **Compatibility with the rest of the stack.** It composes with quantisation (FP8 results[V]), tensor parallelism, paged attention and continuous batching in the engines that support it, and it does not require changing or retraining the target.
5. **Training-free options exist.** N-gram and suffix decoding need no training and no GPU memory, and are very effective on copy-heavy output (Chapter 10).
6. **A mature theory of design.** The formulas of Chapter 4 (speedup, optimal γ, break-even $\alpha > c$), Chapter 5 (when batching erases the gain), Chapter 6 (trees) and Chapter 11 (asynchrony) give quantitative guidance that held up in our simulations and agrees qualitatively with independent measurements.
7. **A fast-moving, healthy ecosystem.** Training frameworks, shared checkpoints and engine support appeared in 2025–2026 (Chapter 12).

15.2 Disadvantages

1. **Gains vanish under load.** The theorem of Chapter 5 and every empirical source agree that speedup falls with batch size and can become a slowdown once the target step is compute-bound. In a goodput study with an LLaMA-7B target on one A100, proposing five tokens helped at low request rates and gave no benefit beyond about 12 requests per second[S] [28].
2. **Wasted compute.** A chain with $\alpha = 0.8$ and $\gamma = 5$ spends $(\gamma + 1)/E[N] = 1.63\times$ the target FLOPs of autoregressive decoding per emitted token ([D]), and wide trees spend far more (Chapter 6). Energy and cost per token can therefore *increase* even when latency falls; we found no measurement of this.
3. **Per-target training and maintenance.** EAGLE-style and parallel drafters must be trained for each target and re-trained when the target changes; training can take hours to days (Chapter 9, Table 9.2; Vertex report of about a day initially and several days later[V]).
4. **Workload sensitivity.** Acceptance varies by position, request and dataset (vLLM study[S]); practitioner sources report production gains 40–60% below laboratory figures[V] [40], which we treat as anecdotal.
5. **Engineering complexity and constraints.** Rollback of KV cache, ragged batches, tree masks, interactions with pipeline parallelism, structured decoding and other features; documented constraints include the lack of dynamic disabling and greedy-only verification in one engine (Chapter 12).
6. **Non-identical outputs in practice.** Even in exact mode, floating-point nondeterminism changes outputs relative to autoregressive decoding (Table 13.2).
7. **Extra hardware for the most aggressive variants.** SSD and SwiftSpec use extra devices (Chapter 14).
8. **Tail behaviour.** Tokens arrive in random bursts of N per round with standard deviation about 2 tokens at $\alpha = 0.8, \gamma = 5$ (Chapter 4; [D]); mean time between tokens falls but the *pattern* of delivery is bursty, which matters for streaming interfaces that display tokens as they arrive.

15.3 A catalogue of failure modes

Table 15.1 lists failure modes, mechanisms, the evidence we found, and detection and mitigation steps. A *failure* here means that speculation is slower than baseline, produces wrong output, or breaks a guarantee; the first kind is common, the second is rare when libraries are used as documented.

Table 15.1: Failure modes of speculative decoding in serving (synthesis; evidence tags as in the text).

Failure mode	Mechanism and evidence	Detection and mitigation
Load-induced slowdown	Verification of $\gamma + 1$ tokens exceeds the ridge: modelled slowdown for $B > B_{\text{be}}$ ([D]), SmartSpec example ([S]), trees below $1\times$ at batch 64 ([S])	Monitor goodput vs. load; adaptive or disabled speculation above a batch threshold; chains not trees at load
Low acceptance from mismatch	Draft trained on different data, template or checkpoint than the served target; tokenizer or vocabulary mismatch collapses acceptance (practitioner report, [V]); P-EAGLE reports up to 25% lower acceptance on long reasoning traces for drafters trained on short sequences ([S])	Track acceptance length per traffic segment; retrain on target-generated, on-policy data; match tokenizer and chat template
Stale drafter after target update	Fine-tuning or RL changes the target's distribution; ATLAS reports acceptance in RL rollouts rising from below 10% to over 80% as the drafter adapts ([V])	Re-train or adapt drafters with the target; alarm on acceptance drops
Temperature sensitivity	Speedups are generally lower at temperature 1 than greedy: DFlash $4.1\times$ vs $4.9\times$ ([S]); flatter distributions reduce $\sum \min(p, q)$ ([D])	Evaluate at the production temperature, not only greedy
Exactness silently broken	Mismatch between sampling and verification distributions (our toy: TV 0.07–0.23[M]); lossy modes (FLy, typical acceptance, relaxed $\lambda > 1$, [M]); greedy-match-only acceptance	Use exact rejection sampling in the sampling path; unit-test the output law on a small vocabulary; label lossy modes
Numerical nondeterminism	Kernels for $(\gamma + 1)$-token verification differ from single-token kernels; generation lengths differ across batch sizes with or without SD ([S])	Compare quality distributions, not individual outputs; fix seeds only where reproducibility is required
MoE verification overhead	Different draft tokens route to different experts; expected touched fraction $1 - (1 - k/E)^n$ ([D]); Gemma-4 and P-EAGLE MoE gains shrink at batch 1 / higher concurrency ([V], [S])	Benchmark per MoE model and batch size
Constrained decoding interplay	Grammar masks must be applied consistently to p and q along the draft path ([D], below); practitioner source recommends disabling speculation on constrained paths ([V])	Mask-aware drafting; fall back to non-speculative decoding for constrained requests if unverified
KV-cache pressure and preemption	Long reasoning outputs trigger preemption once the KV cache is full; the vLLM study restricted to medium batch to avoid this ([S]); draft-model KV adds $1.77\times$ per-token memory for Qwen3-8B/0.6B ([S])	Account for draft KV in capacity planning; prefer EAGLE heads at long context

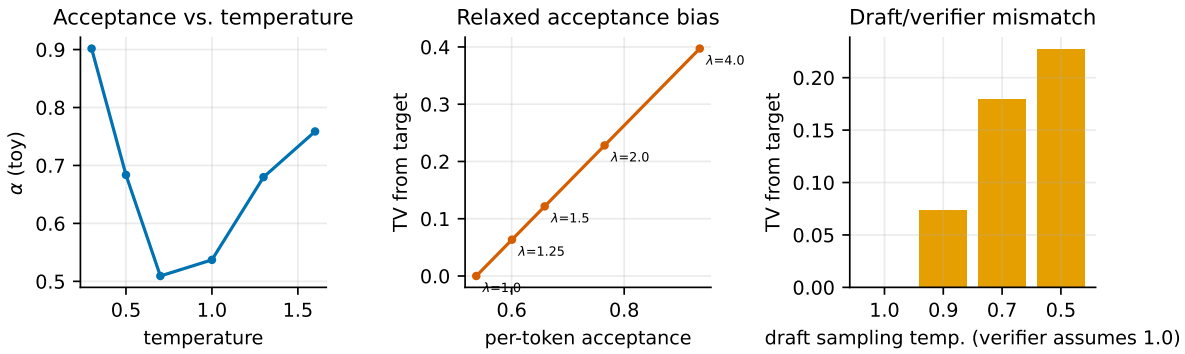


Figure 15.1: Toy failure-mode calculations ([M]). Left: acceptance versus temperature for a single noisy-copy draft (non-monotone in this toy; not a general law). Middle: relaxed acceptance $\min\{1, \lambda p/q\}$, per-token acceptance versus the total-variation distance of the emitted distribution to the target (labels are λ). Right: bias from drafting at temperature T_d while verifying with q at temperature 1.

Failure mode	Mechanism and evidence	Detection and mitigation
Wasted compute and energy	1.63× target FLOPs per token for a chain at $\alpha = 0.8$, $\gamma = 5$ ([D])	Measure tokens per joule or per dollar, not only latency
Burst delivery	Variable tokens per round (sd ≈ 2) ([D])	Smooth streaming at the gateway if UX requires
Cache misses (SSD)	Hit probability enters as p^b at batch b; misses certain at large batch and temperature ([S])	Batch-dependent fallback; restrict SSD to low batch
Operational drift	Version constraints and bugs in specific releases: pipeline parallelism incompatible in vLLM $\leq$0.15.0, one-line patch needed for GPT-OSS-20B with EAGLE drafters in March 2026, incompatible configurations in SGLang 0.5.9 / vLLM 0.16.0 sweeps ([S], [V])	Pin versions; regression-test acceptance length and throughput on upgrades

15.3.1 Illustration: sensitivity of acceptance to the sampling and verification distributions

Figure 15.1 shows three exact single-step calculations on a toy Zipfian vocabulary ([M]; sim/-exp7_failure.py; details in Chapter 3). The left panel plots the acceptance rate $\alpha = \sum \min(p, q)$ against temperature for one fixed noisy-copy draft; it is non-monotone in this single-context toy because acceptance depends on the interplay of the two distributions' peaks (for instance, whether their arg-max tokens coincide) and must not be read as a general law—the real-model evidence is only that speedups are usually lower at temperature 1 than at greedy decoding (DFlash 4.1× versus 4.9×[S]). The middle panel shows that relaxing acceptance by a factor λ buys per-token acceptance but at a total-variation distance to the target that grows roughly linearly in acceptance gain. The right panel shows the bias induced by sampling the draft at a temperature different from the one assumed in the ratio.

15.3.2 Grammar-constrained decoding: a short derivation

Suppose the serving stack samples from a constrained target $p_M(x_t \mid x_{<t}) \propto p(x_t \mid x_{<t})\mathbf{1}[x_t \in M(x_{<t})]$, where the mask M depends on the grammar state of the prefix. Speculative sampling is exact for *any* target conditional that is a function of the prefix, so it remains exact if (i) the verifier uses p_M in the ratio at every draft position, with the grammar state advanced along the draft path, and (ii) the draft's probabilities q used in the ratio are those from which the draft token was sampled. If the draft is sampled *unmasked* and the verifier uses p_M, the scheme is still exact provided q is the unmasked draft law (the ratio will then often reject ungrammatical tokens, which have $p_M = 0$), but acceptance is lower; if the draft is masked and renormalised, acceptance increases but the masked q_M must be the one in the ratio. The costs are practical: a stateful grammar must be advanced and masks computed for $\gamma + 1$ positions per round, possibly on the CPU. These considerations explain why a practitioner source recommends disabling speculation for constrained paths ([V]): the issue is engineering complexity and masking consistency, not a breakdown of the theory ([D]).

15.4 Reading the risk

Which of these risks matter most? We rank them by how often they appear in the sources and how large their effects are:

1. *Load and workload mismatch* (common, large: speedup can drop below $1\times$ or from $2\times$ to $1.2\times$).
2. *Drafter staleness and data mismatch* (common in practice, large effect on acceptance, but evidence mostly vendor-reported).
3. *Exactness misconfiguration* (rare when using library defaults, severe when it occurs, and invisible in throughput metrics).
4. *Operational drift across versions* (common, moderate).
5. *Energy and cost* (probably common, unmeasured).

15.5 Summary

Pros and cons in brief

Pros: exact in standard mode; large latency gains at small batch; tiny memory footprint for learned heads; composes with the stack; mature theory. **Cons:** gains vanish under load; wasted FLOPs; per-target training and maintenance; workload and temperature sensitivity; engineering complexity; non-identical outputs due to numerics; extra hardware for asynchronous variants. **Biggest practical risk:** deploying a configuration tuned at batch size one into a loaded system, or using a stale or mismatched drafter.

Chapter 16

Community Perception and Reception

This chapter reports what we could find about how researchers, practitioners and vendors have received the 2025–2026 developments. The evidence here is thinner and less reliable than in the technical chapters: it consists of peer-review venues, a small number of public discussion threads, practitioner blog posts, and vendor announcements. We give counts and dates where the sources provide them, paraphrase rather than quote, and avoid generalising from single comments. *Nothing in this chapter is a survey or a sentiment analysis*; it is a reading of the sources we were able to retrieve.

16.1 What we looked for and what we found

We searched for discussion of EAGLE-3, P-EAGLE, DFlash, SSD/Saguaro and speculative decoding in production on Hacker News, Reddit (r/LocalLLaMA), practitioner blogs and vendor channels. Retrieval limits matter: Hacker News pages were fetched one at a time and several item pages failed with client errors, Reddit was read through a third-party snapshot service that captured 14 of 44 comments on one thread, one practitioner article was paywalled beyond its introduction, and social-media platforms were not accessible. Table 16.1 lists the sources and how fully each was read.

16.2 Academic reception

Peer-review venues are the most reliable available signal of how the research community regards the 2026 contributions, though acceptance is not an endorsement of every claim.

- **SSD** was presented as an ICLR 2026 poster on 24 April 2026[S] [16]. The Hacker News thread notes a related 2025 paper (SpecBranch) as prior work with lower reported performance, and the SSD paper itself positions SpecBranch as approximately a special case; this is the kind of scrutiny (priority and novelty relative to concurrent work) that we expect for a paper in a crowded area.
- **P-EAGLE** is listed as a NeurIPS 2026 poster scheduled for 10 December 2026[S] [36], so as of this report it has been accepted but not yet presented.
- **"Speculative Decoding: Performance or Illusion?"** was an MLSys 2026 oral on 21 May 2026[S] [34]. An oral at a systems venue for an evaluation paper signals that the community values production-realistic measurement, and that the claim "speculative decoding is slower than advertised at realistic loads" is not controversial.
- **SpecBranch** (ICLR 2026), **SpecForge** (ICML 2026 poster listing) and **DFlash** (an ICML 2026 poster page appeared in search results) indicate that both algorithmic and infrastructure work in this area is being accepted at the top venues[S].

Table 16.1: Sources used for community perception, with extent of reading.

Source	What it is	Extent read
Hacker News item on SSD (61 points, 9 comments)	Link submission of the SSD paper; discussion thread [14]	Full page
r/LocalLLaMA "EAGLE3 has landed in llama.cpp" (229 points, 44 comments)	Feature announcement and replies [51]	14 of 44 comments (snapshot)
Tian Pan, "Speculative Decoding in Production" (Apr 2026)	Practitioner blog [40]	Full
Particula Tech, "Speculative Decoding Slower Than Baseline" (Aug 2026)	Company blog [35]	Full
Kaitchup (Dec 2025)	Newsletter on EAGLE-3 speculators [31]	Introduction only (paywall)
F22 Labs on SSD	Blog with a small reproduction [30]	Full
LinkedIn Engineering	Production report [24]	Via aggregator mirror
Google, NVIDIA, Together AI, LMSYS, vLLM/AMD	Vendor announcements	Full pages, as cited in earlier chapters
ICLR/MLSys/NeurIPS/ICML pages	Venue listings [16, 34, 36]	Listings only

- **Adoption by independent evaluation:** the vLLM-based study's finding that learned drafters are workload-sensitive and that n-gram is competitive on code editing has been echoed in subsequent practitioner posts (Section 16.3.3).

16.3 Public discussion

16.3.1 Hacker News: SSD

The thread on the SSD paper (61 points, 9 comments; the page states only that it was posted "7 months ago", which would be about March 2026) is short[V] [14]. Its themes, paraphrased: (i) one commenter explained SSD as akin to tree-based speculation that additionally precomputes drafts for different rejection points while verification runs, and noted that the paper shows how to combine the two; (ii) another asked whether the claimed speedups (taken from the abstract as up to 2× over optimised speculative decoding and 5× over autoregressive) hold on a per-FLOP basis, a question that our per-device calculation in Chapter 14 partly addresses; (iii) a commenter judged the technique most relevant at low batch size, such as local or private deployments, where decoding is memory-bound; (iv) a commenter pointed to related earlier work; (v) a commenter wondered whether frontier labs already use similar techniques and recommended implementing basic speculative decoding by hand to learn inference internals; (vi) several replies were jokes about the name. Nobody in the thread reported reproducing the results or discussed the extra hardware requirement. The thread is a weak signal: it shows interest and the expected question about efficiency, and little else.

16.3.2 r/LocalLLaMA: EAGLE-3 in llama.cpp

The thread announcing EAGLE-3 support in llama.cpp (229 points; we saw 14 of 44 comments)[V] [51] is a feature announcement that describes EAGLE-3 as similar to MTP but different. The captured comments are mostly questions: how it compares with MTP on speed and VRAM, whether it works for particular new models, and whether it beats DFlash; no benchmarks were provided in the captured portion. One commenter said it is useful for models lacking MTP; another suggested it offers little for a model family that ships with MTP; a community EAGLE-3 draft for one 27B model was mentioned without any report of whether it works; one user asked whether EAGLE-3 hurts prompt processing the way MTP reportedly does. The one detailed measurement was negative: a user reported that on an M2 Max with Gemma 4 31B speculative decoding *reduced* generation speed by about 38% with acceptance under 20%, which they attributed to unified-memory bandwidth being shared by both models. This is a single anecdotal test, but it is consistent with our roofline analysis (Chapter 5: low acceptance and non-negligible draft cost make the break-even condition $\alpha > c$ fail) and with the unified-memory caveat noted in Chapter 12.

16.3.3 Practitioner posts

"Free tokens and hidden traps" (Tian Pan, April 2026). The post argues that speculative decoding's speedups are real but depend on workload conditions that laboratory benchmarks omit, and lists traps: production results trailing laboratory ones by a wide margin (the author states 40–60% without a primary source), token throughput versus user-felt latency diverging as concurrency rises, gains shrinking or reversing at higher batch sizes, tokenizer/vocabulary mismatch collapsing acceptance, structured-output corruption in batch verification, MoE routing differences, and the operational burden of a second model (extra memory, versioning, KV synchronisation)[V] [40]. It recommends using speculation for low-concurrency (roughly batch 1–4), long, predictable outputs; starting with prompt lookup; preferring EAGLE-style drafters over independent small models; monitoring acceptance per traffic segment; and using quantisation, continuous batching and KV management for throughput-oriented work. The post is opinion by the founder of an agent-infrastructure company. Several of its specific statements (the 40–60% gap, the 4–8 concurrency threshold, "typical acceptance 0.6–0.8", "misaligned tensors in batches can produce incorrect outputs at a non-trivial rate in naive implementations") are stated without sources; we treat them as hypotheses to test, not facts. Its qualitative advice agrees with the evidence in Chapters 5 and 13.

"Slower than baseline" (Particula Tech, August 2026). A company blog that sells serving-tuning services[V] [35]. It argues that slowdowns are usually operational (batch size tuned at 1, fixed draft length, an expensive drafter, side effects such as disabled scheduler features or a smaller KV cache) rather than due to a poor draft model, and that the break-even is determined by the ratio of draft cost to target cost, not by a universal acceptance threshold; it explicitly notes that common rule-of-thumb acceptance thresholds lack primary sources. It cites numbers from the vLLM study (1.73× falling to 1.21×; verification 42–95%), and a second-hand report of a llama.cpp issue in which acceptance of 0.898 still lost to plain decoding (164.3 versus 181.3 tokens/s) for a Qwen3-4B target with a 0.6B draft (we did not read the issue). That case is consistent with the break-even $\alpha > c$ together with extra verification overhead (a draft costing more than 10% of the target's step cannot give a large speedup, and on small models the fixed

overheads are large). The post's recommendations (measure at the concurrency you serve, read per-position acceptance counters, make draft length depend on batch size) match our instrumentation checklist in Chapter 12.

Reproductions and tutorials. The F22 Labs post on SSD describes the author's small reproduction (2.74× over vLLM autoregressive on two consumer GPUs) and lists limitations: extra GPUs, higher memory, complexity, and gains shrinking at larger batch[V] [30]. The Kaitchup newsletter planned an evaluation of EAGLE-3 speculators in vLLM on Qwen3 32B contrasting saturated high-concurrency serving with batch size one, with a subtitle suggesting benefits depend on setup; the results were behind a paywall[V] [31]. Several tutorial and aggregator pages appeared in our searches (including walkthroughs of EAGLE-3 training and production use); we did not rely on them because they are secondary and often unsourced.

16.4 Vendor and platform posture

The vendor material shows a strong adoption posture: NVIDIA documents speculative decoding in TensorRT-LLM and published a "3×" headline blog; Google released MTP drafters for Gemma 4 with a "up to 3×" claim; AMD and vLLM published training and serving recipes; Red Hat added a Speculators library in technology preview; AWS published Neuron tutorials; Together AI productised an adaptive speculator; LinkedIn reported running n-gram speculation for an agent workload[V](Chapter 12). Taken together, these indicate that speculative decoding has become a standard feature of the serving stack rather than a research curiosity. Vendor material also shows how claims are framed: headline factors are "up to" figures, baselines are sometimes unstated, and internal inconsistencies occur (the Vertex post quotes both 2–3× and 2×; the ATLAS post calls a 105 → 501 tokens/s change a "400% speedup", which is a 4.8× ratio, as the post's own text acknowledges in its numbers)[V].

16.5 Recurring beliefs, checked against the evidence

Table 16.2 lists beliefs that recur in the sources and in general discussion, with what the evidence says.

16.6 Summary

- Within the research community, speculative decoding is an active, well-published field; the strongest signal of maturity is that a production-realistic evaluation paper was an MLSys oral.
- Public discussion we could retrieve is small in volume. The most informative negative reports (a 38% slowdown on Apple silicon with low acceptance, a high-acceptance llama.cpp case that lost to baseline) are anecdotal but consistent with the theory.
- The practitioner consensus we could find is that speculative decoding is valuable at low concurrency, needs workload-specific tuning, and is easy to deploy badly.
- We found no reliable measure of the share of production deployments using speculative decoding and no independent survey of practitioner satisfaction; this chapter should be

Table 16.2: Common beliefs about speculative decoding and the evidence in this report.

Belief	Assessment
"It is free: same outputs, faster"	Exact in distribution, but not free: extra FLOPs, memory, engineering, and slower or no gain under load. Outputs are not bit-identical in practice (Table 13.2).
"A higher acceptance rate means a bigger speedup"	Only through $E[N]$ and the cost c: an 81%-acceptance 0.6B drafter gave less speedup for an 8B target than a 75%-acceptance 1B drafter for a 70B target[S]; the HKUST(GZ) summary makes the same point[V].
"Use the biggest draft that fits"	Larger drafts raise α and c; the optimum drafter size is far smaller than the target (SDSL fit, with the inconsistency noted in Chapter 10).
"Works at any batch size"	No: Theorem 5.1; trees and long lookahead make it worse at load (Chapter 13).
"Tree drafts are strictly better"	Only at low batch size; the vLLM study found chains more robust[S].
"EAGLE-3 is always the best drafter"	No: draft models beat it on a 70B target in two sources; n-gram beats it on high-overlap code editing[S].
"Parallel or diffusion drafters make EAGLE-3 obsolete"	Reported gains are 1.04–1.7× over tuned EAGLE-3 (P-EAGLE) and up to about 2× in a Transformers-based comparison (DFlash); gains shrink with concurrency and have not been replicated independently.
"SSD is just PEARL/async"	Its contribution is the outcome-prediction framework and the cache, sampling and fallback analyses; its gain over SD with the same drafter is bounded by $1 + T_{\text{SD}}$ (Chapter 11).
"Speculation is only for chatbots"	Evidence of use on agent workloads (LinkedIn), reasoning (vLLM study), RL rollouts (ATLAS, survey) and agents (AgentSpec).

treated as a qualitative reading of available sources.

Chapter 17

Case Studies and Worked Examples

This chapter does two things. First, it collects deployment and research case studies from the retrieved sources and states, for each, what the source actually reports and what it does not. Second, it works through sizing examples with the formulas of Chapters 4–11; these are derivations [D], computed from the closed forms, and are not measurements.

17.1 Reported deployments

17.1.1 LinkedIn: n-gram speculation for an agent workload

LinkedIn Engineering describes speculative decoding in its Hiring Assistant agent and reports that a training-free n-gram/lookup approach was adopted, with a headline throughput improvement of about four times in that setting[V] [24]. We read the post only through an aggregator mirror, and the report is a vendor engineering blog without a public benchmark or a stated baseline configuration. It is nonetheless a useful data point for two reasons. The workload (agent turns that restate large parts of the prompt, such as structured candidate and job records) is the kind where the draft tokens can be copied from the context, which is exactly the regime where Chapter 10 and the vLLM study [27] find n-gram drafting competitive with learned drafters. And it shows an organisation choosing the zero-training option first. We do not infer that the same factor will transfer to other agent workloads.

17.1.2 Google Vertex AI with SGLang: EAGLE-3 serving

The joint Vertex/LMSYS post on serving EAGLE-3 with SGLang reports speedups in the 2–3× range in different places of the text, as noted in Chapter 16[V] [13]. We use it only as evidence that EAGLE-3 serving through SGLang is available as a managed cloud offering; the headline range is vendor-reported and the post's baseline details were not independently checked.

17.1.3 Gemma 4 multi-token-prediction drafters

Google released MTP drafters for Gemma 4 in May 2026 with an "up to 3×" speedup claim, which we read through a secondary summary[V] [12]. Two things are worth noting. A drafter shipped by the model's own developer removes the burden of training and aligning one, which Chapter 15 identifies as a main source of failure. The community report in Chapter 16 of a 38% slowdown on an Apple-silicon laptop (unified memory, acceptance under 20%) involved a Gemma 4 target, but we do not know which drafter was used, so it neither supports nor refutes the vendor claim; it only shows that results for the same model family can differ sharply with hardware and drafter.

17.1.4 AMD Instinct with vLLM

The vLLM/AMD post describes training EAGLE-3 drafters with AMD Quark tooling and serving them on Instinct GPUs through vLLM [54]. It documents that the method and the serving path are not specific to one accelerator vendor. As with the other vendor posts, we use it as evidence of availability, not of a particular speedup.

17.1.5 Together AI ATLAS: an adaptive speculator

ATLAS adapts the speculator online to the traffic of a customer [50]. The post reports a change from 105 to 501 tokens/s for one configuration, a ratio of about 4.8, which the post describes as a 400% speedup[V]. We did not verify what else differs between the two configurations, so the figure should not be read as the gain from adaptation or from speculation alone.

17.1.6 Speculative decoding for Llama at scale

Tang et al. describe the engineering needed to make speculative decoding work for very large models (Llama 4 Maverick, a mixture-of-experts model) at production batch sizes [48]. As summarised from the paper, a Maverick decode step on eight H100 GPUs takes about 4 ms per token, and the reported speedup is between about 1.4× and 2.0× even at large batch sizes[S]. The paper's contribution is the system: kernels and scheduling that keep the verification pass efficient so that a small per-step overhead does not eat the gain. This is a source-reported result on a model class (MoE) where our roofline analysis would otherwise predict weaker gains, because verifying $\gamma+1$ tokens touches more experts (Chapter 5). That the reported gain is positive at large batch makes it a counterexample to the simplified rule "speculation never helps under load"; it is not known to us what fraction of the gain depends on the hardware and kernels.

17.1.7 MoE-Spec: expert budgeting in verification

MoE-Spec addresses the specific MoE problem that verifying many draft tokens activates many experts and makes the verification step expensive. The method caps the number of experts used during verification; the abstract reports gains of roughly 10–30% over EAGLE-3 on MoE targets[S] [32]. Because the cap makes the verification model slightly different from the target, the exactness guarantee of speculative sampling no longer holds in the strict sense, a trade the authors present as an accuracy/speed knob. We have not verified the size of the accuracy effect.

17.1.8 AgentSpec: batched agents

AgentSpec targets batch inference of LLM agents, where many agent trajectories run concurrently and each has structure (tool calls, repeated formats) that can be exploited for drafting [55]. The paper implements the method on vLLM. It is relevant to the thesis of this report because it attacks the regime in which classical speculation is weakest, high concurrency, by choosing what to speculate from the agent structure and not from a single draft model. We did not extract its numbers.

Table 17.1: Speedup $S(\gamma)$ from the closed form for several (α, c) pairs [D]. The final columns give the best integer γ and the continuous optimum $\gamma^\star$.

α	c	γ=1	2	3	4	5	6	7	8	best	$\gamma^\star$
0.80	0.10	1.64	2.03	2.27	2.40	2.46	2.47	2.45	2.41	6	5.76
0.60	0.10	1.46	1.63	1.67	1.65	1.59	1.52	1.45	1.38	3	2.98
0.80	0.30	1.39	1.53	1.55	1.53	1.48	1.41	1.34	1.27	3	2.91
0.90	0.05	1.81	2.46	2.99	3.41	3.75	4.01	4.22	4.38	> 8	13.29
0.70	0.20	1.42	1.56	1.58	1.54	1.47	1.39	1.31	1.23	3	2.70

17.1.9 DeepSeek-V3 and GLM-4.5-Air: native MTP heads

Models that are trained with a multi-token-prediction head provide a drafter for free. For GLM-4.5-Air the vLLM study measures position-wise acceptance of 0.92, 0.68 and 0.38 for the first, second and third drafted tokens on GPQA-Main, and attributes the decline to reusing the single released MTP module autoregressively [27][S]. This is a good case for the product-form acceptance model: with these acceptance values the expected tokens per step under a chain of length 3 is $1 + 0.92 + 0.92 \cdot 0.68 + 0.92 \cdot 0.68 \cdot 0.38 = 2.78$ [D], close to the measured 1.3–1.8$\times$ speedups once draft overhead and imperfect scaling are included, though we do not claim this decomposition for the study.

17.2 Worked sizing examples

All numbers in this section are computed from $E[N] = (1 - \alpha^{\gamma+1})/(1 - \alpha)$ and $S = E[N]/(\gamma c + 1)$ with i.i.d. acceptance. They illustrate the behaviour of the formulas and are not predictions for any real system [D].

17.2.1 Choosing the draft length

Table 17.1 evaluates $S(\gamma)$ for five (α, c) pairs and compares the best integer γ with the continuous Lambert-W optimum of Appendix A.4.

Three observations follow. The optimum is flat: for $(0.8, 0.1)$ any γ from 5 to 7 is within 1% of the best, so precision in estimating $\gamma^\star$ is unnecessary. The optimum is sensitive to α and c jointly: reducing α from 0.8 to 0.6 at the same cost halves the useful draft length. And for a highly predictable stream ($\alpha = 0.9$, $c = 0.05$) the optimum is far beyond the lengths commonly deployed, which is where parallel drafting, which makes the cost of long drafts nearly independent of γ, is most attractive.

17.2.2 Break-even

For $\gamma = 1$ the speedup is $(1 + \alpha)/(1 + c)$, which exceeds 1 exactly when $\alpha > c$ [D]. For example, $\alpha = 0.3$ and $c = 0.4$ give $1.3/1.4 = 0.93$, a slowdown of 7%. This is the arithmetic behind the practitioner reports of slowdowns with draft models that are too large for the target or acceptance that is too low. For longer drafts the break-even acceptance is higher than c, because later draft tokens are accepted with lower probability while their cost is unchanged.

17.2.3 How much can a better drafter be worth?

Take $\gamma = 4$ and $c = 0.1$. Raising α from 0.6 to 0.8 raises $E[N]$ from 2.31 to 3.36 (a factor of 1.46) [D], and the speedup from $2.31/1.4 = 1.65$ to $3.36/1.4 = 2.40$. If the better drafter doubles c to 0.2, the speedup is $3.36/1.8 = 1.87$, still above the cheap weak drafter, but the gain falls from 46% to 13%. This is the calculation that decides whether to adopt a heavier drafter, and it shows why the head-versus-model debate cannot be settled without measuring c on the target hardware.

17.2.4 Speculative speculative decoding: when does it pay?

Under Theorem 11.2 the gain of SSD over plain SD at the same drafter is bounded by $1 + T_{\text{SD}}$ where T_{SD} is the drafting-to-verification time ratio. With $c_1 = 0.04$ and a sequential drafter, Table 11.1's companion simulation (Appendix B) gives 3.07 tokens per unit time for SD, 3.42 for SSD with cache hit probability 0.6 and 3.62 for $p = 0.9$, against a ceiling of 3.69 [M]. When drafting is already cheap relative to verification, the headroom is small; when $c_1 = 0.08$ the same sequential drafter gives 2.64, 3.18 and 3.55 [M], so the gain is much larger. The practical rule is that SSD pays when drafting is a large share of the step, and it pays with an extra device whose cost must be counted (Chapter 14).

17.2.5 A sizing checklist

1. Measure c on the deployed hardware, with the deployed batch size and context length, not from parameter counts.
2. Measure per-position acceptance on traffic that resembles production; compute $E[N]$ from it, without assuming i.i.d. acceptance.
3. Compute the break-even and the speedup at the batch sizes the service runs at; choose γ per batch regime.
4. Compare with the zero-training baselines (n-gram, prompt lookup) before training a drafter.
5. Monitor acceptance per traffic segment and retrain or disable when it drifts.

Chapter 18

Open Problems and Research Directions

The problems below are those that the evidence in this report leaves open. For each we state why it matters, what is known, and what a convincing answer would look like. They are our reading of gaps; where a gap is stated by a source, we say so.

18.1 Evidence gaps

Independent replication of the 2026 methods. We found no independent replication of P-EAGLE, DFlash, ConFu or SpecVocab, and only one small third-party reproduction of SSD (a blog post reporting 2.74× over autoregressive decoding on two consumer GPUs [30]). All headline numbers for the new methods are author-reported. A replication study that runs the released code of each method on one hardware platform, with one target set, tuned baselines, and a common serving engine, would be the most valuable single contribution to the field. The MLSys study of Liu et al. [27] is the template for how it should be done; it does not yet include the 2026 methods.

A joint benchmark under load. Table 13.3 shows that published speedups for different methods use different baselines, batch sizes, hardware, and engines. A benchmark that sweeps batch size and arrival rate for chain, tree, parallel, and SSD drafters on the same engine, and reports goodput under a latency constraint rather than raw speedup, does not exist. Spec-Bench [57] is single-request; the vLLM study is the closest to a load-aware evaluation but predates the parallel drafters.

Per-device and energy accounting. SSD uses an additional device for drafting. Whether the gain justifies the device depends on the metric (latency per request, throughput per device, energy per token). Our per-device normalisation of the published batch-size-one numbers in Chapter 14 shrinks SSD's advantage over SD from 1.58× to about 1.26× per GPU [D], but whether it survives at higher batch sizes, where an operator could instead batch more requests on the same devices, is unmeasured. No source we found reports energy per accepted token for any method.

18.2 Algorithmic questions

SSD with trees, EAGLE heads and MoE targets. The SSD paper's drafter is a separate small model on a separate device. Combining SSD's outcome-prediction cache with tree drafting (cache the subtree for each plausible outcome), with feature-conditioned drafters (the drafter needs the target's hidden state, which is not available until verification ends and so conflicts with pre-verification speculation), and with MoE targets whose verification cost grows with the number of draft tokens is untested. The feature dependency is a structural obstacle: EAGLE-style drafters consume the target's last-layer features for the verified prefix, which SSD must

predict before they exist.

Batch-robust speculation. All the evidence in Chapters 5 and 13 points to a regime change when the target becomes compute-bound. The scheduler questions are how to choose draft length and tree shape per batch (SmartSpec [28] is the main proposal), whether the draft work can be moved to idle devices or phases (as in prefill/decode disaggregation), and whether verification can be made cheaper for rejected tokens (MoE-Spec [32] does this for experts only). A method whose speedup does not collapse above batch 32 on dense models remains to be demonstrated independently.

The information limit of parallel drafting. Proposition 9.1 shows that a drafter that predicts κ positions from the same context pays a penalty equal to the conditional mutual information between the future tokens. How large this penalty is on real text, and how it varies with the position and the domain, is an empirical question that could be measured directly with a strong model and is not reported in the sources we read. The answer would tell us how much headroom parallel drafters have above current numbers (1.04–1.36$\times$ over EAGLE-3 for P-EAGLE).

Drafter staleness and online adaptation. A drafter trained on a fixed corpus loses acceptance when traffic changes or the target is updated. ATLAS [50] proposes online adaptation but has no public ablation. Open questions: how fast does acceptance decay under realistic drift, what is the cheapest update (head-only fine-tuning, vocabulary refresh), and how does adaptation interact with the exactness guarantee (it does not break it, because exactness holds for any drafter, but the drafter's training must not use information that makes outputs depend on other users' data).

Vocabulary and the LM head. The coverage bound $\alpha \leq p(K)$ of Proposition 10.1 says restricted vocabularies cap acceptance; how to choose or adapt K per domain, and whether dynamic vocabularies (SpecVocab [56]) retain their benefit under batching, are open.

Non-i.i.d. acceptance. Our closed forms assume position-independent acceptance. Real acceptance varies by position (0.92, 0.68, 0.38 for GLM-4.5-Air [27]) and by context. A theory with context-dependent acceptance and the optimal policy for adaptive draft length (a stopping problem) is largely missing; the Lambert-W result covers only the stationary case.

18.3 Systems questions

Exactness in practice. Speculative sampling is exact in distribution, but implementations differ in numerics: batched verification can change floating-point results, and some engines support only greedy verification for certain drafters (for example the TensorRT-LLM restrictions in Chapter 12). A systematic test of distributional fidelity (not just benchmark accuracy) across engines is missing.

Interaction with other optimisations. Quantisation, KV compression, sparse attention, prefix caching, and chunked prefill all change the cost ratio c and the roofline position of the target.

Their interaction with speculation is under-studied: for example, a quantised target has a lower bandwidth cost per step and therefore a smaller memory-bound window for free verification.

Reasoning, agents and RL rollouts. Long reasoning traces, tool-using agents, and RL rollout generation are the fastest growing workloads and have different statistics from chat. AgentSpec [55] and the ATLAS use for RL are early. Evidence on acceptance for long chains of thought (the vLLM study reports that acceptance grows with sequence length for n-gram) suggests an opening for retrieval-style drafters.

18.4 What would change our conclusions

We would revise the assessment in Chapter 19 if (i) an independent replication showed that a parallel drafter beats a tuned EAGLE-3 chain at batch sizes of 16 or more on dense targets; (ii) a per-device benchmark showed SSD's advantage over SD disappearing (or persisting) at batch sizes above one; or (iii) acceptance measurements on production traffic showed systematically lower values than the benchmark datasets used in the papers.

Chapter 19

Conclusions

This report examined three developments in speculative decoding between October 2025 and October 2026: EAGLE-3 as the practical standard drafter, parallel drafting (P-EAGLE and relatives), and speculative speculative decoding (SSD/Saguaro). We close with what we think is established, what is not, and what a practitioner should do.

19.1 What the evidence supports

1. **Speculative decoding is exact in distribution and its speedup follows a simple model.** The expected tokens per step, the cost ratio, and the break-even condition $\alpha > c$ [D] explain most of the successes and failures we found, from a high-acceptance configuration that still lost to baseline to the sensitivity to draft length.
2. **The benefit is a low-load phenomenon unless engineering intervenes.** The independent vLLM study reports speedups that decline with batch size and that verification time dominates (42–95%) [27][S]. Tree drafts help at batch 1 and hurt at batch 64 in that study. The one counterexample we found (Llama at scale, 1.4–2.0× at large batch on an MoE model) depends on specialised system work [48].
3. **EAGLE-3 is a strong default but not universally the best.** Draft models beat it on a 70B target in the vLLM study, and n-gram beats it on high-overlap code editing; both are cheaper to adopt.
4. **Parallel drafting trades information for latency.** Proposition 9.1 quantifies the penalty. Reported gains over EAGLE-3 are modest (about 1.04–1.36× for P-EAGLE) and author-reported.
5. **SSD is a latency-hiding technique with a provable ceiling.** Its gain over SD with the same drafter is bounded by $1 + T_{\text{SD}}$ and it requires an extra device, so its value is greatest when drafting is expensive and latency, not throughput, is the objective; per device, the published batch-one advantage over SD shrinks from 1.58× to about 1.26× [D].

19.2 What is not established

No independent replication of the 2026 methods exists in the sources we found. Production-scale numbers are mostly vendor claims. Community discussion is thin and anecdotal, and we cannot say what fraction of deployments use speculation or how often it hurts. Our own simulations are toy models whose role is to check derivations, not to estimate real speedups.

19.3 Practical guidance

- For single-user or low-concurrency serving with a model that has a released EAGLE-3 drafter or native MTP head, enable speculation, measure acceptance per position, and tune γ using Table 17.1 as a guide.
- For repetitive, context-copying workloads (code editing, structured agents), try n-gram lookup first.
- For high-concurrency throughput serving, benchmark with speculation off before assuming a benefit, and use a scheduler that can disable it or shorten drafts under load.
- Treat parallel drafters and SSD as options to evaluate, not defaults, until independent results appear.

19.4 Closing remark

The field has moved from asking whether a drafter can be made accurate to asking where the remaining latency goes: drafting cost, verification cost under load, and system overheads. The 2026 work attacks the first of these. The largest practical uncertainty is the second, and it is the one that evaluations most often omit.

Appendix A

Proofs and Derivations

This appendix collects derivations that Chapters 3 and 4 state without full proof. All are standard results re-derived here [D]; the exactness proof follows the argument of Leviathan et al. [20] and Chen et al. [6].

A.1 Exactness of speculative sampling

Fix a context and let p and q be the target and draft distributions on a finite vocabulary $\mathcal{V}$. Draw $x \sim q$, draw $u \sim U(0,1)$, and accept x if $u \le \min(1, p(x)/q(x))$. If rejected, draw x' from the residual $r(y) = (p(y) - q(y))_+ / Z$ with $Z = \sum_y (p(y) - q(y))_+$. We claim the output has distribution p.

The probability that $x = y$ is output through acceptance is $q(y)\min(1, p(y)/q(y)) = \min(p(y), q(y))$. The total acceptance probability is $\alpha = \sum_y \min(p(y), q(y))$, so the rejection probability is $1 - \alpha$. Note that

$$Z = \sum_y \big(p(y) - q(y)\big)_+ = \sum_y \big(p(y) - \min(p(y), q(y))\big) = 1 - \alpha.$$

The probability of outputting y through the residual branch is therefore $(1-\alpha)\cdot(p(y) - q(y))_+/(1-\alpha) = (p(y) - q(y))_+$. Summing the two branches,

$$\mathbb{P}[\text{output} = y] = \min(p(y), q(y)) + \big(p(y) - q(y)\big)_+ = p(y),$$

because $\min(a,b) + (a-b)_+ = a$. If $\alpha = 1$ then $p = q$ and the residual branch is never used. $\square$

For a draft of length γ, the argument is applied position by position: the first rejection at position i replaces the draft token by a residual sample, and all later draft tokens are discarded; if all γ are accepted a bonus token is drawn from the target at position $\gamma + 1$. Conditioned on the accepted prefix, each emitted token has the target conditional distribution, so by induction on the position the emitted sequence has the target joint distribution.

A.2 Expected tokens per step

Suppose each draft token is accepted independently with probability α given that the previous ones were accepted. Let N be the number of tokens emitted in one step (accepted draft tokens plus one token from the residual or bonus sample). Then $\mathbb{P}[N \ge k+1] = \alpha^k$ for $k = 0, \dots, \gamma$, and

$$E[N] = \sum_{k=0}^{\gamma} \mathbb{P}[N \ge k+1] = \sum_{k=0}^{\gamma} \alpha^k = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}.$$

With a draft step costing c target steps and a verification step costing one, the time per round is $\gamma c + 1$ in target-step units, so the speedup over autoregressive decoding (one token per unit time) is $S = E[N]/(\gamma c + 1)$.

A.3 Break-even for $\gamma = 1$

With $\gamma = 1$, $S = (1+\alpha)/(1+c)$, so $S > 1$ iff $\alpha > c$. For general γ the condition is $E[N] > \gamma c + 1$, i.e. $\sum_{k=1}^{\gamma} \alpha^k > \gamma c$, which says the mean of the first γ geometric terms α^k must exceed c. Since $\alpha^k \leq \alpha$, the mean is below α, so the break-even acceptance for $\gamma > 1$ is above c.

A.4 Optimal draft length via the Lambert W function

Treat γ as continuous. Let $s = \gamma + 1/c$, so that $\gamma c + 1 = cs$ and

$$S(\gamma) = \frac{1 - \alpha^{\gamma+1}}{(1-\alpha)\, c\, s}.$$

Differentiating, $S'(\gamma) = 0$ is equivalent to $-\alpha^{\gamma+1} \ln \alpha \cdot s - (1 - \alpha^{\gamma+1}) = 0$, that is,

$$\alpha^{\gamma+1}\big(1 - s \ln \alpha\big) = 1. \tag{A.1}$$

Put $y = -s \ln \alpha > 0$. Since $\gamma + 1 = s - 1/c + 1$, we have $\alpha^{\gamma+1} = \alpha^s \alpha^{1-1/c} = e^{-y} \alpha^{1-1/c}$, and (A.1) becomes $(1+y)e^{-y} = \alpha^{1/c-1} =: \kappa$. Multiplying by e^{-1},

$$\big(-(1+y)\big)\, e^{-(1+y)} = -\kappa/e.$$

So $w = -(1+y)$ solves $we^w = -\kappa/e$, i.e. $w = W(-\kappa/e)$. For $c < 1$ and $0 < \alpha < 1$ we have $0 < \kappa < 1$, so the argument lies in $(-1/e, 0)$ where W has two real branches. Since $y > 0$ means $w < -1$, the relevant branch is W_{-1}. Hence $y = -1 - W_{-1}(-\kappa/e)$, $s = y/(-\ln \alpha)$, and

$$\gamma^\star = \frac{-1 - W_{-1}\big(-\alpha^{1/c-1}/e\big)}{-\ln \alpha} - \frac{1}{c}.$$

The stationary point is a maximum: $S(0) = 1$, $S(\gamma) \to 0$ as $\gamma \to \infty$ for $c > 0$ (the numerator is bounded by $1/(1-\alpha)$ while the denominator grows), and (A.1) has a single root for $y > 0$ because $(1+y)e^{-y}$ is strictly decreasing there. When $\gamma^\star \leq 0$ (for instance when $\alpha \leq c$) the optimum is not to speculate. Numerical values are in Table 17.1; they agree with direct enumeration of integer γ (Appendix B, experiment 9).

A.5 Jensen effect of heterogeneous acceptance

Let α vary across steps with distribution μ and let $f(\alpha) = (1 - \alpha^{\gamma+1})/(1-\alpha) = \sum_{k=0}^{\gamma} \alpha^k$. Each term α^k is convex in α for $k \geq 2$, so f is convex on $[0,1]$ for $\gamma \geq 2$, and Jensen's inequality gives $E_\mu[f(\alpha)] \geq f(E_\mu[\alpha])$. Thus the mean tokens per step under heterogeneous acceptance is at least the value computed from the mean acceptance. The speedup computed from an average acceptance rate therefore understates the per-step expected token count, but the actual wall-time effect also depends on how the batch is scheduled (a rejected step still costs the full verification), so this is a statement about $E[N]$ only.

A.6 Variance of tokens per step

With $N = 1 + \min(G, \gamma)$ where G is geometric (number of consecutive acceptances), $\mathbb{P}[N \geq k+1] = \alpha^k$, $E[N^2] = \sum_{k=0}^{\gamma}(2k+1)\alpha^k$ and $\mathrm{Var}(N) = E[N^2] - E[N]^2$. This is the variance reported in experiment 9 and explains why individual steps fluctuate widely around the mean even when the mean speedup is moderate.

Appendix B

Simulation Code and Results

All simulations are ours, written in Python with NumPy, SciPy and Matplotlib, and are included in full below so that every [M] and [D] number in the report can be regenerated. They are toy or modelled: Experiments 1, 4, 7 and 10 use synthetic distributions; Experiments 3 and 5 use modelling assumptions stated in the text (hardware efficiency parameters, timing models); Experiment 11 uses real text but treats the text itself as the target, so it measures how copyable the text is and not an LLM's behaviour. None of these is a measurement of a real inference system. The shared helpers are in `common.py`; result files are JSON under `sim/results`.

B.1 Summary of experiments

Script	Purpose
`exp0_walkthrough.py`	Experiment 0: four-token worked example.
`exp1_lossless.py`	Experiment 1: Monte-Carlo test of exactness, with deliberately broken variants.
`exp2_formulas.py`	Experiment 2: check of $E[N]$, reproduction of a published table, speedup surfaces, optimal γ.
`exp3_roofline.py`	Experiment 3: roofline model of speedup versus batch size.
`exp3b_piecewise.py`	Experiment 3b: closed-form piecewise speedup versus simulator.
`exp4_trees.py`	Experiment 4: multi-draft verification and trees versus chains.
`exp5_ssd.py`	Experiment 5: SSD timing model, fan-out and Saguaro sampling.
`exp6_parallel_draft.py`	Experiment 6: implied acceptance rates and sequential versus parallel drafting cost.
`exp7_failure.py`	Experiment 7: failure-mode demonstrations.
`exp9_extras.py`	Experiment 9: Lambert-W optimal γ, Jensen effect, variance.
`exp10_vocab.py`	Experiment 10: restricted-vocabulary toy.
`exp11_ngram.py`	Experiment 11: prompt-lookup drafting on real text.
`exp12_substitutes.py`	Experiment 12: parallel drafting versus SSD as substitutes.

B.2 Shared helpers

```
"""Shared helpers for the speculative-decoding simulations."""
import numpy as np, json, os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
ROOT = os.path.dirname(os.path.abspath(__file__))
RES = os.path.join(ROOT, "results")
FIG = os.path.join(ROOT, "..", "figs")
os.makedirs(RES, exist_ok=True); os.makedirs(FIG, exist_ok=True)
# Okabe-Ito colour-blind-safe palette
C = dict(blue="#0072B2", orange="#E69F00", green="#009E73", verm="#D55E00",
         sky="#56B4E9", pink="#CC79A7", yellow="#B8A200", grey="#555555")
```

```
CYC = [C["blue"], C["orange"], C["green"], C["verm"], C["sky"], C["pink"], C["grey"]]
plt.rcParams.update({
    "figure.dpi": 150, "savefig.dpi": 200, "font.size": 9, "axes.titlesize": 10,
    "axes.labelsize": 9, "legend.fontsize": 8, "axes.spines.top": False,
    "axes.spines.right": False, "axes.grid": True, "grid.alpha": 0.25,
    "axes.prop_cycle": matplotlib.cycler(color=CYC), "font.family": "DejaVu Sans",
})
def save(fig, name):
    fig.savefig(os.path.join(FIG, name + ".pdf"), bbox_inches="tight")
    plt.close(fig)
def dump(name, obj):
    with open(os.path.join(RES, name + ".json"), "w") as f:
        json.dump(obj, f, indent=1, default=float)
def E_tokens(alpha, g):
    alpha = np.asarray(alpha, float)
    return np.where(np.isclose(alpha, 1.0), g + 1.0, (1 - alpha ** (g + 1)) / (1 - np.minimum(alpha, 1 - 1
    e-12)))
def speedup(alpha, g, c):
    return E_tokens(alpha, g) / (g * c + 1.0)
def spec_step(p, q, g, rng, ctx_dist=None):
    """One round of speculative sampling with context-independent p, q (vectors).
    Returns list of emitted tokens (accepted drafts + one extra token)."""
    V = len(p)
    xs = rng.choice(V, size=g, p=q)
    out = []
    for x in xs:
        if rng.random() < min(1.0, p[x] / q[x]):
            out.append(int(x))
        else:
            r = np.maximum(p - q, 0); r /= r.sum()
            out.append(int(rng.choice(V, p=r)))
            return out
    out.append(int(rng.choice(V, p=p)))
    return out
```

B.3 exp0_walkthrough.py

```
"""Exp 0: the four-token worked example of Chapter 3 (Example 'walk-through')."""
import numpy as np
from common import *
p = np.array([0.50, 0.30, 0.15, 0.05]); q = np.array([0.25, 0.45, 0.20, 0.10])
rng = np.random.default_rng(0); N = 200000
cnt = np.zeros(4); acc = 0
for _ in range(N):
    x = rng.choice(4, p=q)
    if rng.random() < min(1, p[x] / q[x]): cnt[x] += 1; acc += 1
    else:
        r = np.maximum(p - q, 0); r /= r.sum(); cnt[rng.choice(4, p=r)] += 1
out = dict(alpha_theory=float(np.minimum(p, q).sum()), alpha_mc=acc / N, freq=list(cnt / N), p=list(p))
dump("exp0_walkthrough", out); print(out)
```

B.4 exp1_lossless.py

```
"""Exp 1: Monte-Carlo verification that speculative sampling reproduces the target
distribution exactly (context-dependent toy Markov LM), plus three deliberately
broken variants that must FAIL the test."""
import numpy as np
from scipy import stats
from common import *
rng = np.random.default_rng(1234)
V = 6
```

```
def rand_stoch(rng, V, conc):
    return rng.dirichlet(np.full(V, conc), size=V)
P = rand_stoch(rng, V, 0.5)                       # target transition matrix
Q = 0.6 * P + 0.4 * rand_stoch(rng, V, 0.5)       # draft: mixture -> imperfect approx
Q /= Q.sum(1, keepdims=True)
s0 = 0; L = 3; g = 3
alphas = [float(np.minimum(P[i], Q[i]).sum()) for i in range(V)]
# exact joint of first L tokens under target
exact = np.zeros((V,) * L)
for a in range(V):
    for b in range(V):
        for c_ in range(V):
            exact[a, b, c_] = P[s0, a] * P[a, b] * P[b, c_]
def run(variant, N, rng):
    counts = np.zeros((V,) * L, int)
    for _ in range(N):
        seq = []; s = s0
        while len(seq) < L:
            # draft g tokens autoregressively from Q
            xs = []; t = s
            for _i in range(g):
                x = rng.choice(V, p=Q[t]); xs.append(x); t = x
            # verify
            t = s; emitted = []; rejected = False
            for x in xs:
                p, q = P[t], Q[t]
                if variant == "no_min_accept_all": ok = True
                else: ok = rng.random() < min(1.0, p[x] / q[x])
                if ok:
                    emitted.append(x); t = x
                else:
                    if variant == "resample_from_p": r = p.copy()
                    elif variant == "resample_from_q": r = q.copy()
                    else:
                        r = np.maximum(p - q, 0); r = r / r.sum()
                    y = rng.choice(V, p=r); emitted.append(y); rejected = True; break
            if not rejected:
                emitted.append(rng.choice(V, p=P[t]))
            seq += emitted; s = seq[-1]
        counts[tuple(seq[:L])] += 1
    return counts
out = {}
N = 400000
for variant in ["correct", "resample_from_p", "resample_from_q", "no_min_accept_all"]:
    cnt = run(variant, N, np.random.default_rng(7))
    emp = cnt / N
    tv = 0.5 * np.abs(emp - exact).sum()
    # chi-square on cells with expected count >= 5
    mask = exact * N >= 5
    chi2 = (((cnt[mask] - exact[mask] * N) ** 2) / (exact[mask] * N)).sum()
    dof = mask.sum() - 1
    pval = 1 - stats.chi2.cdf(chi2, dof)
    # noise floor: TV between two independent AR samples of same size
    out[variant] = dict(N=N, TV=tv, chi2=chi2, dof=int(dof), pvalue=pval)
    print(variant, out[variant])
# AR baseline noise floor
rs = np.random.default_rng(99)
flat = exact.ravel() / exact.sum()
ar = rs.multinomial(N, flat).reshape(exact.shape) / N
out["ar_reference_TV"] = 0.5 * np.abs(ar - exact).sum()
out["alpha_per_state"] = alphas
out["cells"] = int(exact.size)
dump("exp1_lossless", out); print(out["ar_reference_TV"], alphas)
# figure
fig, ax = plt.subplots(1, 2, figsize=(7.2, 2.8))
names = ["correct", "resample_from_p", "resample_from_q", "no_min_accept_all"]
lab = ["Correct\n(residual)", "Resample\nfrom p", "Resample\nfrom q", "Always\naccept"]
tvs = [out[n]["TV"] for n in names]
```

```
ax[0].bar(range(4), tvs, color=[C["green"], C["verm"], C["verm"], C["verm"]], width=0.6)
ax[0].axhline(out["ar_reference_TV"], color=C["grey"], ls="--", lw=1)
ax[0].text(3.4, out["ar_reference_TV"], "AR sampling\nnoise floor", fontsize=7, va="bottom", ha="right")
ax[0].set_xticks(range(4)); ax[0].set_xticklabels(lab, fontsize=7); ax[0].set_ylabel("TV distance to exact
      joint (3 tokens)")
ax[0].set_yscale("log"); ax[0].set_title("Distance from target distribution")
emp = run("correct", 100000, np.random.default_rng(5)) / 100000
ax[1].scatter(exact.ravel(), emp.ravel(), s=8, color=C["blue"])
m = exact.max(); ax[1].plot([0, m], [0, m], color=C["grey"], lw=1)
ax[1].set_xlabel("exact target joint probability"); ax[1].set_ylabel("empirical (speculative sampling)")
ax[1].set_title("216 sequence probabilities (N=100k)")
fig.tight_layout(); save(fig, "exp1_lossless")
```

B.5 exp2_formulas.py

```
"""Exp 2: (a) MC check of E[tokens]=(1-a^(g+1))/(1-a); (b) reproduce Leviathan Table 1;
(c) speedup surfaces; (d) optimal gamma table; (e) non-iid example."""
import numpy as np
from common import *
rng = np.random.default_rng(2024)
out = {}
# (a) construct p,q with prescribed alpha = sum min(p,q)
def make_pq(alpha, V=10, rng=rng):
    p = rng.dirichlet(np.ones(V));
    r = rng.dirichlet(np.ones(V))
    # q = alpha-overlap by mixing: sum min(p,q) = 1 - TV. Mix q=(1-t)p + t r, TV scales linearly in t only
      approx; solve by bisection
    lo, hi = 0.0, 1.0
    for _ in range(60):
        t = (lo + hi) / 2; q = (1 - t) * p + t * r
        a = np.minimum(p, q).sum()
        if a > alpha: lo = t
        else: hi = t
    return p, q
rows = []
for alpha in [0.5, 0.7, 0.8, 0.9]:
    for g in [1, 3, 5, 8]:
        p, q = make_pq(alpha)
        a_true = float(np.minimum(p, q).sum())
        n = 60000; tot = 0
        for _ in range(n): tot += len(spec_step(p, q, g, rng))
        rows.append(dict(alpha=a_true, gamma=g, mc=tot / n, formula=float(E_tokens(a_true, g))))
out["mc_vs_formula"] = rows
err = max(abs(r["mc"] - r["formula"]) / r["formula"] for r in rows)
out["max_rel_err"] = err; print("max rel err", err)
# (b) Leviathan Table 1 (c=0): speed and op factor
tbl = []
for a, g in [(0.6, 2), (0.7, 3), (0.8, 2), (0.8, 5), (0.9, 2), (0.9, 10)]:
    e = float(E_tokens(a, g)); ops = (g + 1) / e   # c_hat = 0
    tbl.append(dict(alpha=a, gamma=g, speed=e, ops=ops))
out["leviathan_table"] = tbl
# (c) surfaces
al = np.linspace(0.3, 0.98, 120); gs = np.arange(1, 17)
fig, axs = plt.subplots(1, 3, figsize=(7.4, 2.7), sharey=True)
for ax, c in zip(axs, [0.0, 0.05, 0.2]):
    S = np.array([[float(speedup(a, g, c)) for a in al] for g in gs])
    im = ax.pcolormesh(al, gs, S, shading="auto", cmap="viridis")
    cs = ax.contour(al, gs, S, levels=[1, 2, 3, 4], colors="white", linewidths=0.7)
    ax.clabel(cs, fmt="%gx", fontsize=6)
    gstar = [gs[np.argmax([float(speedup(a, g, c)) for g in gs])] for a in al]
    ax.plot(al, gstar, color=C["orange"], lw=1.4)
    ax.set_title(f"c = {c}"); ax.set_xlabel(r"acceptance rate $\alpha$"); ax.grid(False)
axs[0].set_ylabel(r"draft length $\gamma$"); fig.colorbar(im, ax=axs, label="speedup", shrink=0.9)
save(fig, "exp2_surface")
```

```
# (d) optimal gamma table
tab = {}
for c in [0.0, 0.02, 0.05, 0.1, 0.2, 0.4]:
    tab[str(c)] = {}
    for a in [0.5, 0.6, 0.7, 0.8, 0.9, 0.95]:
        s = [float(speedup(a, g, c)) for g in range(1, 33)]
        gi = int(np.argmax(s)); tab[str(c)][str(a)] = dict(gamma=gi + 1, speedup=s[gi])
out["opt_gamma"] = tab
# speedup vs gamma curves
fig, ax = plt.subplots(1, 2, figsize=(7.2, 2.8))
g = np.arange(1, 21)
for a, col in zip([0.6, 0.7, 0.8, 0.9], CYC):
    ax[0].plot(g, speedup(a, g, 0.05), color=col, label=fr"$\alpha$={a}")
    k = int(np.argmax(speedup(a, g, 0.05))); ax[0].plot(g[k], speedup(a, g, 0.05)[k], "o", color=col)
ax[0].set_xlabel(r"draft length $\gamma$"); ax[0].set_ylabel("speedup (c=0.05)"); ax[0].legend()
ax[0].set_title("Speedup vs. draft length")
for c, col in zip([0.0, 0.05, 0.1, 0.2], CYC):
    ax[1].plot(a1, [float(speedup(a, 1 + int(np.argmax([float(speedup(a, gg, c)) for gg in range(1, 33)])),
      c)) for a in a1], color=col, label=f"c={c}")
ax[1].set_xlabel(r"$\alpha$"); ax[1].set_ylabel("best achievable speedup"); ax[1].legend(); ax[1].
    set_title(r"Envelope over $\gamma$")
fig.tight_layout(); save(fig, "exp2_curves")
# (e) non-iid: published GLM-4.5-Air MTP positional acceptance 0.92,0.68,0.38 (cumulative? treat as
    conditional per-position rates)
pos = [0.92, 0.68, 0.38]
E_non = 1.0; cum = 1.0
for a in pos: cum *= a; E_non += cum
mean_a = float(np.mean(pos)); E_iid = float(E_tokens(mean_a, 3))
out["noniid"] = dict(pos=pos, E_noniid=E_non, mean_alpha=mean_a, E_iid=E_iid)
# draft-length-vs-cost with decaying alpha: alpha_k = a0*rho^(k-1)
fig, ax = plt.subplots(figsize=(3.6, 2.7))
for rho, col in zip([1.0, 0.95, 0.9, 0.8], CYC):
    best = []
    for g in range(1, 17):
        cum, E = 1.0, 1.0
        for k in range(1, g + 1):
            cum *= 0.85 * rho ** (k - 1); E += cum
        best.append(E / (g * 0.05 + 1))
    ax.plot(range(1, 17), best, color=col, label=fr"decay $\rho$={rho}")
ax.set_xlabel(r"$\gamma$"); ax.set_ylabel("speedup (c=0.05, a0=0.85)"); ax.legend(); ax.set_title("
    Position-decaying acceptance")
save(fig, "exp2_decay")
dump("exp2_formulas", out)
print(out["leviathan_table"], out["noniid"])
```

B.6 exp3_roofline.py

```
"""Exp 3: roofline model of speculative-decoding speedup vs batch size and context length.
ALL hardware/efficiency parameters are modelling assumptions (stated in the paper)."""
import numpy as np
from common import *
HW = dict(bw=3.35e12, flops=989e12, bw_eff=0.8, flop_eff=0.6)   # H100 SXM; dense BF16 = 1979/2 (datasheet
    footnote 'with sparsity')
M = dict(P=8.03e9, L=32, H=32, dh=128, kv_tok=131072.0, hbm_free=80e9 - 16.06e9 - 4e9)
def t_target(B, m, S, hw=HW, M=M):
    byt = 2 * M["P"] + B * S * M["kv_tok"]
    fl = B * m * (2 * M["P"] + 4 * M["L"] * M["H"] * M["dh"] * S)
    return max(byt / (hw["bw"] * hw["bw_eff"]), fl / (hw["flops"] * hw["flop_eff"]))
def t_draft(B, S, frac=0.03, hw=HW, M=M, t_launch=1.5e-4):
    # one decoder-layer EAGLE-style head: ~frac of target params (+ own KV, ignored) ; one token per
    request
    Pd = frac * M["P"]
    byt = 2 * Pd + B * S * M["kv_tok"] / M["L"]
    fl = B * 2 * Pd
```

```
    return max(byt / (hw["bw"] * hw["bw_eff"]), fl / (hw["flops"] * hw["flop_eff"])) + t_launch
def sd_speedup(B, S, alpha, g):
    if g == 0: return 1.0
    ar = t_target(B, 1, S)
    sd = g * t_draft(B, S) + t_target(B, g + 1, S)
    return float(E_tokens(alpha, g)) * ar / sd
def best(B, S, alpha, gmax=12):
    v = [(sd_speedup(B, S, alpha, g), g) for g in range(0, gmax + 1)]
    return max(v)
Bs = np.array([1, 2, 4, 8, 16, 32, 64, 128, 256, 512])
out = {"params": dict(HW=HW, M=M)}
ridge_tokens = HW["flops"] * HW["flop_eff"] / (HW["bw"] * HW["bw_eff"])  # tokens-per-weight-pass at the ridge (2 FLOP/param, 2 B/param)
out["ridge_tokens"] = ridge_tokens
fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.9))
res = {}
for S, col in zip([512, 4096, 16384, 32768], CYC):
    feas = [B for B in Bs if B * S * M["kv_tok"] <= M["hbm_free"]]
    sp = [sd_speedup(B, S, 0.8, 4) for B in feas]
    ax[0].plot(feas, sp, "o-", ms=3, color=col, label=f"ctx={S}")
    res[str(S)] = dict(B=[int(b) for b in feas], speedup_a08_g4=sp,
                       best=[best(B, S, 0.8) for B in feas])
ax[0].axhline(1, color=C["grey"], lw=0.8, ls="--"); ax[0].set_xscale("log", base=2)
ax[0].set_xlabel("batch size (concurrent requests)"); ax[0].set_ylabel("throughput speedup")
ax[0].set_title(r"$\alpha$=0.8, $\gamma$=4 (fixed)"); ax[0].legend()
for a, col in zip([0.6, 0.7, 0.8, 0.9], CYC):
    S = 512
    ax[1].plot(Bs, [sd_speedup(B, S, a, 4) for B in Bs], "o-", ms=3, color=col, label=fr"$\alpha$={a}")
ax[1].axhline(1, color=C["grey"], lw=0.8, ls="--"); ax[1].set_xscale("log", base=2)
ax[1].set_xlabel("batch size"); ax[1].set_title("ctx=512, $\\gamma$=4"); ax[1].legend()
fig.tight_layout(); save(fig, "exp3_roofline")
out["by_ctx"] = res
# crossover batch sizes (first B where speedup<1) for a grid
cross = {}
for a in [0.6, 0.7, 0.8, 0.9]:
    cross[str(a)] = {}
    for g in [2, 4, 6]:
        bb = None
        for B in range(1, 1025):
            if sd_speedup(B, 512, a, g) < 1: bb = B; break
        cross[str(a)][str(g)] = bb
out["crossover_ctx512"] = cross
# adaptive gamma (SmartSpec-like) vs fixed under load
fig, ax = plt.subplots(figsize=(3.7, 2.8))
S = 1024; a = 0.75
ad = [best(B, S, a)[0] for B in Bs]
ax.plot(Bs, ad, "o-", ms=3, color=C["green"], label="adaptive $\\gamma$ (incl. 0)")
for g, col in zip([2, 5, 8], [C["blue"], C["orange"], C["verm"]]):
    ax.plot(Bs, [sd_speedup(B, S, a, g) for B in Bs], "o--", ms=2.5, color=col, label=fr"fixed $\gamma$={g}")
ax.axhline(1, color=C["grey"], lw=0.8); ax.set_xscale("log", base=2); ax.legend(fontsize=7)
ax.set_xlabel("batch size"); ax.set_ylabel("throughput speedup"); ax.set_title(r"Adaptive vs. fixed ($\alpha$=0.75, ctx=1024)")
save(fig, "exp3_adaptive")
out["adaptive"] = dict(B=[int(b) for b in Bs], best=[best(B, S, a) for B in Bs],
        fixed={str(g): [sd_speedup(B, S, a, g) for B in Bs] for g in [2, 5, 8]})
# time-per-step decomposition figure (memory vs compute)
fig, ax = plt.subplots(figsize=(3.7, 2.8))
for m, col in zip([1, 5, 9, 17], CYC):
    ax.plot(Bs, [1e3 * t_target(B, m, 512) for B in Bs], color=col, label=f"{m} tokens/req")
ax.set_xscale("log", base=2); ax.set_yscale("log"); ax.set_xlabel("batch size"); ax.set_ylabel("target step time (ms)")
ax.legend(); ax.set_title("Modelled verification step time")
save(fig, "exp3_steptime")
out["steptime_ms"] = {str(m): [1e3 * t_target(B, m, 512) for B in Bs] for m in [1, 5, 9, 17]}
dump("exp3_roofline", out)
print(ridge_tokens, cross)
for S in res: print(S, [round(x,2) for x in res[S]["speedup_a08_g4"]], res[S]["B"])
```

```
# sensitivity of crossover batch (alpha=0.8, gamma=4, ctx=512) to achievable FLOP efficiency
sens = {}
for fe in [0.6, 0.3, 0.15, 0.08]:
    HW["flop_eff"] = fe
    bb = None
    for B in range(1, 2049):
        if sd_speedup(B, 512, 0.8, 4) < 1: bb = B; break
    sens[str(fe)] = bb
HW["flop_eff"] = 0.6
out["flop_eff_sensitivity"] = sens; dump("exp3_roofline", out); print(sens)
```

B.7 exp3b_piecewise.py

```
"""Exp 3b: check the closed-form piecewise speedup (weights-only, free draft) against the roofline
    simulator with S=0, no draft cost."""
import numpy as np
import exp3_roofline as R
from common import *
a, g = 0.8, 4
E = float(E_tokens(a, g)); n = R.ridge_tokens
rows = []
for B in [1, 8, 32, 44, 64, 100, 148, 200, 221, 300, 512]:
    ar = R.t_target(B, 1, 0); sd = R.t_target(B, g + 1, 0)
    sim = E * ar / sd
    th = E if B <= n / (g + 1) else (E * n / (B * (g + 1)) if B <= n else E / (g + 1))
    rows.append(dict(B=B, sim=sim, formula=th))
res = dict(E=E, n_star=n, B_break_even=E * n / (g + 1), rows=rows)
dump("exp3b_piecewise", res); print(res)
s = "\\begin{tabular}{rrr}\\toprule\nBatch $B$ & Simulator (ctx 0, free draft) & Piecewise formula \\\\\\
    midrule\n"
for r in rows: s += f"{r['B']} & {r['sim']:.3f} & {r['formula']:.3f} \\\\\n"
s += "\\bottomrule\\end{tabular}\n"; open("../tables/exp3b.tex", "w").write(s)
```

B.8 exp4_trees.py

```
"""Exp 4: multi-draft (tree) verification: (a) losslessness + acceptance of recursive rejection sampling,
(b) best-first tree under node budget vs chain, (c) tree vs chain under the roofline cost model."""
import numpy as np, heapq
from common import *
import exp3_roofline as R3
rng = np.random.default_rng(11)
out = {}
# ---- (a) recursive rejection sampling with k i.i.d. candidates from q
V = 8
p = rng.dirichlet(np.full(V, 0.6)); q = 0.5 * p + 0.5 * rng.dirichlet(np.full(V, 0.6))
def rrs(p, q, k, rng):
    cur = p.copy(); V = len(p)
    for _ in range(k):
        x = rng.choice(V, p=q)
        if rng.random() < min(1.0, cur[x] / q[x]): return x, True
        r = np.maximum(cur - q, 0); cur = r / r.sum()
    return rng.choice(V, p=cur), False
def rrs_accept_theory(p, q, k):
    cur = p.copy(); rej = 1.0
    for _ in range(k):
        tv = 0.5 * np.abs(cur - q).sum(); rej *= tv
        r = np.maximum(cur - q, 0); cur = r / r.sum()
    return 1 - rej
a_rows = []
N = 300000
for k in [1, 2, 3, 4, 8]:
```

```
    cnt = np.zeros(V); acc = 0
    for _ in range(N):
        x, ok = rrs(p, q, k, rng); cnt[x] += 1; acc += ok
    tv = 0.5 * np.abs(cnt / N - p).sum()
    a_rows.append(dict(k=k, TV_to_p=tv, acc_mc=acc / N, acc_theory=rrs_accept_theory(p, q, k)))
out["rrs"] = a_rows; print(a_rows)
# ---- (b) best-first tree vs chain
def values(a, b, R, depth, n):
    f = [a * b ** r for r in range(R)]
    heap = [(-f[r], 1, (r,)) for r in range(R)]; heapq.heapify(heap)
    sel = [];
    while heap and len(sel) < n:
        v, d, path = heapq.heappop(heap); sel.append((-v, d, path))
        if d < depth:
            for r in range(R):
                heapq.heappush(heap, (v * f[r], d + 1, path + (r,)))
    return sel
def exp_len(sel): return 1.0 + sum(v for v, d, pth in sel)
a, b, R, depth = 0.6, 0.3, 6, 10
rows = []
for n in [1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64]:
    sel = values(a, b, R, depth, n)
    chain = 1.0 + sum(a ** j for j in range(1, n + 1)) if n <= depth else 1.0 + sum(a ** j for j in range
     (1, depth + 1))
    rows.append(dict(n=n, tree=exp_len(sel), chain=chain, maxdepth=max(d for _, d, _ in sel)))
out["tree_vs_chain"] = rows; print(rows)
# MC validation of tree expected length for n=16
sel = values(a, b, R, depth, 16); S = {p_ for _, _, p_ in sel}
def mc_len(S, trials=200000):
    f = np.array([a * b ** r for r in range(R)]); other = 1 - f.sum()
    tot = 0
    for _ in range(trials):
        node = (); L = 1
        while True:
            r = rng.choice(R + 1, p=np.append(f, other))
            if r == R: break
            nxt = node + (r,)
            if nxt in S: node = nxt; L += 1
            else: break
        tot += L
    return tot / trials
out["tree16_mc"] = mc_len(S); out["tree16_theory"] = exp_len(sel); print(out["tree16_mc"], out["
     tree16_theory"])
# ---- (c) throughput under roofline
Bs = [1, 2, 4, 8, 16, 32, 64, 128]
fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.8))
cres = {}
for n, col in zip([4, 8, 16, 32, 64], CYC):
    sel = values(a, b, R, depth, n); El = exp_len(sel); dmax = max(d for _, d, _ in sel)
    sp = []
    for B in Bs:
        ar = R3.t_target(B, 1, 512)
        w = min(n, 8)
        sd = dmax * R3.t_draft(B * w, 512) + R3.t_target(B, n + 1, 512)
        sp.append(El * ar / sd)
    chain_gamma = n if n <= 8 else None
    cres[str(n)] = dict(E=El, depth=dmax, speedup=sp)
    ax[1].plot(Bs, sp, "o-", ms=3, color=col, label=f"{n} nodes")
ax[1].axhline(1, color=C["grey"], lw=0.8, ls="--"); ax[1].set_xscale("log", base=2); ax[1].legend(fontsize
     =7)
ax[1].set_xlabel("batch size"); ax[1].set_ylabel("throughput speedup (modelled)"); ax[1].set_title("Tree
     budget vs. batch (roofline)")
n_ = [r["n"] for r in rows]
ax[0].plot(n_, [r["tree"] for r in rows], "o-", ms=3, color=C["blue"], label="best-first tree")
ax[0].plot(n_, [r["chain"] for r in rows], "s-", ms=3, color=C["orange"], label="chain")
ax[0].set_xscale("log", base=2); ax[0].set_xlabel("verified draft nodes $n$"); ax[0].set_ylabel("expected
     tokens / step"); ax[0].legend()
ax[0].set_title(r"Tree vs. chain ($a$=0.6, $b$=0.3)")
```

```
fig.tight_layout(); save(fig, "exp4_trees")
out["roofline_trees"] = cres; dump("exp4_trees", out)
print({k: [round(x, 2) for x in v["speedup"]] for k, v in cres.items()})
```

B.9 exp5_ssd.py

```
"""Exp 5: analysis/simulation of speculative speculative decoding (SSD) timing model.
All SSD numbers here are OUR simulations of the model in Kumar et al. (2026), not their measurements."""
import numpy as np
from scipy.optimize import minimize
from common import *
rng = np.random.default_rng(5)
out = {}
def ssd_speedup(p_hit, E_hit, E_miss, Tp, Tb, b=1):
    ph = p_hit ** b
    return (p_hit * E_hit + (1 - p_hit) * E_miss) / (ph * max(1.0, Tp) + (1 - ph) * (1 + Tb))
# ---- (a) stationary hit-rate formula vs simulation
rows = []
for pp, pb in [(0.9, 0.5), (0.8, 0.3), (0.7, 0.6), (0.95, 0.2), (0.6, 0.6)]:
    n = 400000; hit = True; hits = 0   # round 0 uses primary
    prev_hit = True
    for _ in range(n):
        h = rng.random() < (pp if prev_hit else pb); hits += h; prev_hit = h
    rows.append(dict(p_p=pp, p_b=pb, sim=hits / n, formula=pb / (1 + pb - pp)))
out["stationary"] = rows; print(rows)
# ---- (b) DES check of Theorem 7 (batch 1)
des = []
for (alpha, K, Tp, Tb, pp, pb) in [(0.8, 5, 0.9, 0.25, 0.9, 0.4), (0.7, 6, 1.1, 0.1, 0.8, 0.3), (0.85, 4,
     0.7, 0.3, 0.6, 0.6)]:
    # E_hit uses primary with same alpha; E_miss uses backup with alpha_b
    ab = alpha - 0.1
    def draw(a):   # tokens emitted per round
        k = 0
        while k < K and rng.random() < a: k += 1
        return k + 1
    n = 200000; tok = 0; time = 0.0; prev = True; hits = 0
    for _ in range(n):
        h = rng.random() < (pp if prev else pb)
        if h: tok += draw(alpha); time += max(1.0, Tp)
        else: tok += draw(ab); time += 1.0 + Tb
        hits += h; prev = h
    ph = pb / (1 + pb - pp)
    th = ssd_speedup(ph, float(E_tokens(alpha, K)), float(E_tokens(ab, K)), Tp, Tb)
    des.append(dict(alpha=alpha, K=K, Tp=Tp, Tb=Tb, p_hit=ph, des=tok / time, theory=th))
out["des"] = des; print(des)
# ---- (c) SSD vs SD heatmap over p_hit, T_SD   (same drafter used as backup, E_miss=E_hit=E)
alpha, K = 0.8, 5
E = float(E_tokens(alpha, K))
ph_ax = np.linspace(0.0, 1.0, 101); T_ax = np.linspace(0.02, 0.6, 100)
Z = np.zeros((len(T_ax), len(ph_ax)))
for i, T in enumerate(T_ax):
    for j, ph in enumerate(ph_ax):
        Z[i, j] = ssd_speedup(ph, E, E, T, T) / (E / (1 + T))
fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.8))
im = ax[0].pcolormesh(ph_ax, T_ax, Z, shading="auto", cmap="viridis"); ax[0].grid(False)
cs = ax[0].contour(ph_ax, T_ax, Z, levels=[1.05, 1.1, 1.2, 1.4], colors="white", linewidths=0.7); ax[0].
     clabel(cs, fmt="%.2f", fontsize=6)
ax[0].set_xlabel(r"cache-hit probability $p_{hit}$"); ax[0].set_ylabel(r"draft time $T_{SD}$ (rel. to
     verify)")
fig.colorbar(im, ax=ax[0], label="SSD / SD speedup"); ax[0].set_title("Gain over SD, same drafter")
# batch dependence & fallback
E_hit, E_miss, ph, Tp = 4.5, 1.0, 0.97, 0.5
bs = np.arange(1, 129)
slow = [ssd_speedup(ph, E_hit, E_hit, Tp, Tp, b) for b in bs]   # slow backup = primary (E_miss=E_hit)
```

```
fast = [ssd_speedup(ph, E_hit, E_miss, Tp, 0.0, b) for b in bs]
ax[1].plot(bs, slow, color=C["blue"], label="slow backup (= primary)"); ax[1].plot(bs, fast, color=C["orange"], label="fast backup (T_b=0, E_miss=1)")
# b* from paper's formula: solve numerically instead
cross = next((b for b, s, f in zip(bs, slow, fast) if f > s), None)
bstar = np.log((1 + Tp - E_hit / (ph * E_hit + (1 - ph) * E_miss)) / Tp) / np.log(ph)
ax[1].axvline(bstar, color=C["grey"], ls=":"); ax[1].set_xscale("log", base=2)
ax[1].set_xlabel("batch size $b$"); ax[1].set_ylabel("speedup vs AR (this model)"); ax[1].legend(fontsize=7)
ax[1].set_title(r"Fallback choice ($p_{hit}$=0.97, $T_p$=0.5)")
fig.tight_layout(); save(fig, "exp5_ssd_model")
out["fallback"] = dict(bstar_formula=float(bstar), crossover_numeric=cross, E_hit=E_hit, E_miss=E_miss, p_hit=ph, Tp=Tp)
print(out["fallback"])
out["worked"] = dict(alpha=alpha, K=K, E=E, T_SD=0.25, sd=E / 1.25, ssd_09=ssd_speedup(0.9, E, E, 0.25, 0.25), bound=E)
# ---- (d) fan-out allocation: geometric vs uniform vs numerical optimum
def phit_of(F, alpha, K, r):
    F = np.maximum(np.asarray(F, float), 1e-9)
    Pk = np.array([alpha ** k * (1 - alpha) for k in range(K)] + [alpha ** K])
    return float((Pk * (1 - F ** (-r))).sum())
def geometric(B, alpha, K, r):
    w = np.array([alpha ** (k / (1 + r)) for k in range(K)] + [alpha ** (K / (1 + r)) * (1 - alpha) ** (-1 / (1 + r))])
    return B * w / w.sum()
fan = []
for (alpha, K, r, B) in [(0.8, 5, 0.6, 24), (0.7, 5, 0.6, 24), (0.8, 5, 1.0, 24), (0.9, 7, 0.6, 48), (0.8, 5, 0.6, 96)]:
    Fu = np.full(K + 1, B / (K + 1)); Fg = geometric(B, alpha, K, r)
    res = minimize(lambda x: -phit_of(np.abs(x), alpha, K, r), Fu, constraints=[{"type": "eq", "fun": lambda x: np.abs(x).sum() - B}], method="SLSQP")
    fan.append(dict(alpha=alpha, K=K, r=r, B=B, uniform=phit_of(Fu, alpha, K, r), geometric=phit_of(Fg, alpha, K, r),
                    numeric=-res.fun, F_geo=list(Fg), F_num=list(np.abs(res.x))))
out["fanout"] = fan; print([(f["uniform"], f["geometric"], f["numeric"]) for f in fan])
fig, ax = plt.subplots(1, 2, figsize=(7.2, 2.7))
f0 = fan[0]; xs = np.arange(f0["K"] + 1)
ax[0].bar(xs - 0.2, [f0["B"] / (f0["K"] + 1)] * (f0["K"] + 1), 0.4, color=C["sky"], label="uniform")
ax[0].bar(xs + 0.2, f0["F_geo"], 0.4, color=C["blue"], label="geometric (Thm.)")
ax[0].plot(xs, f0["F_num"], "o", color=C["verm"], ms=4, label="numerical optimum")
ax[0].set_xlabel("accepted tokens $k$"); ax[0].set_ylabel("fan-out $F_k$"); ax[0].legend(fontsize=7); ax[0].set_title(r"Budget $B$=24, $\alpha$=0.8, $K$=5, $r$=0.6")
Bs_ = np.array([6, 12, 24, 48, 96, 192])
for r, col in zip([0.4, 0.6, 1.0], CYC):
    ax[1].plot(Bs_, [phit_of(geometric(B, 0.8, 5, r), 0.8, 5, r) for B in Bs_], "o-", ms=3, color=col, label=f"geometric r={r}")
    ax[1].plot(Bs_, [phit_of(np.full(6, B / 6), 0.8, 5, r) for B in Bs_], "x--", ms=3, color=col, alpha=0.7)
ax[1].set_xscale("log", base=2); ax[1].set_xlabel("cache budget $B$"); ax[1].set_ylabel(r"$p_{hit}$"); ax[1].legend(fontsize=7)
ax[1].set_title("solid: geometric, dashed: uniform")
fig.tight_layout(); save(fig, "exp5_fanout")
# ---- (e) Saguaro-sampling toy
V = 2000
ranks = np.arange(1, V + 1)
lp = -1.1 * np.log(ranks) ; rg = np.random.default_rng(3)
zt = lp + rg.normal(0, 0.0, V)                  # target logits (Zipf)
zd = zt + rg.normal(0, 0.9, V)                  # draft logits: noisy copy
p = np.exp(zt - zt.max()); p /= p.sum()
qraw = np.exp(zd - zd.max()); qraw /= qraw.sum()
def metrics(C_, F, K=5):
    top = np.argsort(-zd)[:F]
    w = qraw.copy(); w[top] *= C_; q = w / w.sum()
    alpha = np.minimum(p, q).sum()
    r = np.maximum(p - q, 0); r /= r.sum()
    h_rej = r[top].sum(); h_full = p[top].sum()
    Pk = np.array([alpha ** k * (1 - alpha) for k in range(K)] + [alpha ** K])
    hk = np.array([h_rej] * K + [h_full])
```

```
    ph = float((Pk * hk).sum())
    return alpha, h_rej, ph
Cs = np.array([1.0, 0.8, 0.6, 0.4, 0.25, 0.1, 0.05, 0.0])
sag = []
for F in [4, 16]:
    for C_ in Cs:
        a_, hr, ph = metrics(C_, F); K = 5
        s_fast = ssd_speedup(ph, float(E_tokens(a_, K)), 1.0, 1.0, 0.0)
        sag.append(dict(F=F, C=float(C_), alpha=float(a_), h_rej=float(hr), p_hit=ph, speedup_fastbackup=
     float(s_fast)))
out["saguaro_toy"] = sag
fig, ax = plt.subplots(1, 3, figsize=(7.4, 2.5))
for F, col in zip([4, 16], [C["blue"], C["orange"]]):
    sub = [s for s in sag if s["F"] == F]
    ax[0].plot([s["C"] for s in sub], [s["alpha"] for s in sub], "o-", ms=3, color=col, label=f"F={F}")
    ax[1].plot([s["C"] for s in sub], [s["p_hit"] for s in sub], "o-", ms=3, color=col)
    ax[2].plot([s["C"] for s in sub], [s["speedup_fastbackup"] for s in sub], "o-", ms=3, color=col)
for a_, t in zip(ax, [r"acceptance $\alpha$", r"round hit rate $p_{hit}$", "modelled SSD speedup"]):
    a_.set_xlabel("downweight $C$"); a_.set_title(t); a_.invert_xaxis()
ax[0].legend(fontsize=7); fig.tight_layout(); save(fig, "exp5_saguaro_sampling")
dump("exp5_ssd", out)
for s in sag: print(s)
```

B.10 exp6_parallel_draft.py

```
"""Exp 6: implied acceptance rates from published acceptance lengths and an illustrative
draft-cost model contrasting sequential (K passes) and parallel (1 pass) drafting."""
import numpy as np
from scipy.optimize import brentq
from common import *
out = {}
def implied_alpha(AL, K):
    f = lambda a: float(E_tokens(a, K)) - AL
    return brentq(f, 1e-6, 0.999999)
pub = [("P-EAGLE blog (B200, GPT-OSS-20B) K=7", "HumanEval", 3.94, 7, "P-EAGLE"), ("P-EAGLE blog K=7", "
     HumanEval", 3.03, 7, "EAGLE-3"),
       ("P-EAGLE blog K=7", "SPEED-Bench", 3.38, 7, "P-EAGLE"), ("P-EAGLE blog K=7", "SPEED-Bench", 2.59,
     7, "EAGLE-3"),
       ("P-EAGLE blog K=7", "MT-Bench", 3.70, 7, "P-EAGLE"), ("P-EAGLE blog K=7", "MT-Bench", 3.27, 7, "
     EAGLE-3"),
       ("P-EAGLE paper K=5 (Qwen3-Coder-30B)", "HumanEval", 4.5, 5, "P-EAGLE"), ("P-EAGLE paper K=5 (Qwen3-
     Coder-30B)", "HumanEval", 4.4, 5, "EAGLE-3")]
rows = []
for src, ds, AL, K, m in pub:
    rows.append(dict(src=src, ds=ds, AL=AL, K=K, method=m, alpha=implied_alpha(AL, K)))
out["implied"] = rows
for r in rows: print(r)
# cost model
c1 = 0.04
fig, ax = plt.subplots(1, 2, figsize=(7.2, 2.8))
a = 0.78
Ks = np.arange(1, 13)
seq = [float(E_tokens(a, K)) / (1 + K * c1) for K in Ks]
ax[0].plot(Ks, seq, "o-", ms=3, color=C["blue"], label="sequential (cost $K c_1$)")
res = {"seq": dict(best_K=int(Ks[np.argmax(seq)]), best=max(seq))}
for kappa, col in zip([1.5, 2.5, 4.0], [C["orange"], C["green"], C["verm"]]):
    # parallel drafter: one pass, cost kappa*c1 ; acceptance decays slightly vs sequential at depth (rho)
    for rho in [0.97]:
        vals = []
        for K in Ks:
            cum, E = 1.0, 1.0
            for k in range(1, K + 1):
                cum *= a * rho ** (k - 1); E += cum
            vals.append(E / (1 + kappa * c1))
```

```
        ax[0].plot(Ks, vals, "s--", ms=3, color=col, label=fr"parallel, $\kappa$={kappa}")
        res[f"par_{kappa}"] = dict(best_K=int(Ks[np.argmax(vals)]), best=max(vals))
ax[0].set_xlabel("speculation depth $K$"); ax[0].set_ylabel("modelled speedup (low batch)"); ax[0].legend(fontsize=6.5)
ax[0].set_title(r"$\alpha$=0.78, $c_1$=0.04, parallel decay $\rho$=0.97")
out["cost_model"] = res; print(res)
# right: ratio vs c1
c1s = np.linspace(0.01, 0.15, 30)
for kappa, col in zip([1.5, 2.5, 4.0], [C["orange"], C["green"], C["verm"]]):
    r = []
    for c in c1s:
        s = max(float(E_tokens(a, K)) / (1 + K * c) for K in range(1, 13))
        p = max((1 + sum(np.prod([a * 0.97 ** (j - 1) for j in range(1, k + 1)]) for k in range(1, K + 1))) / (1 + kappa * c) for K in range(1, 13))
        r.append(p / s)
    ax[1].plot(c1s, r, color=col, label=fr"$\kappa$={kappa}")
ax[1].axhline(1, color=C["grey"], lw=0.8, ls="--"); ax[1].set_xlabel("single-layer draft pass cost $c_1$"); ax[1].set_ylabel("parallel / sequential speedup"); ax[1].legend(fontsize=7)
ax[1].set_title("When parallel drafting pays off")
fig.tight_layout(); save(fig, "exp6_parallel")
dump("exp6_parallel", out)
```

B.11 exp7_failure.py

```
"""Exp 7: failure-mode demonstrations (exact calculations, single verification step)."""
import numpy as np
from common import *
out = {}
V = 2000; ranks = np.arange(1, V + 1)
zt = -1.1 * np.log(ranks); rg = np.random.default_rng(3); zd = zt + rg.normal(0, 0.9, V)
def sm(z, T):
    e = np.exp((z - z.max()) / T); return e / e.sum()
def step_dist(p, q_sample, q_used, lam=1.0):
    """Distribution of emitted token: draft x~q_sample, accept w.p. min(1, lam*p/q_used), else residual of (p,q_used)."""
    acc = np.minimum(1.0, lam * p / q_used)
    A = (q_sample * acc).sum()
    r = np.maximum(p - q_used, 0); r = r / r.sum()
    return q_sample * acc + (1 - A) * r, A
tv = lambda a, b: 0.5 * np.abs(a - b).sum()
# alpha vs temperature
Ts = [0.3, 0.5, 0.7, 1.0, 1.3, 1.6]
al = []
for T in Ts:
    p, q = sm(zt, T), sm(zd, T); al.append(float(np.minimum(p, q).sum()))
out["alpha_vs_T"] = dict(T=Ts, alpha=al)
# relaxed acceptance
p, q = sm(zt, 1), sm(zd, 1); rel = []
for lam in [1.0, 1.25, 1.5, 2.0, 4.0]:
    d, A = step_dist(p, q, q, lam); rel.append(dict(lam=lam, acc=float(A), TV=float(tv(d, p)), E_tokens_g4=float(E_tokens(A, 4))))
out["relaxed"] = rel
# stale/mismatched draft probabilities
mm = []
for Td in [1.0, 0.9, 0.7, 0.5]:
    qs = sm(zd, Td); d, A = step_dist(p, qs, q); mm.append(dict(T_draft=Td, acc=float(A), TV=float(tv(d, p))))
out["mismatch"] = mm
# draft/target tied to temperature but wrong residual normalisation (top-k truncation of draft but untruncated q used)
k = 50; qt = q.copy(); qt[np.argsort(-q)[k:]] = 0; qt /= qt.sum()
d, A = step_dist(p, qt, q); out["topk_mismatch"] = dict(k=k, acc=float(A), TV=float(tv(d, p)))
d2, A2 = step_dist(p, qt, qt); out["topk_correct"] = dict(k=k, acc=float(A2), TV=float(tv(d2, p)))
dump("exp7_failure", out)
```

```
for k_, v in out.items(): print(k_, v)
fig, ax = plt.subplots(1, 3, figsize=(7.4, 2.5))
ax[0].plot(Ts, al, "o-", ms=3, color=C["blue"]); ax[0].set_xlabel("temperature"); ax[0].set_ylabel(r"$\alpha$ (toy)"); ax[0].set_title("Acceptance vs. temperature")
ax[1].plot([r["acc"] for r in rel], [r["TV"] for r in rel], "o-", ms=3, color=C["verm"])
for r in rel: ax[1].annotate(fr"$\lambda$={r['lam']}", (r["acc"], r["TV"]), fontsize=6, xytext=(3, -8), textcoords="offset points")
ax[1].set_xlabel(r"per-token acceptance"); ax[1].set_ylabel("TV from target"); ax[1].set_title("Relaxed acceptance bias")
ax[2].bar(range(4), [m["TV"] for m in mm], color=C["orange"]); ax[2].set_xticks(range(4)); ax[2].set_xticklabels([f"{m['T_draft']}" for m in mm])
ax[2].set_xlabel("draft sampling temp. (verifier assumes 1.0)"); ax[2].set_ylabel("TV from target"); ax[2].set_title("Draft/verifier mismatch")
fig.tight_layout(); save(fig, "exp7_failure")
```

B.12 exp9_extras.py

```
"""Exp 9: closed-form optimal gamma via Lambert W, Jensen effect of heterogeneous alpha, variance of tokens/round."""
import numpy as np
from scipy.special import lambertw
from common import *
out = {}
def gstar(a, c):
    kappa = a ** (1 / c - 1)
    w = lambertw(-kappa / np.e, k=-1).real
    y = -1 - w
    s = y / (-np.log(a))
    return s - 1 / c
rows = []
for a in [0.6, 0.7, 0.8, 0.9]:
    for c in [0.02, 0.05, 0.1, 0.2]:
        gc = gstar(a, c)
        grid = [float(speedup(a, g, c)) for g in range(1, 200)]
        gi = int(np.argmax(grid)) + 1
        rows.append(dict(alpha=a, c=c, gamma_cont=float(gc), gamma_grid=gi, speed_at_floor=float(speedup(a, max(1, int(np.floor(gc))), c)), speed_grid=max(grid)))
out["lambert"] = rows
# Jensen: alpha ~ Beta with mean 0.7 and varying concentration; gamma=5, c=0.05
rng = np.random.default_rng(0); jr = []
for conc in [1e6, 20, 5, 2]:
    m = 0.7; a_ = rng.beta(m * conc, (1 - m) * conc, size=400000)
    g = 5
    jr.append(dict(conc=conc, mean_alpha=float(a_.mean()), E_het=float(E_tokens(a_, g).mean()), E_iid=float(E_tokens(a_.mean(), g)),
                   speed_het=float((E_tokens(a_, g).mean()) / (g * 0.05 + 1)), speed_iid=float(speedup(a_.mean(), g, 0.05))))
out["jensen"] = jr
# variance of emitted tokens per round
def var_tokens(a, g):
    k = np.arange(1, g + 1)
    EM = (a ** k).sum(); EM2 = ((2 * k - 1) * a ** k).sum()
    return float(EM2 - EM ** 2)
out["var"] = [dict(alpha=a, gamma=g, mean=float(E_tokens(a, g)), sd=float(np.sqrt(var_tokens(a, g)))) for a in [0.6, 0.8, 0.9] for g in [3, 5, 8]]
# P(all accepted) for tail
out["p_full"] = [dict(alpha=a, gamma=g, p=float(a ** g)) for a in [0.7, 0.8, 0.9] for g in [3, 5, 8]]
dump("exp9_extras", out)
for r in rows[:6]: print(r)
for r in jr: print(r)
print(out["var"][:4])
```

B.13 exp10_vocab.py

```
"""Exp 10: toy illustration of why a static reduced vocabulary caps acceptance and a
context-aware subset needs fewer candidates. TOY distributions, not model outputs.
Bound used: for a draft q_K supported on K, alpha = sum_x min(p,q_K) <= p(K)."""
import numpy as np
from common import *
rng = np.random.default_rng(10)
V, T, ctxs = 32000, 40, 400
glob = 1.0 / np.arange(1, V + 1) ** 1.05; glob /= glob.sum()          # global Zipf prior
# each "topic" boosts a random set of 600 tokens (drawn from the whole vocabulary, biased to the tail)
topics = []
for t in range(T):
    ids = rng.choice(V, size=600, replace=False)
    w = np.ones(V); w[ids] = 40.0
    topics.append(w)
def make_ctx():
    t = rng.integers(T); w = topics[t] * glob
    # context-level sharpening: lognormal noise on the topic distribution
    w = w * np.exp(rng.normal(0, 0.6, V)); p = w ** 1.3; return p / p.sum()
P = np.stack([make_ctx() for _ in range(ctxs)])
freq = P.mean(0); order = np.argsort(-freq)                           # static frequency ranking
rows = []
for k in [512, 2048, 8192, 32000]:
    stat = P[:, order[:k]].sum(1)                                     # static subset coverage p(K)
    orac = np.sort(P, axis=1)[:, ::-1][:, :k].sum(1)                  # per-context oracle top-k
    # imperfect ranker: top-k of log p + gaussian noise
    noisy = []
    for i in range(0, ctxs, 4):
        s = np.log(P[i]) + rng.normal(0, 1.0, V); idx = np.argpartition(-s, k - 1)[:k] if k < V else np.
    arange(V)
        noisy.append(P[i, idx].sum())
    rows.append(dict(k=k, static=float(stat.mean()), oracle=float(orac.mean()), noisy=float(np.mean(noisy)
    ),
                     static_p10=float(np.percentile(stat, 10))))
    print(rows[-1])
dump("exp10_vocab", rows)
# LM-head arithmetic (derivation): parameters read per drafted token
d = 4096; Vt = 128256
cost = dict(full=d * Vt / 1e6, k32768=d * 32768 / 1e6, k2048=d * 2048 / 1e6)
print(cost); dump("exp10_cost", cost)
```

B.14 exp11_ngram.py

```
"""Exp 11: prompt-lookup (n-gram) drafting simulated on REAL text, with the text itself playing the
role of a deterministic 'target'. This measures how copy-able the text is; it is not an LLM
measurement. Tokens are regex word/punctuation pieces (leading whitespace attached), not BPE.
Cases: (a) code edit  = prompt: original stdlib module, target text: module after a mechanical identifier
    rename
       (b) code from scratch = no prompt, target text: the module itself (self-repetition only)
       (c) prose = no prompt, target text: body of a research-paper text (self-repetition only)"""
import re, glob, os, sys, collections, keyword
import numpy as np
from common import *
TOK = re.compile(r"\s*\w+|\s*[^\w\s]|\s+")
def toks(s): return TOK.findall(s)
def lookup_round(ctx, gen, i, K, nmin, nmax, last):
    e = len(ctx)
    for n in range(nmax, nmin - 1, -1):
        if e < n: continue
        j = last.get(tuple(ctx[e - n:e]))
        if j is not None:
            draft = ctx[j + 1: j + 1 + K]
```

```
            # draft may run into the current position: allow it only up to e
            return draft
    return []
def run(prompt, target, K=5, nmin=2, nmax=7):
    ctx = list(prompt); i = 0; ins = 0; last = {}
    rounds = 0; toks_out = 0; drafted = 0; acc_when_draft = 0; nd = 0
    def insert_upto(limit):  # insert keys ending at indices <= limit
        nonlocal ins
        while ins <= limit:
            for n in range(nmin, nmax + 1):
                if ins + 1 >= n: last[tuple(ctx[ins - n + 1: ins + 1])] = ins
            ins += 1
    while i < len(target):
        insert_upto(len(ctx) - 2)
        d = lookup_round(ctx, target, i, K, nmin, nmax, last)
        a = 0
        while a < len(d) and i + a < len(target) and d[a] == target[i + a]: a += 1
        n_new = min(a + 1, len(target) - i)
        ctx.extend(target[i:i + n_new]); i += n_new
        rounds += 1; toks_out += n_new
        if d: nd += 1; acc_when_draft += a
    return dict(EN=toks_out / rounds, draft_rate=nd / rounds, acc_given_draft=(acc_when_draft / nd if nd
     else 0.0), rounds=rounds, tokens=toks_out)
libdir = os.path.dirname(os.__file__)
mods = [f for f in sorted(glob.glob(libdir + "/*.py")) if 300 < sum(1 for _ in open(f, errors="ignore")) <
      1500][:60]
def rename(src):
    ids = collections.Counter(w for w in re.findall(r"[A-Za-z_]\w{5,}", src) if not keyword.iskeyword(w))
    if not ids: return None
    w, c = ids.most_common(1)[0]
    return re.sub(r"\b%s\b" % re.escape(w), w + "_v2", src)
papers = sorted(glob.glob("/tmp/claude-0/-home-claude/edf7f09b-fbfa-5fa7-86ee-2b925800d2e1/scratchpad/sd/*.
     raw.txt"))
res = {}
for K in (3, 5, 10):
    ce, cs, pr = [], [], []
    for f in mods:
        src = open(f, errors="ignore").read(); new = rename(src)
        if new is None: continue
        ce.append(run(toks(src), toks(new)[:6000], K)); cs.append(run([], toks(src)[:6000], K))
    for f in papers:
        t = toks(open(f, errors="ignore").read()); pr.append(run([], t[500:6500], K))
    def agg(L): return dict(EN=float(np.mean([x["EN"] for x in L])), draft_rate=float(np.mean([x["
     draft_rate"] for x in L])),
                            acc_given_draft=float(np.mean([x["acc_given_draft"] for x in L])), n=len(L),
                            EN_min=float(np.min([x["EN"] for x in L])), EN_max=float(np.max([x["EN"] for x
      in L])))
    res[K] = dict(code_edit=agg(ce), code_scratch=agg(cs), prose=agg(pr))
    print(K, res[K])
dump("exp11_ngram", res)
```

B.15 exp12_substitutes.py

```
"""Exp 12: are parallel drafting (P-EAGLE-style) and SSD complements or substitutes? Closed-form model (
     derivation, not measurement).
Drafter step cost c1 (relative to a target step). Sequential drafter: T = gamma*c1. Parallel drafter: T =
     kappa*c1.
SD speedup = E/(1+T). SSD (same drafter as backup, T<1): E/(1+T-p*T)."""
from common import *
alpha, gamma = 0.8, 5
E = float(E_tokens(alpha, gamma))
rows = []
for c1 in (0.04, 0.08, 0.15):
    for name, T in (("sequential", gamma * c1), ("parallel kappa=2", 2 * c1)):
```

```
        sd = E / (1 + T)
        r = dict(c1=c1, drafter=name, T=T, sd=sd)
        for p in (0.6, 0.9):
            r[f"ssd_p{p}"] = E / (1 + T - p * T)
        r["ceiling"] = E
        rows.append(r)
for r in rows: print(r)
dump("exp12_substitutes", dict(E=E, rows=rows))
```

Appendix C

Glossary and Notation

Notation

p, q	Target and draft next-token distributions.
α	Acceptance probability per draft token ($\sum_x \min(p, q)$ for one position).
γ (or k)	Number of draft tokens per round.
c	Cost of one draft forward pass relative to one target forward pass.
$E[N]$	Expected tokens emitted per round, $(1 - \alpha^{\gamma+1})/(1 - \alpha)$.
S	Speedup over autoregressive decoding, $E[N]/(\gamma c + 1)$.
W_{-1}	Lower real branch of the Lambert W function.
T_{SD}	Ratio of drafting time to verification time in plain speculative decoding.
p_b, p_p	Cache hit probabilities used in the SSD model (Chapter 11).

Evidence tags

[S] Source-reported: stated in a paper or page we retrieved and read.
[D] Derivation: follows from formulas in this report, not measured.
[M] Our simulation: produced by scripts in Appendix B; toy or modelled.
[V] Vendor, blog or anecdotal claim, not independently verified.

Terms

Acceptance length
Mean number of tokens emitted per target verification pass (draft tokens accepted plus the one extra token).

Autoregressive (AR) decoding
Generating one token per target forward pass.

Bonus token
The extra token sampled from the target distribution when all draft tokens are accepted.

Chain draft
A single sequence of draft tokens, as opposed to a tree.

Draft tree
A tree of candidate continuations verified in one pass with a tree attention mask.

Drafter
Any mechanism that proposes tokens: a small model, a feature-conditioned head (EAGLE), a lookup (n-gram), or a native MTP head.

EAGLE / EAGLE-3
Feature-level drafters that condition on the target's hidden states; EAGLE-3 fuses features from several layers and is trained with a training-time test.

Goodput
Useful tokens per second, counting only tokens that are emitted, under a latency constraint.

Memory-bound / compute-bound
Whether step time is limited by moving weights and KV cache or by arithmetic throughput.

MTP
Multi-token prediction: a training objective or auxiliary head that predicts several future tokens.

Parallel drafting
Producing several draft tokens in one drafter forward pass.

Prompt lookup / n-gram drafting
Proposing tokens by matching the recent suffix against earlier text and copying what followed.

Residual distribution
$(p - q)_+$ normalised, used to resample after a rejection.

Ridge point
Arithmetic intensity at which a kernel moves from memory-bound to compute-bound on a roofline.

SSD / Saguaro
Speculative speculative decoding: the drafter speculates on the verification outcome and prepares drafts in advance.

Verification
The target forward pass that scores draft tokens and applies the accept/reject rule.

Bibliography

[1] Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost parallel draft model adaptation. *arXiv preprint arXiv:2504.18583*, 2025.

[2] Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024.

[3] Nikhil Bhendawade, Kumari Nishu, Arnav Kundu, Chris Bartels, Minsik Cho, and Irina Belousova. Mirror speculative decoding: Breaking the serial barrier in LLM inference. *arXiv preprint arXiv:2510.13161*, 2025.

[4] Amirhossein Bozorgkhoo and Igor Molybog. Speculative decoding scaling laws (SDSL). *arXiv preprint arXiv:2603.11053*, 2026.

[5] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *ICML*, 2024. arXiv:2401.10774.

[6] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[7] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036 (v2)*, 2026.

[8] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable and robust speculative decoding. In *NeurIPS*, 2024. arXiv:2402.12374.

[9] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[10] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, et al. LayerSkip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*, 2024.

[11] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of LLM inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.

[12] Google. Gemma 4 multi-token prediction drafters (announced may 5, 2026). Read via secondary summary: rits.shanghai.nyu.edu/ai/gemma-4-gets-multi-token-prediction-drafters-3x-faster-inference-same-outputs/, 2026.

[13] Google Cloud Vertex AI team and LMSYS. EAGLE-3 on Vertex AI with SGLang. lmsys.org/blog/2025-12-01-eagle3-vertex, 2025.

[14] Hacker News commenters. "speculative speculative decoding (ssd)" (hacker news item 47242637; 61 points, 9 comments). news.ycombinator.com/item?id=47242637, read October 2026; date shown only as "7 months ago", 2026.

[15] Mude Hui, Xin Huang, Jaime Campos Salas, Yue Sun, Nathan Pemberton, Xiang Song, Ashish Khetan, and George Karypis. P-EAGLE: Parallel-drafting EAGLE with scalable training. *arXiv preprint arXiv:2602.01469*, 2026. NeurIPS 2026 poster.

[16] ICLR. Iclr 2026 poster page for "speculative speculative decoding". iclr.cc/virtual/2026/poster/10008711, 2026.

[17] Inception Labs. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.

[18] Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding. In *ICLR*, 2026. arXiv:2603.03251 (v3); code: github.com/tanishqkumar/ssd.

[19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[20] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.

[21] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *ICML*, 2024. arXiv:2401.15077.

[22] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *EMNLP*, 2024. arXiv:2406.16858.

[23] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840 (v3, April 2025)*, 2025.

[24] LinkedIn Engineering. Accelerating LLM inference with speculative decoding: lessons from LinkedIn's hiring assistant. Engineering blog (read via an aggregator mirror), 2026.

[25] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. PEARL: Parallel speculative decoding with adaptive draft length. In *ICLR*, 2025. arXiv:2408.11850.

[26] Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Online speculative decoding. *arXiv preprint arXiv:2310.07177*, 2023.

[27] Xiaoxuan Liu, Jiaxiang Yu, Jongseok Park, Ion Stoica, and Alvin Cheung. Speculative decoding: Performance or illusion? In *Proceedings of MLSys 2026 (oral)*, 2026. arXiv:2601.11580 (v2).

[28] Xiaoxuan Liu et al. Optimizing speculative decoding for serving large language models using goodput. *arXiv preprint arXiv:2406.14066*, 2024.

[29] LMSYS / SpecForge team. SpecBundle & SpecForge v0.2: Production-ready speculative decoding models and framework. lmsys.org/blog/2025-12-23-spec-bundle-phase-1, 2025.

[30] Shanmugapriyan M. Speculative speculative decoding explained. F22 Labs blog (secondary commentary with a small author reproduction), 2026.

[31] Benjamin Marie. EAGLE-3 speculators: when to use them. The Kaitchup (Substack), 9 December 2025; paywalled, only introduction read, 2025.

[32] Bradley McDanel, Steven Li, Sruthikesh Surineni, and Harshit Khaitan. MoE-Spec: Expert budgeting for efficient speculative decoding. *arXiv preprint arXiv:2602.16052*, 2026.

[33] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *ASPLOS*, 2024. arXiv:2305.09781.

[34] MLSys. Mlsys 2026 oral page for "speculative decoding: Performance or illusion?" (21 may 2026). mlsys.org/virtual/2026/oral/3782, 2026.

[35] Sebastian Mondragon. Speculative decoding slower than baseline: How to fix it. Particula Tech blog, 21 August 2026 (company blog; secondary synthesis of cited sources), 2026.

[36] NeurIPS. Neurips 2026 poster page for "p-eagle". neurips.cc/virtual/2026/poster/152869, 2026.

[37] NVIDIA. Boost llama 3.3 70b inference throughput 3x with NVIDIA TensorRT-LLM speculative decoding. NVIDIA Technical Blog, December 2024 measurements, 2024.

[38] NVIDIA. H100 tensor core GPU datasheet page. nvidia.com/en-us/data-center/h100, 2026.

[39] NVIDIA. TensorRT-LLM documentation: Speculative decoding (v1.2.0rc7). nvidia.github.io/TensorRT-LLM, 2026.

[40] Tian Pan. Speculative decoding in production: Free tokens and hidden traps. Blog post, tianpan.co, April 2026, 2026.

[41] Zongyue Qin, Raghavv Goel, Mukul Gagrani, Risheek Garrepalli, Mingu Lee, and Yizhou Sun. ConFu: Contemplate the future for better speculative sampling. *arXiv preprint arXiv:2603.08899 (v3, April 2026)*, 2026.

[42] Red Hat. Speculators: speculative decoding model library (red hat ai inference 3.5 documentation, technology preview). docs.redhat.com, 2026.

[43] Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian En-Hsu Yen, Avner May, Tianqi Chen, and Beidi Chen. MagicDec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. In *ICLR*, 2025. arXiv:2408.11049; author list as recalled, not re-verified.

[44] Yuhao Shen, Junyi Shen, Quan Kong, Tianyu Liu, Yao Lu, and Cong Wang. Speculative decoding via hybrid drafting and rollback-aware branch parallelism (SpecBranch). In *ICLR*, 2026. arXiv:2506.01979.

[45] SpecForge authors. SpecForge: A flexible and efficient open-source training framework for speculative decoding. arXiv:2603.18567 (ICML 2026 poster listing); abstract-level only, 2026.

[46] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[47] SwiftSpec authors. SwiftSpec: Ultra-low latency LLM decoding by scaling asynchronous and disaggregated speculative decoding. *arXiv preprint arXiv:2506.11309*, 2025. ByteDance; author list not verified here.

[48] Bangsheng Tang et al. Efficient speculative decoding for Llama at scale: Challenges and solutions. *arXiv preprint arXiv:2508.08192*, 2025.

[49] Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Oren Pereg, Moshe Wasserblat, Tomer Galanti, Michal Gordon, and David Harel. Distributed speculative inference (DSI): Speculation parallelism for provably faster lossless language model inference. In *ICLR*, 2025. arXiv:2405.14105.

[50] Together AI. ATLAS: AdapTive-LeArning speculator system. Blog post, together.ai/blog/adaptive-learning-speculator-system-atlas, 2025.

[51] u/jacek2023 and commenters. "eagle3 has landed in llama.cpp" (r/localllama thread, snapshot of 14 of 44 comments). Via reddit.sentinel-team.org snapshot, June 2026, 2026.

[52] vLLM Project. Speculative decoding (documentation). docs.vllm.ai/en/latest/features/speculative_decoding/, accessed October 2026, 2026.

[53] vLLM Project. P-EAGLE: Faster LLM inference with parallel speculative decoding in vLLM. Blog post, vllm.ai/blog/p-eagle (March 2026), 2026.

[54] vLLM Project and AMD. EAGLE-3 speculative decoding on AMD instinct GPUs: Training and serving with vLLM and AMD Quark. Blog post, vllm.ai/blog/2026-07-13-eagle-3-amd-instinct, 2026.

[55] Xin Wang, Ziming Miao, Yi Zhu, Hui Shen, Zhongwei Wan, Fan Yang, and Mi Zhang. AgentSpec: Speculative decoding for batch inference of LLM agents. *arXiv preprint arXiv:2608.24004*, 2026.

[56] Miles Williams, Young D. Kwon, Rui Li, Alexandros Kouris, and Stylianos I. Venieris. Speculative decoding with a speculative vocabulary. *arXiv preprint arXiv:2602.13836 (v2, July 2026)*, 2026.

[57] Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of ACL*, 2024. arXiv:2401.07851; introduces Spec-Bench.

[58] Tong Yuan. Efficient inference with speculative decoding (phd qualifying examination announcement). HKUST(GZ) DSA Thrust, 10 June 2026; dsa.hkust-gz.edu.cn, 2026.

[59] Weilin Zhao, Tengyu Pan, Xu Han, Yudi Zhang, Ao Sun, Yuxiang Huang, Kaihuo Zhang, Weilun Zhao, Yuxuan Li, Jianyong Wang, Zhiyuan Liu, and Maosong Sun. FR-Spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. *arXiv preprint arXiv:2502.14856*, 2025.